C++

CHEAT SHEETS

A Complete Reference for Modern C++

Types & Strings · Control Flow · Functions & Lambdas · STL Containers · OOP · Move Semantics · Pointers & Memory · Templates & Concepts · Ranges & Algorithms · Concurrency · Modules & Build · C++26 Preview

123 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

001–005 Introduction & Setup
006–009 Input, Output & Comments
010–017 Variables & Data Types
018–023 Strings
024–028 Operators
029–034 Control Flow

CORE LANGUAGE

035–039 Error Handling
040–048 Functions
049–059 Data Structures: STL Containers
060–074 OOP
075–079 Files & I/O
080–086 Headers, Modules & Standard Library

SYSTEMS & GENERIC

087–090 External Libraries
091–100 Advanced Topics
101–104 Pointers & References
105–109 Memory Management
110–116 Templates & Generic Programming
117–123 STL Iterators, Algorithms & Ranges

CPP

Table of Contents

123 sheets · syntax · examples · notes

INTRODUCTION & SETUP 001 – 005

001 Introduction & Stand…
002 Compilers & Commo…
003 Program Structure
004 Compilation Model & …
005 CMake Basics

INPUT, OUTPUT & COMMENTS 006 – 009

006 Output with iostream
007 std::print & std::format
008 Console Input
009 Comments

VARIABLES & DATA TYPES 010 – 017

010 Variables & Initializati…
011 Integer Types
012 Floating-Point Types
013 bool & Character Typ…
014 auto, decltype & Typ…
015 Constants: const, co…
016 Conversions & Casts
017 Enumerations

STRINGS 018 – 023

018 C-Style Strings
019 std::string Basics
020 std::string — Search
021 std::string — Modify …
022 std::string_view
023 String Conversions

OPERATORS 024 – 028

024 Arithmetic Operators
025 Comparison & Three-…
026 Logical Operators
027 Bitwise Operators
028 Assignment & Other …

CONTROL FLOW 029 – 034

029 if / else & Conditional…
030 switch
031 for Loops
032 Range-Based for
033 while & do-while
034 Loop Controls & goto

ERROR HANDLING 035 – 039

035 Exception Basics
036 Standard Exception H…
037 Custom Exceptions &…
038 std::optional & std::ex…
039 Assertions & Error Co…

FUNCTIONS 040 – 048

040 Defining & Calling Fun…
041 Parameter Passing
042 Overloading & Defaul…
043 Linkage, inline & static
044 constexpr & constev…
045 Lambdas — Basics
046 Lambdas — Advanced
047 Callables & std::functi…
048 Attributes & Specifiers

DATA STRUCTURES: STL CONTAINERS 049 – 059

049 C-Style Arrays & std::…
050 std::vector — Basics
051 std::vector — Operati…
052 std::pair & std::tuple
053 std::map & std::multi…
054 std::unordered_map
055 Sets
056 std::deque & std::list
057 Stacks, Queues & Pri…
058 std::span & std::mdsp…
059 Choosing a Container

OOP 060 – 074

060 Classes & Objects
061 Constructors
062 Destructors
063 Access Control & frie…
064 const & Qualified Me…
065 Static Members
066 Aggregates & Design…
067 Copy Semantics
068 Move Semantics
069 Inheritance
070 Virtual Functions & P…
071 Abstract Classes & In…
072 Multiple & Virtual Inh…
073 Operator Overloading…
074 Operator Overloading…

FILES & I/O 075 – 079

075 Reading Files
076 Writing & Appending …
077 Binary Files
078 std::filesystem
079 Parsing Text & CSV

HEADERS, MODULES & STANDARD LIBRARY 080 – 086

080 Headers & Include Gu…
081 The Preprocessor
082 Namespaces
083 C++20 Modules
084 Package Managers
085 Math & Random
086 Dates, Times & std::c…

EXTERNAL LIBRARIES 087 – 090

087 GoogleTest
088 Catch2
089 {fmt} & spdlog
090 nlohmann/json

ADVANCED TOPICS 091 – 100

091 Threads
092 Mutexes & Locks
093 Condition Variables &…
094 Atomics & the Memo…
095 Futures & std::async
096 Coroutines
097 std::variant & std::any
098 Regular Expressions
099 UB, Sanitizers & Debu…
100 C++26 Preview

POINTERS & REFERENCES 101 – 104

101 Pointer Basics
102 Pointer Arithmetic & …
103 References
104 Value Categories & F…

MEMORY MANAGEMENT 105 – 109

105 Stack, Heap & Storag…
106 new & delete
107 std::unique_ptr
108 std::shared_ptr & std…
109 RAII & Ownership Gui…

TEMPLATES & GENERIC PROGRAMMING 110 – 116

110 Function Templates
111 Class Templates
112 Non-Type Parameter…
113 Variadic Templates & …
114 Concepts & requires
115 Type Traits & SFINAE
116 Template Idioms

STL ITERATORS, ALGORITHMS & RANGES 117 – 123

117 Iterators
118 Algorithms — Non-M…
119 Algorithms — Modifyi…
120 Sorting & Searching
121 Numeric Algorithms
122 Ranges — Basics
123 Range Views & Adapt…

Sheet 001

Introduction & Standards

Bjarne Stroustrup · C with Classes · C++98 → C++26 · ISO WG21 · zero-overhead principle · C++ Core Guidelines

WHAT C++ IS

C++ is a compiled, statically typed, multi-paradigm language created by **Bjarne Stroustrup** at Bell Labs, starting in 1979 as "C with Classes". It adds classes, templates, exceptions, and a large standard library to C while keeping C's direct access to memory and hardware. The guiding rule is the **zero-overhead principle**: you don't pay for what you don't use, and what you do use is as fast as hand-written code. The language is standardized by **ISO WG21**, which ships a new standard every three years.

AT A GLANCE

- Files: `.cpp` / `.cc` / `.cxx` for source, `.h` / `.hpp` for headers, `.cppm` / `.ixx` for modules
- Used for games, browsers, databases, trading systems, embedded, compilers, and HPC
- Compiled ahead of time to native machine code; there is no VM or garbage collector
- The **C++ Core Guidelines** (Stroustrup & Sutter) are the de facto style rulebook

HELLO, WORLD (C++23)

```cpp
#include <print>                  // C++23: std::print / std::println

int main() {
    std::println("Hello, World!");   // Hello, World!
}

// Pre-C++23 equivalent you'll see everywhere:
#include <iostream>

int main() {
    std::cout << "Hello, World!\n";  // Hello, World!
    return 0;
}

// $ g++ -std=c++23 hello.cpp -o hello && ./hello
```

CODE EXPLAINED

- → `#include <print>` — pulls in the standard header that declares `std::println`
- → `int main()` — program entry point; returning from it ends the program
- → `std::` — every standard library name lives in the `std` namespace
- → `return 0;` — optional in `main` only; falling off the end returns 0
- → `-std=c++23` — tells the compiler which standard to compile against

STANDARDS TIMELINE

STANDARD	HEADLINE FEATURES
C++98 / 03	STL, templates, exceptions, `std::string`
C++11	`auto`, lambdas, move semantics, smart pointers, threads
C++14	generic lambdas, `make_unique`, relaxed `constexpr`
C++17	`optional`, `variant`, structured bindings, `filesystem`
C++20	concepts, ranges, coroutines, modules, <=>
C++23	`std::print`, `expected`, `import std;`, deducing `this`
C++26	reflection, contracts, `std::execution`, `#embed`

KEY TERMS

TERM	MEANING
WG21	ISO committee that writes the standard
TS	Technical Specification (experimental feature track)
UB	undefined behavior: anything can happen
STL	the containers/algorithms part of the std library
ABI	binary interface between compiled pieces
cppreference	the go-to online reference site

⚠ COMMON MISTAKES

- Treating C++ as "C with classes": modern C++ avoids raw `new`, C arrays, and macros in everyday code
- Learning from pre-2011 tutorials; they teach manual memory management that smart pointers replaced
- Assuming every compiler supports the newest standard fully; check the support tables first

✓ TIPS & BEST PRACTICES

- Bookmark cppreference.com; each page marks which standard added a feature (e.g. "since C++20")
- Try snippets on Compiler Explorer (godbolt.org) to compare compilers and see the generated assembly
- Turn on warnings from day one: `-Wall -Wextra` catches many beginner bugs

QUICK SUMMARY

- `C++` — compiled, multi-paradigm, zero-overhead systems language
- `WG21` — ISO committee; new standard every 3 years
- `C++23` — current baseline for this set; C++26 flagged where relevant
- `std::println` — the modern Hello, World

Sheet 002

Compilers & Common Flags

g++ · clang++ · MSVC cl · -std=c++23 · -Wall -Wextra -Werror · -O0 / -O2 / -O3 · -g · Compiler Explorer

THE BIG THREE COMPILERS

Three compilers dominate: **GCC** (g++), **Clang** (clang++, also the default on macOS via Xcode), and **MSVC** (cl.exe, part of Visual Studio). All three implement most of C++23, but library support lags the language, so check cppreference's compiler support page for newer features. Flags pick the **standard**, the **warning level**, the **optimization level**, and whether to emit **debug info**.

INSTALL

- Linux: `sudo apt install g++ clang` (or your distro's equivalent)
- macOS: `xcode-select --install` (Clang) or `brew install gcc llvm`
- Windows: Visual Studio "Desktop development with C++", or MSYS2 for GCC
- Verify: `g++ --version`, `clang++ --version`, `cl` (from a Developer Prompt)

TYPICAL COMPILE COMMANDS

```
# Debug build: all warnings, debug symbols, no optimization
g++ -std=c++23 -Wall -Wextra -Wpedantic -g -O0 main.cpp -o app

# Release build: optimized, warnings are errors
clang++ -std=c++23 -Wall -Wextra -Werror -O2 -DNDEBUG main.cpp -o app

# Multiple files + include dir + link a library
g++ -std=c++23 -Iinclude src/*.cpp -Llib -lfmt -o app

# Runtime bug hunting with sanitizers
g++ -std=c++23 -g -fsanitize=address,undefined main.cpp -o app

# MSVC equivalent of a debug build
cl /std:c++latest /W4 /Zi /EHsc main.cpp
```

CODE EXPLAINED

- → `-std=c++23` — language standard (`c++20`, `c++26` / `c++2c` also exist)
- → `-Wall -Wextra` — enable the common warning sets; `-Wpedantic` flags non-standard extensions
- → `-O0` / `-O2` — no optimization vs. full safe optimization
- → `-g` — embed debug info for gdb / lldb
- → `-I` / `-L` / `-l` — include path, library path, library name
- → `/EHsc` — MSVC's standard exception handling model

OPTIMIZATION LEVELS

FLAG	USE
`-O0`	default; fastest compile, easiest debugging
`-Og`	debug-friendly optimization (GCC)
`-O1`	basic optimization
`-O2`	standard release level
`-O3`	aggressive (vectorizing, inlining)
`-Os / -Oz`	optimize for size

WARNINGS & DEFINES

FLAG	EFFECT
`-Wall -Wextra`	broad warning coverage
`-Wshadow`	variable hides another
`-Wconversion`	lossy implicit conversions
`-Werror`	treat warnings as errors
`-DNAME=val`	define a macro
`-DNDEBUG`	disable `assert()`

COMPILER MAP

GCC / CLANG	MSVC
`-std=c++23`	/std:c++latest
`-Wall`	/W4
`-O2`	/O2
`-g`	/Zi
`-o app`	/Fe:app.exe
`-DX`	/DX

⚠ COMMON MISTAKES

- Forgetting `-std=c++20` / `23` on older GCC, so new features fail with confusing errors
- Shipping `-O0` builds; unoptimized C++ can be 10× slower than `-O2`
- Ignoring warnings; `-Wall` warnings like "unused variable" or "comparison of signed and unsigned" often point to real bugs

✓ TIPS & BEST PRACTICES

- Keep two build configs: debug (`-g -O0 -fsanitize=…`) and release (`-O2 -DNDEBUG`)
- Add `-Werror` in CI so new warnings break the build instead of piling up
- Paste code into Compiler Explorer to check a feature on GCC, Clang, and MSVC at once

QUICK SUMMARY

- `g++ / clang++ / cl` — the three major compilers
- `-std=c++23` — choose the language standard
- `-Wall -Wextra -Werror` — warnings on, and fatal in CI
- `-O2 -DNDEBUG` — typical release flags

Program Structure

ANATOMY OF A PROGRAM

Every C++ program needs exactly one `main` function, which the runtime calls after initializing globals. Code is organized into **statements** that end with semicolons, grouped into **blocks** with braces. Whitespace and indentation don't matter to the compiler. `#include` lines are processed by the **preprocessor** before compilation and paste in the declarations your code uses.

RULES

- `main` returns `int`; `0` (or `EXIT_SUCCESS`) means success
- Two legal forms: `int main()` and `int main(int argc, char* argv[])`
- `argv[0]` is the program name; real arguments start at `argv[1]`
- Names are case-sensitive: `Count` and `count` are different

ANNOTATED PROGRAM

```cpp
#include <print>        // std::println
#include <string>       // std::string
#include <cstdlib>      // EXIT_SUCCESS / EXIT_FAILURE
// A function must be declared before it is used
int square(int x) {
    return x * x;
}
int main(int argc, char* argv[]) {      // entry point
    if (argc < 2) {                     // no argument given
        std::println("usage: {} NAME", argv[0]);
        return EXIT_FAILURE;            // non-zero = error
    }
    std::string name = argv[1];         // first real argument
    std::println("Hi {}, 7^2 = {}", name, square(7));
    return EXIT_SUCCESS;                // 0
}
// $ ./app Ada   →   Hi Ada, 7^2 = 49
```

CODE EXPLAINED

- → `#include <…>` — standard headers use angle brackets; your own use quotes
- → `int square(int x)` — return type, name, typed parameter list
- → `argc` / `argv` — count and array of C-string arguments
- → `{ … }` — a block; also creates a new scope for variables
- → `;` — ends every statement (but not a function body's closing brace)

BUILDING BLOCKS

ELEMENT	EXAMPLE
Preprocessor directive	`#include <vector>`
Declaration	`int count = 0;`
Expression statement	`count += 1;`
Compound statement	`{ int y = 2; }`
Function definition	`void log(std::string_view m) { … }`
Namespace qualifier	`std::vector<int>`

EXIT CODES & TERMINATION

CALL	EFFECT
`return 0;`	normal exit from `main`
`return EXIT_FAILURE;`	portable failure code
`std::exit(n)`	exit now; runs static destructors
`std::quick_exit(n)`	exit without static cleanup
`std::abort()`	abnormal termination, no cleanup
`std::terminate()`	called on uncaught exception

⚠ COMMON MISTAKES

- Writing `void main()`; it's not standard C++ and some compilers reject it
- Calling a function above the point where it's declared; add a prototype or move the definition up
- Reading `argv[1]` without checking `argc` first, which is undefined behavior when no argument was passed

✓ TIPS & BEST PRACTICES

- Use `EXIT_SUCCESS` / `EXIT_FAILURE` from `<cstdlib>` for readable, portable exit codes
- Convert `argv` to `std::vector<std::string_view> args(argv, argv + argc);` to work with it safely
- Keep `main` short: parse arguments, call into real functions, return a status

QUICK SUMMARY

- `int main()` — single required entry point
- `argc / argv` — command-line argument count and values
- `#include` — preprocessor pastes in declarations
- `;` and `{}` — statements and blocks; whitespace is free-form

Sheet 004

Compilation Model & Linking

preprocess → compile → link · translation units · object files · ODR · static .a/.lib vs shared .so/.dll · "undefined reference"

FROM SOURCE TO EXECUTABLE

Building C++ is a three-stage pipeline. The **preprocessor** expands `#include` and macros, producing a **translation unit** (TU). The **compiler** turns each TU into an **object file** independently. The **linker** combines object files and libraries into one executable, resolving every name used in one TU to its definition in another. Most confusing build errors make sense once you know which stage produced them.

THE ODR

- **One Definition Rule**: each function/variable has exactly one definition program-wide
- Declarations (in headers) may repeat; definitions (in `.cpp`) may not
- `inline` functions and templates may be defined in headers: once per TU, all identical
- Breaking the ODR is often silent UB, not a compile error

BUILDING IN STAGES

```
g++ -E main.cpp -o main.ii      # 1. preprocess only
g++ -c main.cpp -o main.o       # 2. compile one TU → object file
g++ -c math.cpp -o math.o
g++ main.o math.o -o app        # 3. link objects → executable

# Static library: archive of .o files, copied into the app
ar rcs libmath.a math.o
g++ main.o -L. -lmath -o app

# Shared library: loaded at run time
g++ -fPIC -shared math.cpp -o libmath.so
g++ main.o -L. -lmath -Wl,-rpath,. -o app

nm -C math.o                    # list symbols in an object file
```

CODE EXPLAINED

- → `-E` — stop after preprocessing; shows what the compiler really sees
- → `-c` — compile without linking; one `.o` per `.cpp`
- → `ar rcs` — bundle objects into a static `.a` archive
- → `-fPIC -shared` — build position-independent shared library
- → `-Wl,-rpath,.` — tell the loader where to find `.so` at run time
- → `nm -C` — dump (demangled) symbols to debug link errors

WHICH STAGE FAILED?

ERROR	STAGE & USUAL CAUSE
`fatal error: x.h: No such file`	preprocessor; missing –I path
`expected ';' before …`	compiler; syntax error
`'foo' was not declared`	compiler; missing `#include`/prototype
`undefined reference to foo`	linker; definition not compiled/linked
`multiple definition of foo`	linker; definition in a header
`cannot open shared object`	loader at run time; rpath / PATH

LIBRARY TYPES

KIND	LINUX / MACOS	WINDOWS
`Object file`	.o	.obj
`Static lib`	.a	.lib
`Shared lib`	.so / .dylib	.dll (+ import .lib)
`Executable`	(no ext)	.exe

⚠ COMMON MISTAKES

- Putting a non-`inline` function definition in a header included by two `.cpp` files, which causes "multiple definition"
- Compiling only `main.cpp` and getting "undefined reference" because `math.cpp` was never passed to the build
- Listing libraries before the objects that use them (`-lmath main.o`); GNU ld resolves left to right

✓ TIPS & BEST PRACTICES

- Read the first error only; later errors are often fallout from it
- Use `extern "C"` when linking against C libraries to stop C++ name mangling
- Let a build system (CMake) track which TUs need recompiling instead of hand-written commands

QUICK SUMMARY

- `preprocess → compile → link` — the three build stages
- `translation unit` — one `.cpp` after preprocessing
- `ODR` — one definition per entity across the program
- `undefined reference` — linker error: missing definition

CMake Basics

WHY CMAKE

CMake is the de facto build system generator for C++. You describe **targets** (executables and libraries) and their properties in `CMakeLists.txt`; CMake generates Ninja, Makefile, Visual Studio, or Xcode projects from it. Modern CMake is **target-based**: settings are attached to a target with `target_*` commands and flow to whatever links it, instead of global variables.

WORKFLOW

- Configure once: `cmake -B build -G Ninja`
- Build: `cmake --build build` (works with any generator)
- Out-of-source builds keep generated files in `build/`
- `PRIVATE` / `PUBLIC` / `INTERFACE` control what propagates to dependents

CMAKELISTS.TXT

```
cmake_minimum_required(VERSION 3.28)
project(Inventory VERSION 1.0 LANGUAGES CXX)

# A library target with its public headers
add_library(core src/item.cpp src/store.cpp)
target_include_directories(core PUBLIC include)
target_compile_features(core PUBLIC cxx_std_23)

# The executable links the library
add_executable(inventory src/main.cpp)
target_link_libraries(inventory PRIVATE core)

# Warnings, per compiler
if(MSVC)
  target_compile_options(core PRIVATE /W4)
else()
  target_compile_options(core PRIVATE -Wall -Wextra)
endif()
```

CODE EXPLAINED

- → `cmake_minimum_required` — first line; pins policy behavior
- → `project(... LANGUAGES CXX)` — names the project, enables C++
- → `add_library` / `add_executable` — define targets from sources
- → `target_compile_features(... cxx_std_23)` — require C++23
- → `PUBLIC` include dir — also applied to anything linking `core`
- → `target_link_libraries` — link and inherit usage requirements

COMMAND LINE

COMMAND	PURPOSE
`cmake -B build`	configure into `build/`
`-DCMAKE_BUILD_TYPE=Release`	single-config build type
`cmake --build build -j`	build in parallel
`--build build --config Release`	multi-config (VS / Xcode)
`ctest --test-dir build`	run registered tests
`cmake --install build`	install targets

KEYWORD SCOPE

KEYWORD	APPLIES TO
`PRIVATE`	this target only
`PUBLIC`	this target + dependents
`INTERFACE`	dependents only (header-only libs)

⚠ COMMON MISTAKES

- Using global `include_directories()` / `add_compile_options()`; they leak into every target. Prefer `target_*`
- Setting `CMAKE_BUILD_TYPE` with Visual Studio or Xcode; those generators use `--config` at build time
- Building in the source folder, which mixes generated files with your code

✓ TIPS & BEST PRACTICES

- Set `CMAKE_EXPORT_COMPILE_COMMANDS=ON` so clangd and other tools understand your build
- Use `CMakePresets.json` to share configure/build options with your team
- Mark header-only libraries `add_library(name INTERFACE)`

QUICK SUMMARY

- `CMakeLists.txt` — declares targets and their properties
- `cmake -B build` — configure; `cmake --build build` — build
- `target_link_libraries` — link and propagate usage requirements
- `cxx_std_23` — request the standard per target

Output with iostream

STREAM OUTPUT

`std::cout` is the standard output **stream** from `<iostream>`. The insertion operator `<<` sends values to it and returns the stream, so calls chain left to right. Formatting is controlled by **manipulators** from `<iomanip>`. Most of them are **sticky**: they stay in effect until changed. `std::cerr` writes to standard error and is unbuffered.

STREAMS

- `std::cout` — buffered stdout
- `std::cerr` — unbuffered stderr, for errors
- `std::clog` — buffered stderr, for logging
- `'\n'` ends a line; `std::endl` ends a line **and flushes** (slower in loops)

COUT & MANIPULATORS

```cpp
#include <iostream>
#include <iomanip>

int qty = 42; double price = 3.14159;
std::cout << "Qty: " << qty << '\n';              // Qty: 42
std::cout << std::fixed << std::setprecision(2)
          << price << '\n';                       // 3.14
std::cout << std::setw(6) << qty << "|\n";        //     42|
std::cout << std::left << std::setw(6) << qty
          << "|\n";                               // 42    |
std::cout << std::setfill('0') << std::right
          << std::setw(5) << 7 << '\n';           // 00007
std::cout << std::hex << 255 << std::dec << '\n'; // ff
std::cout << std::boolalpha << true << '\n';      // true
std::cerr << "error: file not found\n";           // → stderr
```

CODE EXPLAINED

- → `<<` — insertion; returns the stream so it can chain
- → `std::fixed` + `setprecision(2)` — 2 digits after the decimal point
- → `std::setw(6)` — width for the **next** item only (not sticky)
- → `std::left` / `std::right` — alignment within the width
- → `std::setfill('0')` — pad character (sticky)
- → `std::hex` — sticky; switch back with `std::dec`

MANIPULATORS

MANIPULATOR	EFFECT	STICKY?
`std::setw(n)`	field width	no
`std::setprecision(n)`	significant / decimal digits	yes
`std::fixed / scientific`	float notation	yes
`std::left / right`	alignment	yes
`std::setfill(c)`	pad character	yes
`std::hex / oct / dec`	integer base	yes
`std::showbase`	prefix `0x` / `0`	yes
`std::boolalpha`	`true` instead of 1	yes

ESCAPE SEQUENCES

ESCAPE	OUTPUT
`\n`	newline
`\t`	tab
`\\`	backslash
`\"`	double quote
`\0`	null character
`R"(...)"`	raw string; no escapes

⚠ COMMON MISTAKES

- Using `std::endl` in tight loops; each flush is a system call. Use `'\n'`
- Forgetting `std::hex` is sticky, so every later integer also prints in hex
- Expecting `setw` to persist; it resets after the next insertion

✓ TIPS & BEST PRACTICES

- Prefer `std::print` / `std::format` (next sheet) for new code: clearer and type-safe
- Call `std::ios::sync_with_stdio(false);` for faster iostream I/O when you don't mix in `printf`
- Save and restore format state with `std::ios state(nullptr); state.copyfmt(std::cout);`

QUICK SUMMARY

- `std::cout <<` — chained output to stdout
- `'\n'` — newline without flushing
- `<iomanip>` — `setw`, `setprecision`, `fixed`, `setfill`
- `std::cerr` — unbuffered error output

std::print & std::format

PYTHON-STYLE FORMATTING

C++20 added `std::format`, which builds a `std::string` from a format string with `{}` placeholders, checked at **compile time**. C++23 added `std::print` and `std::println` in <print>, which write formatted text straight to stdout (or a `FILE*`). The format-spec mini-language after a colon controls fill, alignment, width, precision, and type.

SPEC GRAMMAR

- `{[index]:[fill][align][sign][#][0][width][.precision][type]}`
- Align: `<` left, `>` right, `^` center
- Mismatched argument types fail to **compile**, not at run time
- Use `{{` and `}}` for literal braces

FORMAT / PRINT / PRINTLN

```
#include <format>
#include <print>

std::string s = std::format("{} + {} = {}", 2, 3, 5); // "2 + 3 = 5"
std::println("{:>8}|", "right");      //    right|
std::println("{:<8}|", "left");       // left    |
std::println("{:*^9}", "mid");        // ***mid***
std::println("{:.2f}", 3.14159);      // 3.14
std::println("{:08.3f}", -3.5);       // -003.500
std::println("{:+d} {:x} {:#b}", 7, 255, 5); // +7 ff 0b101
std::println("{1} before {0}", "a", "b");    // b before a
std::println("{:%Y-%m-%d}",
    std::chrono::system_clock::now());       // 2026-10-11
std::print(stderr, "warn: {}\n", 42);        // → stderr
std::vector v{1, 2, 3};
std::println("{}", v);                // [1, 2, 3]  (C++23)
```

CODE EXPLAINED

- → `std::format` — returns a `std::string`
- → `std::println` — prints and appends a newline
- → `{:>8}` — right-align in a field of 8
- → `{:.2f}` — fixed notation, 2 decimals
- → `{1}` / `{0}` — explicit argument indexes
- → Ranges and containers format directly in C++23

TYPE CHARACTERS

TYPE	MEANING	EXAMPLE → OUTPUT
d	decimal integer	{:d}, 42 → 42
x / X	hex	{:X}, 255 → FF
b	binary	{:b}, 5 → 101
o	octal	{:o}, 8 → 10
f	fixed float	{:.1f}, 2.25 → 2.2
e	scientific	{:.2e}, 1234.5 → 1.23e+03
g	shortest repr.	{:g}, 0.5 → 0.5
s / c	string / char	{:c}, 65 → A

RELATED APIS

API	PURPOSE
`std::format_to(it, …)`	write into an output iterator
`std::format_to_n(it, n, …)`	write at most n chars
`std::formatted_size(…)`	length without formatting
`std::vformat(fmt, args)`	runtime format string
`std::formatter<T>`	specialize for your types

⚠ COMMON MISTAKES

- Passing a runtime string as the format: `std::format(userStr)` fails to compile; use `std::vformat` with `std::make_format_args`
- Mixing automatic `{}` and manual `{0}` indexing in one string, which is an error
- Assuming `std::print` exists on older standard libraries; it needs GCC 14+, Clang/libc++ 18+, or MSVC 19.37+

✓ TIPS & BEST PRACTICES

- Use `{fmt}` (`fmt::print`) as a drop-in on older toolchains; `std::format` is based on it
- Specialize `std::formatter<Point>` once so `std::println("{}", p)` works everywhere
- Prefer `println` over `print("...\n")` ; it reads better

QUICK SUMMARY

- `std::format` — type-safe string building with `{}`
- `std::println` — formatted print plus newline (C++23)
- `{:>10.2f}` — fill/align, width, precision, type
- `std::formatter<T>` — make your own types formattable

Console Input

READING STDIN

`std::cin` reads from standard input. The extraction operator >> skips leading whitespace, parses one value of the target's type, and stops at the next whitespace. `std::getline` reads a whole line including spaces. When extraction fails, the stream enters a **fail state** and ignores every later read until you call `clear()` and discard the bad input.

STREAM STATE

- `if (std::cin >> x)` — true when the read succeeded
- `cin.fail()` — wrong type or EOF
- `cin.clear()` — reset the error flags
- `cin.ignore(n, '\n')` — throw away the rest of the line

SAFE INPUT LOOP

```cpp
#include <iostream>
#include <limits>
#include <string>

int age;
std::cout << "Age: ";
while (!(std::cin >> age) || age < 0) {            // typed "abc"?
    std::cin.clear();                              // reset fail flag
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::cout << "Enter a non-negative number: ";
}
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

std::string name;
std::cout << "Full name: ";
std::getline(std::cin, name);                      // "Ada Lovelace"

int a, b;
std::cin >> a >> b;                                // "3 4" → a=3, b=4
```

CODE EXPLAINED

- `!(std::cin >> age)` — extraction failed (not a number)
- `clear()` then `ignore(...)` — the standard recovery pair
- second `ignore` — drops the `'\n'` left by `>>` before `getline`
- `std::getline` — reads up to (and removes) the newline
- `>> a >> b` — chained reads; any whitespace separates values

INPUT FUNCTIONS

CALL	READS
`std::cin >> x`	one whitespace-delimited token
`std::getline(cin, s)`	whole line, no newline
`std::getline(cin, s, ',')`	up to a custom delimiter
`cin.get(ch)`	one char, including whitespace
`cin.peek()`	next char without consuming
`cin >> std::ws`	skip leading whitespace

STATE CHECKS

CALL	TRUE WHEN
`cin.good()`	no error flags set
`cin.fail()`	parse failed or EOF
`cin.eof()`	end of input reached
`cin.bad()`	unrecoverable stream error
`cin.clear()`	resets all flags

⚠ COMMON MISTAKES

- Calling `getline` right after `cin >> n`; it reads the leftover newline and returns an empty string
- Not checking the result of `>>`, so a bad value leaves `x` at 0 (C++11+) and the loop spins forever
- Using `while (!cin.eof())`; check the read itself: `while (cin >> x)`

✓ TIPS & BEST PRACTICES

- Read whole lines with `getline`, then parse with `std::from_chars` or `std::stringstream` for robust input
- Add `std::cin >> std::ws` before `getline` to skip leftover whitespace in one step
- `while (std::cin >> x)` processes all numbers until EOF (Ctrl+D / Ctrl+Z)

QUICK SUMMARY

- `std::cin >> x` — read one typed token
- `std::getline` — read a full line
- `clear() + ignore()` — recover from bad input
- `if (cin >> x)` — always test the read

Comments

COMMENT FORMS

C++ has two comment styles inherited from C and C99: `//` runs to the end of the line, and `/* … */` spans any number of lines. Comments are removed before compilation and never affect the program. **Doxygen**-style comments (`///` or `/** … */`) placed before a declaration are picked up by IDEs for hover text and by Doxygen to generate HTML/PDF API docs.

RULES

- `/* … */` comments do **not** nest
- `#if 0 … #endif` safely disables code that contains block comments
- Comments explain **why**; the code shows what
- IDE hover (VS Code, CLion, Visual Studio) reads `///` and `/** */`

COMMENT STYLES

```cpp
// Single-line comment
int retries = 3;   // trailing comment

/* Block comment
   spanning several lines */

/**
 * @brief  Compute the average of a list of scores.
 * @param  scores  Values to average; must not be empty.
 * @return The arithmetic mean.
 * @throws std::invalid_argument if scores is empty.
 */
double average(std::span<const double> scores);
/// Short doc comment for a single declaration.
int clamp_percent(int v);
#if 0   // disabled block: may contain /* comments */
legacy_code();
#endif
```

CODE EXPLAINED

- → `//` — everything to end of line is ignored
- → `/** … */` — Doxygen block attached to the next declaration
- → `@brief` / `@param` / `@return` — common Doxygen tags
- → `///` — one-line Doxygen comment
- → `#if 0` — preprocessor removes the block entirely

DOXYGEN TAGS

TAG	PURPOSE
`@brief`	one-line summary
`@param name`	describe a parameter
`@tparam T`	describe a template parameter
`@return`	describe the return value
`@throws / @exception`	exception that can escape
`@note / @warning`	highlighted remarks
`@see`	cross-reference
`@deprecated`	mark as obsolete

TOOLING

TOOL	WHAT IT DOES
`doxygen -g`	create a default `Doxyfile`
`doxygen Doxyfile`	generate HTML / LaTeX docs
`clang-format`	reflows comments to column limit
`-Wcomment`	warns on `/*` inside a comment
`// TODO: / FIXME:`	picked up by IDE task lists

⚠ COMMON MISTAKES

- Nesting `/* … /* … */ … */`; the first `*/` ends the comment and the rest becomes code
- Commenting out code with `//` on every line and leaving it in; delete it, version control remembers
- Writing comments that restate the code (`i++; // increment i`)

✓ TIPS & BEST PRACTICES

- Document the contract in the header: preconditions, units, ownership, thrown exceptions
- Use `[[deprecated("use foo2")]]` instead of a comment so the compiler warns callers
- Keep `TODO(name):` comments searchable with a consistent prefix

QUICK SUMMARY

- `//` — single-line comment
- `/* */` — block comment (no nesting)
- `/** @brief */` — Doxygen API docs and IDE hover
- `#if 0` — disable a block of code

Sheet 010

Variables & Initialization

declaration · copy init = · direct init () · brace init {} · narrowing errors · default init · uninitialized reads

DECLARING VARIABLES

A variable has a **type**, a **name**, and (ideally) an initial value. C++ offers several initialization syntaxes. **Brace initialization** `{}` is the most uniform: it works for every type and rejects **narrowing** conversions that would lose data. A local variable of built-in type that isn't initialized holds an **indeterminate** value, and reading it is undefined behavior.

NAMING

- Letters, digits, `_`; can't start with a digit
- Names starting with `_` + uppercase, or containing `__`, are reserved
- Common styles: `snake_case` (std library) or `camelCase`
- Declare variables close to first use, in the smallest scope

INITIALIZATION FORMS

```
int a;                // default-init: indeterminate if local!
int b = 5;            // copy initialization
int c(5);             // direct initialization
int d{5};             // brace (list) initialization
int e{};              // value-init → 0
int f = {5};          // copy-list initialization

int g{3.7};           // ERROR: narrowing double → int
int h = 3.7;          // compiles, h == 3 (silent truncation)

double x = 1.5, y = 2.0;     // several on one line
auto   n = 10;               // int (type deduced)
const int max_users = 100;   // can't change after init

std::vector<int> v1(3, 7);   // 3 elements: 7 7 7
std::vector<int> v2{3, 7};   // 2 elements: 3 7
```

CODE EXPLAINED

- → `int a;` — local built-ins are **not** zeroed; globals and statics are
- → `{}` — uniform syntax; blocks narrowing
- → `int e{};` — value-initialization; zero for numbers, `nullptr` for pointers
- → `int h = 3.7;` — allowed but truncates; `-Wconversion` warns
- → `v1(3, 7)` vs `v2{3, 7}` — braces prefer the `initializer_list` constructor

SYNTAX COMPARISON

FORM	NARROWING	NOTES
`T x = v;`	allowed	classic, readable
`T x(v);`	allowed	can trigger "most vexing parse"
`T x{v};`	error	uniform, safest
`T x{};`	n/a	zero / default value
`auto x = v;`	n/a	type from initializer

STORAGE DEFAULTS

WHERE DECLARED	UNINITIALIZED VALUE
`local int x;`	indeterminate (UB to read)
`global / namespace`	zero
`static local`	zero
`class member, no init`	indeterminate
`new int`	indeterminate; `new int{}` → 0

⚠ COMMON MISTAKES

- Reading an uninitialized local: `int total; total += x;` is UB. Write `int total = 0;`
- `Widget w();` declares a **function**, not an object (most vexing parse). Use `Widget w;` or `Widget w{};`
- Expecting `std::vector<int> v{5};` to have five elements; it has one element, 5

✓ TIPS & BEST PRACTICES

- Default to `{}` for initialization; switch to `()` when you need a non-list constructor like `vector(n, val)`
- Turn on `-Wuninitialized` / `-Wmaybe-uninitialized`; also MSVC's `/W4`
- Give class members default values: `int count = 0;`

QUICK SUMMARY

- `T x{v};` — brace init, rejects narrowing
- `T x{};` — value-init to zero
- `int a;` — local built-ins start indeterminate
- `const` — value fixed after initialization

Integer Types

WHOLE NUMBERS

C++ has several integer types whose sizes are **implementation-defined**, only guaranteed in minimum ranges. When exact width matters, use the fixed-width aliases from `<cstdint>`. **Signed overflow is undefined behavior**; unsigned arithmetic wraps around modulo 2^n. `std::size_t` is the unsigned type returned by `sizeof` and container `size()`.

GUARANTEES

- `char` ≤ `short` ≤ `int` ≤ `long` ≤ `long long`
- `int` ≥ 16 bits (32 in practice); `long long` ≥ 64 bits
- `long` is 32 bits on Windows, 64 on Linux/macOS (LP64)
- Since C++20, signed integers are defined as two's complement

LITERALS & LIMITS

```cpp
#include <cstdint>
#include <limits>

int         dec  = 1'000'000;      // digit separators (C++14)
int         hex  = 0xFF;           // 255
int         oct  = 017;            // 15  (leading 0 = octal!)
int         bin  = 0b1010;         // 10
unsigned    u    = 42u;            // unsigned int
long long   big  = 9'000'000'000LL;
std::size_t n    = 10uz;           // size_t literal (C++23)

std::int32_t  i32 = -5;            // exactly 32 bits
std::uint8_t  byte = 255;
std::int64_t  ms  = 1'700'000'000'000;

std::numeric_limits<int>::max();   // 2147483647
std::numeric_limits<int>::min();   // -2147483648
unsigned w = 0u - 1;               // 4294967295 (wraps)
int ov = INT_MAX + 1;              // UB: signed overflow
```

CODE EXPLAINED

- → `1'000'000` — `'` separators are ignored by the compiler
- → `017` — leading zero means octal, a classic trap
- → `LL` / `u` / `uz` — literal suffixes choose the type
- → `int32_t` — exact width; `int_fast32_t` / `int_least32_t` also exist
- → unsigned wraps; signed overflow is UB (optimizer assumes it never happens)

TYPICAL SIZES (64-BIT LINUX)

TYPE	BYTES	RANGE
`short`	2	−32,768 … 32,767
`int`	4	±2.1 × 10^9
`long`	8	±9.2 × 10^{18} (4 on Windows)
`long long`	8	±9.2 × 10^{18}
`unsigned int`	4	0 … 4.29 × 10^9
`std::size_t`	8	0 … 1.8 × 10^{19}

SUFFIXES & ALIASES

ITEM	MEANING
`u / U`	unsigned
`l / L`	long
`ll / LL`	long long
`uz / z`	size_t / signed size_t (C++23)
`std::ptrdiff_t`	signed pointer difference
`std::intmax_t`	widest signed integer

⚠ COMMON MISTAKES

- Comparing signed and unsigned: `-1 < 0u` is **false** because -1 converts to a huge unsigned value
- Looping `for (unsigned i = n - 1; i >= 0; --i)` ; the condition is always true, so it never ends
- Assuming `long` is 64 bits; on Windows it's 32. Use `std::int64_t`

✓ TIPS & BEST PRACTICES

- Use `std::ssize(container)` (C++20) to get a signed size and avoid mixed comparisons
- Check overflow-prone math with `-fsanitize=undefined`
- Use `std::cmp_less(a, b)` (C++20) to compare mixed signed/unsigned values safely

QUICK SUMMARY

- `int` — default integer, ≥ 32 bits in practice
- `<cstdint>` — `int32_t`, `uint64_t` for exact widths
- signed overflow — UB; unsigned — wraps
- `0x / 0b / 1'000` — hex, binary, separators

Floating-Point Types

REAL NUMBERS

Floating-point types follow **IEEE 754** on all mainstream platforms: `float` is 32-bit (~7 decimal digits), `double` is 64-bit (~15–16 digits), and `long double` is 80- or 128-bit on some platforms but just `double` on MSVC. Floating-point values are binary approximations, so `0.1 + 0.2 != 0.3`. Special values **NaN** and **infinity** come from operations like `0.0/0.0` and `1.0/0.0`.

DEFAULTS

- Unsuffixed literals like `3.14` are `double`
- `3.14f` is `float`; `3.14L` is `long double`
- C++23 `<stdfloat>`: `std::float16_t`, `float32_t`, `float64_t`, `bfloat16_t` (if supported)
- `NaN != NaN` is always true

LITERALS, LIMITS & SPECIAL VALUES

```cpp
#include <cmath>
#include <limits>

double d = 3.14;            // double
float  f = 2.5f;            // float
double e = 6.02e23;         // scientific
double h = 0x1.8p1;         // hex float: 3.0

0.1 + 0.2 == 0.3;                          // false
std::abs((0.1 + 0.2) - 0.3) < 1e-9;        // true
double inf = 1.0 / 0.0;                    // inf
double nan = std::sqrt(-1.0);              // nan
std::isnan(nan);                           // true
std::isinf(inf);                           // true
using lim = std::numeric_limits<double>;
lim::max();        // 1.79769e+308
lim::epsilon();    // 2.22045e-16
lim::infinity();   // inf
lim::quiet_NaN();  // nan
```

CODE EXPLAINED

- → `2.5f` — without `f` it's a `double` narrowed to `float`
- → `0x1.8p1` — hex mantissa × 2^exponent; exact binary value
- → `std::abs(a - b) < eps` — compare with a tolerance, never `==`
- → `std::isnan` / `std::isinf` — test special values
- → `epsilon()` — gap between 1.0 and the next representable double

TYPES

TYPE	BITS	PRECISION	MAX
`float`	32	~7 digits	3.4×10^{38}
`double`	64	~15–16 digits	1.8×10^{308}
`long double`	64–128	≥ double	platform

<CMATH> HELPERS

FUNCTION	RESULT
`std::round(2.5)`	3 (half away from 0)
`std::floor(-2.5)`	-3
`std::ceil(2.1)`	3
`std::trunc(-2.7)`	-2
`std::fmod(7.5, 2)`	1.5
`std::isfinite(x)`	not inf / NaN

⚠ COMMON MISTAKES

- Testing `x == 0.3` after arithmetic; use a relative tolerance like `std::abs(a-b) <= 1e-9 * std::max(std::abs(a), std::abs(b))`
- Storing money in `double`; rounding errors accumulate. Use integer cents or a decimal library
- Integer division before conversion: `double r = 1 / 2;` gives `0.0`. Write `1.0 / 2`

✓ TIPS & BEST PRACTICES

- Default to `double`; use `float` only for large arrays, GPUs, or memory-bound code
- Print round-trippable values with `std::println("{}", x)`; it prints the shortest exact representation
- Use `std::numbers::pi` (C++20) instead of hand-typed constants

QUICK SUMMARY

- `double` — default floating type (~15 digits)
- `3.0f` — float literal suffix
- `NaN / inf` — check with `std::isnan` / `std::isinf`
- never `==` — compare floats with a tolerance

bool & Character Types

BOOLEANS & CHARACTERS

`bool` holds `true` or `false` and converts to `1`/`0` in arithmetic. Any number or pointer converts to `bool`: zero and `nullptr` are false, everything else is true. `char` is one byte, and **whether plain `char` is signed is implementation-defined**. C++ adds explicit character types for each Unicode encoding (`char8_t`, `char16_t`, `char32_t`) and `std::byte` for raw memory that isn't text.

FACTS

- `sizeof(char)` is always 1
- `'A'` is a `char` literal; `"A"` is a string (array of 2 chars)
- `char8_t` (C++20) is UTF-8; `u8"…"` literals now use it
- `std::byte` supports bitwise ops but not arithmetic

BOOL, CHAR & FRIENDS

```cpp
#include <cctype>
#include <cstddef>

bool ready = true;
int  n = ready + ready;          // 2
bool ok = 42;                    // true (non-zero)

char c = 'A';
int  code = c;                   // 65
char next = c + 1;               // 'B'
std::toupper('q');               // 'Q' (returns int)

char8_t  u8 = u8'a';             // UTF-8 code unit
char16_t u16 = u'é';             // UTF-16
char32_t u32 = U'😀';            // UTF-32 code point
wchar_t  w = L'Ω';               // 16-bit Win, 32-bit Linux
std::byte b{0b1010'0000};
b |= std::byte{0x0F};            // 0xAF
std::to_integer<int>(b);         // 175
```

CODE EXPLAINED

- → `ready + ready` — bools promote to `int` in arithmetic
- → `c + 1` — chars are small integers; arithmetic works on codes
- → `std::toupper` — pass `unsigned char` values to avoid UB on negative chars
- → `u8'' / u'' / U'' / L''` — prefixes pick the character type
- → `std::byte` — raw memory; convert with `std::to_integer`

CHARACTER TYPES

TYPE	SIZE	LITERAL
`char`	1	'a'
`signed / unsigned char`	1	—
`char8_t`	1	u8'a'
`char16_t`	2	u'a'
`char32_t`	4	U'a'
`wchar_t`	2 / 4	L'a'

<CCTYPE>

FUNCTION	TESTS / RETURNS
`std::isalpha(c)`	letter
`std::isdigit(c)`	0–9
`std::isspace(c)`	space, \t, \n …
`std::isalnum(c)`	letter or digit
`std::toupper(c)`	uppercase version
`std::tolower(c)`	lowercase version

⚠ COMMON MISTAKES

- Calling `std::toupper(c)` with a negative `char` (e.g. UTF-8 bytes); cast first: `std::toupper((unsigned char)c)`
- Using `char` for small numbers and printing it: `std::cout << int8_t{65}` prints `A`, not `65`
- Treating one `char` as one character in UTF-8 text; non-ASCII characters take 2–4 bytes

✓ TIPS & BEST PRACTICES

- Print a `char` as a number with `+c` or `static_cast<int>(c)`
- Use `std::byte` for buffers of raw data so nobody accidentally does arithmetic on them
- Write `if (ptr)` and `if (!vec.empty())` instead of comparing to `true`

QUICK SUMMARY

- `bool` — `true` / `false`; zero converts to false
- `char` — 1 byte; signedness varies by platform
- `char8_t / char16_t / char32_t` — Unicode code units
- `std::byte` — raw memory, bitwise ops only

auto, decltype & Type Aliases

LET THE COMPILER INFER

`auto` deduces a variable's type from its initializer using template deduction rules: top-level `const` and references are **dropped** unless you write them (`const auto&`, `auto&`). `decltype(expr)` gives the declared type of an expression without evaluating it. **Type aliases** (`using`) give a long type a readable name, and `sizeof` / `alignof` report size and alignment in bytes.

DEDUCTION RULES

- `auto` → copies, drops `const` and `&`
- `auto&` / `const auto&` → binds a reference
- `auto&&` → forwarding reference; binds to anything
- `decltype(auto)` → keeps references exactly as written

AUTO, DECLTYPE, USING

```
auto i  = 42;              // int
auto d  = 3.0;             // double
auto s  = "hi";            // const char*  (not std::string!)
auto s2 = std::string{"hi"};
using namespace std::literals;
auto s3 = "hi"s;           // std::string

const int& cr = i;
auto a = cr;               // int        (const & dropped)
const auto& b = cr;        // const int&
for (const auto& [k, v] : myMap) {}   // no copies

decltype(i)  x = 0;        // int
decltype((i)) y = i;       // int&  (parenthesized → ref)

using Grid = std::vector<std::vector<int>>;
typedef unsigned long ulong;          // old style
template <class T> using Vec = std::vector<T>;

sizeof(int);               // 4
alignof(double);           // 8
```

CODE EXPLAINED

- → `"hi"` — string literals deduce to `const char*`; use `"hi"s` for `std::string`
- → `auto a = cr;` — makes a copy; `const` and reference vanish
- → `const auto&` — standard choice for read-only loop variables
- → `decltype((i))` — extra parentheses yield a reference
- → `using` — preferred over `typedef`; supports templates

AUTO FORMS

DECLARATION	RESULT FOR `const int& r`
`auto x = r;`	int (copy)
`auto& x = r;`	const int&
`const auto& x = r;`	const int&
`auto&& x = r;`	const int&
`auto&& x = 5;`	int&&
`decltype(auto) x = r;`	const int&

WHEN TO USE AUTO

SITUATION	RECOMMENDATION
`iterators, lambdas`	always `auto`
`make_unique<T>() result`	`auto` (type is visible)
`numeric literals`	explicit type if size matters
`proxy types (vector<bool>)`	careful: copies a proxy
`range-for over big objects`	`const auto&`

⚠ COMMON MISTAKES

- `for (auto x : bigVector)` copies every element; use `const auto&`
- `auto flag = bits[3];` with `std::vector<bool>` stores a proxy that dangles if the vector changes
- Expecting `auto s = "text";` to be a `std::string`

✓ TIPS & BEST PRACTICES

- Use `auto` when the type is obvious from the right-hand side or irrelevant to the reader
- Hover in your IDE (or use `decltype` in a `static_assert`) to confirm what `auto` deduced
- Prefer `using Alias = Type;` over `typedef` everywhere

QUICK SUMMARY

- `auto` — deduce type; drops const/ref
- `const auto&` — read-only reference, no copy
- `decltype(expr)` — declared type of an expression
- `using X = T;` — type alias (template-capable)

Constants: const, constexpr, constinit

IMMUTABILITY LEVELS

`const` means "can't be modified through this name" and can be initialized at run time. `constexpr` means "computable at **compile time**" and implies `const`, so the value can be used for array sizes, template arguments, and `static_assert`. `constinit` (C++20) guarantees a static/global is initialized at compile time but leaves it **mutable**, which avoids the static initialization order problem.

POINTERS & CONST

- `const int* p` — pointee is const (low-level)
- `int* const p` — pointer itself is const (top-level)
- Read declarations right to left: "p is a const pointer to int"
- `constexpr` variables must have a constant initializer

CONST, CONSTEXPR, CONSTINIT

```cpp
const int limit = readConfig();     // run-time value, then frozen
// limit = 5;                       // ERROR: assignment to const

constexpr int kSize = 16;           // compile-time constant
std::array<int, kSize> buf{};       // OK: needs a constant
static_assert(kSize % 8 == 0, "kSize must be a multiple of 8");

constexpr double kPi = 3.14159265358979;
constexpr int sq(int x) { return x * x; }
constexpr int kArea = sq(kSize);    // 256, computed by compiler

constinit int counter = 0;          // static init guaranteed
void tick() { ++counter; }          // still mutable

int n = 10;
const int* p1 = &n;   // *p1 = 3;  ERROR   p1 = &other; OK
int* const p2 = &n;   // *p2 = 3;  OK      p2 = &other; ERROR
const int* const p3 = &n;           // neither may change
```

CODE EXPLAINED

- → `const` — read-only after initialization; value may come from run time
- → `constexpr` — must be known at compile time; usable in constant expressions
- → `static_assert` — compile-time check with a message
- → `constinit` — only for static/thread storage; not const
- → `const int*` vs `int* const` — what the `const` sits next to is what's frozen

COMPARISON

KEYWORD	COMPILE-TIME?	MUTABLE?	APPLIES TO
`const`	maybe	no	any variable
`constexpr`	yes	no	variables, functions
`constinit`	init only	yes	static / thread_local
`consteval`	yes, always	—	functions only

REPLACING MAGIC NUMBERS

INSTEAD OF	WRITE
`#define MAX 100`	`constexpr int kMax = 100;`
`if (code == 404)`	`constexpr int kNotFound = 404;`
`buf[256]`	`std::array<char, kBufSize>`
`enum hack`	`static constexpr` member

⚠ COMMON MISTAKES

- Using `#define` for constants; macros ignore scope and types. Use `constexpr`
- Expecting `const int x = f();` to work as an array bound; only `constexpr` values do
- Casting away `const` with `const_cast` and writing to an object that was **defined** const is UB

✓ TIPS & BEST PRACTICES

- Make everything `const` by default; only drop it when the variable truly needs to change
- In headers, use `inline constexpr` for namespace-scope constants so every TU shares one definition
- Use `static_assert` to lock in assumptions like `sizeof(Packet) == 16`

QUICK SUMMARY

- `const` — read-only after init
- `constexpr` — compile-time constant
- `constinit` — compile-time init, still mutable
- `static_assert` — compile-time assertion

Conversions & Casts

CHANGING TYPES

C++ performs many **implicit conversions**: small integers promote to `int` in arithmetic, mixed int/double expressions convert to `double`, and signed/unsigned mixes convert to unsigned. For explicit conversions, use the four **named casts**. Each says what it does, can be searched for, and refuses conversions outside its job. C-style casts `(int)x` try all of them silently and should be avoided.

PROMOTION RULES

- `char`, `short`, `bool` → `int` before arithmetic
- int op double → double
- int op unsigned → unsigned (danger!)
- Narrowing (double → int) truncates toward zero

NAMED CASTS

```
double price = 9.99;
int dollars = static_cast<int>(price);          // 9
int avg     = static_cast<double>(7) / 2;       // 3.5 → 3

char c = 'A';
int code = static_cast<int>(c);                 // 65
enum class Color { Red, Green };
int g = static_cast<int>(Color::Green);         // 1

const std::string& ref = name;
auto& mut = const_cast<std::string&>(ref);      // removes const

Base* b = getShape();
if (auto* d = dynamic_cast<Circle*>(b)) {}      // null if not a Circle
std::uint32_t bits = 0x3F800000;
float f = std::bit_cast<float>(bits);           // 1.0f (C++20)
auto* raw = reinterpret_cast<std::byte*>(&bits);// view as bytes
int bad = (int)price;                           // C-style: avoid
```

CODE EXPLAINED

- → `static_cast` — ordinary, compile-time-checked conversions
- → `const_cast` — add/remove `const`; only legal to write if the object isn't really const
- → `dynamic_cast` — run-time-checked downcast in polymorphic hierarchies
- → `std::bit_cast` — reinterpret bits between same-size types safely
- → `reinterpret_cast` — low-level pointer reinterpretation; easy to make UB

THE CASTS

CAST	USE FOR	SAFETY
`static_cast<T>`	numeric, enum, up/down casts	compile-time checked
`dynamic_cast<T>`	polymorphic downcast	run-time checked
`const_cast<T>`	remove const for legacy APIs	risky
`reinterpret_cast<T>`	pointer ↔ pointer / integer	dangerous
`std::bit_cast<T>`	type punning (C++20)	safe if same size
`(T)x`	any of the above	avoid

STRING ↔ NUMBER

FROM → TO	HOW
`int → string`	`std::to_string(42)`
`string → int`	`std::stoi("42")`
`string → double`	`std::stod("3.5")`
`fast, no throw`	`std::from_chars`
`any → string`	`std::format("{}", x)`

⚠ COMMON MISTAKES

- Integer division before casting: `static_cast<double>(a / b)` truncates first. Cast an operand: `static_cast<double>(a) / b`
- Using `reinterpret_cast` to read a `float` as an `int`; this breaks aliasing rules. Use `std::bit_cast`
- `dynamic_cast` on a class with no virtual functions; it doesn't compile because there's no RTTI

✓ TIPS & BEST PRACTICES

- Enable `-Wconversion -Wsign-conversion` to surface hidden implicit conversions
- Use `std::in_range<T>(x)` (C++20) before narrowing to check the value fits
- Grep for `_cast<` in reviews; every cast should have a reason

QUICK SUMMARY

- `static_cast` — everyday explicit conversion
- `dynamic_cast` — checked downcast
- `const_cast / reinterpret_cast` — rare, low-level
- `std::bit_cast` — safe bit reinterpretation

Enumerations

NAMED CONSTANTS

An **enumeration** defines a type with a fixed set of named values. Old-style `enum` names leak into the surrounding scope and convert silently to `int`. **Scoped enums** (`enum class`) keep names inside the type and require explicit conversion, so they're the default in modern code. You can choose the **underlying type** to control size.

KEY POINTS

- Values start at 0 and count up unless assigned
- `enum class` requires `Color::Red`
- `using enum Color;` (C++20) brings names into scope locally
- `std::to_underlying(e)` (C++23) converts to the integer value

ENUM VS ENUM CLASS

```
enum Fruit { Apple, Banana };              // unscoped: names leak
int f = Banana;                            // 1, implicit conversion

enum class Status : std::uint8_t {         // scoped, 1 byte
    Pending = 1, Active, Closed = 10       // 1, 2, 10
};
Status s = Status::Active;
// int bad = s;                            // ERROR: no implicit conv.
auto v = std::to_underlying(s);            // 2 (uint8_t)
Status t = static_cast<Status>(10);        // Status::Closed

switch (s) {
    using enum Status;                     // C++20
    case Pending: break;
    case Active:  break;
    case Closed:  break;                   // -Wswitch warns if missing
}
enum class Perm : unsigned { Read = 1, Write = 2, Exec = 4 };
constexpr Perm operator|(Perm a, Perm b) {
    return Perm(std::to_underlying(a) | std::to_underlying(b));
}
auto rw = Perm::Read | Perm::Write;        // flags pattern
```

CODE EXPLAINED

- → `enum Fruit` — names are visible in the enclosing scope
- → `: std::uint8_t` — fixes the size and allows forward declaration
- → `std::to_underlying` — cleaner than `static_cast<std::underlying_type_t<E>>`
- → `using enum` — avoids repeating `Status::` inside a switch
- → `operator|` — opt-in bitwise ops for flag enums

ENUM VS ENUM CLASS

FEATURE	ENUM	ENUM CLASS
names scoped	✖	✔
implicit → int	✔	✖
forward-declarable	only with type	✔
name clashes	likely	none
recommended	legacy/C APIs	✔

UTILITIES

TOOL	PURPOSE
`std::underlying_type_t<E>`	the integer type
`std::to_underlying(e)`	value as integer (C++23)
`static_cast<E>(n)`	integer → enum
`using enum E;`	import names (C++20)
`magic_enum (lib)`	enum ↔ string names

⚠ COMMON MISTAKES

- Using unscoped enums in headers; `Red` from one enum can clash with `Red` in another
- Casting an out-of-range integer to an enum without a fixed underlying type, which can be UB
- Adding a `default:` to a switch over an enum; it hides the `-Wswitch` warning when new values are added

✓ TIPS & BEST PRACTICES

- Make every new enum an `enum class` with an explicit underlying type when size matters
- Add a `Count` sentinel as the last value to size arrays indexed by the enum
- Convert to strings with a `constexpr` lookup function or C++26 reflection when available

QUICK SUMMARY

- `enum class E` — scoped, type-safe enumeration
- `: std::uint8_t` — choose underlying type
- `std::to_underlying` — enum → integer (C++23)
- `using enum` — local shorthand (C++20)

C-Style Strings

STRINGS INHERITED FROM C

A **C-string** is an array of `char` ending in a **null terminator** `'\0'`. String literals like `"hi"` have type `const char[3]` and decay to `const char*`. The `<cstring>` functions walk memory until they hit `'\0'`, and none of them know the buffer's size, so they are a classic source of **buffer overflows**. You still meet C-strings at API boundaries such as `main`'s `argv`, C libraries, and `c_str()`.

KEY FACTS

- `"cat"` occupies 4 bytes: `c a t \0`
- `sizeof(arr)` counts the terminator; `strlen` doesn't
- Modifying a string literal is UB
- Convert: `std::string s = cstr;` and back with `s.c_str()`

C-STRING BASICS

```
#include <cstring>

char name[6] = "Alice";              // 5 chars + '\0'
const char* greet = "Hello";         // points to read-only literal

std::strlen(name);                   // 5
sizeof(name);                        // 6
std::strcmp("abc", "abd");           // < 0  (a before b)
std::strcmp(name, "Alice") == 0;     // true: equal

char buf[16];
std::strncpy(buf, greet, sizeof buf - 1);
buf[sizeof buf - 1] = '\0';          // strncpy may not terminate!
std::strncat(buf, "!", sizeof buf - std::strlen(buf) - 1);  // "Hello!"
std::strchr(greet, 'l');             // pointer to first 'l'
std::strstr(greet, "ll");            // pointer to "llo"
std::string s = greet;               // C-string → std::string
const char* back = s.c_str();        // std::string → C-string
```

CODE EXPLAINED

→ `char name[6] = "Alice"` — the compiler appends `'\0'`

→ `strcmp` returns 0 when equal; `==` would compare **addresses**

→ `strncpy` — bounded copy, but doesn't terminate if the source is too long

→ `strchr` / `strstr` — search, returning a pointer or `nullptr`

→ `c_str()` — valid until the `std::string` is modified or destroyed

<CSTRING> FUNCTIONS

FUNCTION	PURPOSE
`strlen(s)`	length without '\0'
`strcmp(a, b)`	compare: <0, 0, >0
`strncmp(a, b, n)`	compare first n chars
`strcpy(d, s)`	copy (unbounded, unsafe)
`strncpy(d, s, n)`	bounded copy
`strcat(d, s)`	append (unsafe)
`strchr / strrchr`	find first / last char
`strstr(h, n)`	find substring

PITFALLS

CODE	PROBLEM
`char b[4]; strcpy(b, "hello");`	overflow, UB
`if (s == "yes")`	compares pointers
`char* p = "lit"; p[0]='L';`	writes read-only memory
`strlen(nullptr)`	UB, crash
`missing '\0'`	functions read past the end

⚠ COMMON MISTAKES

- Comparing C-strings with `==`; that compares pointers. Use `std::strcmp(a, b) == 0` or convert to `std::string_view`
- Forgetting space for `'\0'` when sizing a buffer, so the terminator overruns it
- Storing `s.c_str()` and using it after `s` changes; the pointer dangles

✓ TIPS & BEST PRACTICES

- Use `std::string` or `std::string_view` for everything except C interop
- Wrap incoming C-strings immediately: `std::string_view sv{argv[1]};`
- Prefer `std::snprintf` or `std::format_to_n` when you must fill a fixed `char` buffer

QUICK SUMMARY

- `'\0'` — terminator that ends every C-string
- `strlen / strcmp` — length and comparison
- `strncpy` — bounded, but terminate manually
- `c_str()` — get a C-string from `std::string`

std::string Basics

THE STANDARD STRING

`std::string` from `<string>` is a growable, owning sequence of `char` that manages its own memory. It supports value semantics: copies are independent, == compares contents, and + concatenates. Most implementations use **small string optimization (SSO)**, which stores short strings (about 15–22 chars) inside the object with no heap allocation. `std::string` doesn't know about encodings; it's bytes.

NOTES

- `s[i]` is unchecked; `s.at(i)` throws `std::out_of_range`
- `size()` and `length()` are identical
- `"abc"s` (with `using namespace std::literals`) is a `std::string` literal
- Always null-terminated internally, so `c_str()` is O(1)

CREATING & USING STRINGS

```
#include <string>
using namespace std::literals;

std::string a;                    // ""
std::string b = "hello";
std::string c(3, 'x');            // "xxx"
std::string d = b + ", world";    // "hello, world"

b.size();                         // 5
b.empty();                        // false
b[0];                             // 'h'
b.at(10);                         // throws std::out_of_range
b += '!';                         // "hello!"
b.push_back('?');                 // "hello!?"
b.pop_back();                     // "hello!"
b == "hello!";                    // true (content compare)
b < "help";                       // true (lexicographic)
std::string copy = b;             // independent copy
```

CODE EXPLAINED

- `std::string c(3, 'x')` — count + char constructor (parentheses, not braces)
- `+` / `+=` — concatenation; at least one side must be `std::string`
- `at()` — bounds-checked access
- `==`, `<` — compare contents lexicographically
- copies are deep; use `std::move` or references to avoid them

CORE MEMBERS

MEMBER	PURPOSE
`size() / length()`	number of chars
`empty()`	true if size() == 0
`capacity()`	allocated space
`reserve(n)`	pre-allocate
`clear()`	make empty
`c_str() / data()`	pointer to chars
`front() / back()`	first / last char

CONSTRUCTION

EXPRESSION	RESULT
`std::string s;`	""
`std::string s("abc");`	"abc"
`std::string s(4, '-');`	"----"
`std::string s("abcdef", 3);`	"abc"
`std::string s(other, 2);`	from index 2
`std::string s(it1, it2);`	from iterator range

⚠ COMMON MISTAKES

- `"a" + "b"` doesn't compile (two pointers); make one side a `std::string`: `"a"s + "b"`
- Using `s[i]` with an index from user input; use `at()` or check `i < s.size()`
- Passing `std::string` by value to read-only functions; use `const std::string&` or `std::string_view`

✓ TIPS & BEST PRACTICES

- Call `reserve()` before building a long string in a loop
- Use `std::string_view` parameters so callers can pass literals, `std::string`, or slices without copies
- `std::getline(std::cin, s)` reads a whole line into a string

QUICK SUMMARY

- `std::string` — owning, growable byte string
- `+ / +=` — concatenate
- `at()` — checked access; `[]` unchecked
- `size() / empty()` — length checks

std::string — Search

FINDING TEXT

The `find` family returns the **index** of the first match, or the special value `std::string::npos` when nothing is found. Each takes an optional starting position. C++20 added `starts_with` / `ends_with`, and C++23 added `contains`, which cover most yes/no checks without index juggling. The same members exist on `std::string_view`.

NPOS

- `npos` is `size_t(-1)`, the largest possible size
- Always compare: `if (pos != std::string::npos)`
- `find_first_of("aeiou")` matches **any** char in the set
- All searches are case-sensitive

SEARCH MEMBERS

```
std::string s = "banana split";

s.find("an");                  // 1
s.find("an", 2);               // 3   (search from index 2)
s.rfind("an");                 // 3   (last occurrence)
s.find('z');                   // npos
s.find_first_of("aeiou");      // 1   (first vowel)
s.find_last_of("aeiou");       // 10  ('i')
s.find_first_not_of("ban");    // 6   (' ')
s.find_last_not_of(" \t\n");   // 11  (trim helper)

s.starts_with("ban");          // true   (C++20)
s.ends_with("split");          // true   (C++20)
s.contains("nan");             // true   (C++23)

// Count every occurrence
std::size_t count = 0;
for (auto pos = s.find('a'); pos != std::string::npos;
     pos = s.find('a', pos + 1)) ++count;   // count == 3
```

CODE EXPLAINED

- → `find(str, pos)` — first match at or after `pos`
- → `rfind` — searches backward from the end
- → `find_first_of(set)` — any character from the set
- → `find_first_not_of` — first char **outside** the set (good for trimming)
- → loop pattern — restart at `pos + 1` to find all matches

SEARCH API

MEMBER	RETURNS
`find(x, pos=0)`	first index of x or npos
`rfind(x, pos=npos)`	last index of x
`find_first_of(set)`	first char in set
`find_last_of(set)`	last char in set
`find_first_not_of(set)`	first char not in set
`find_last_not_of(set)`	last char not in set

BOOLEAN CHECKS

MEMBER	SINCE	EXAMPLE
`starts_with(x)`	C++20	`url.starts_with("https")`
`ends_with(x)`	C++20	`file.ends_with(".cpp")`
`contains(x)`	C++23	`log.contains("ERROR")`
`compare(x)`	C++98	0 when equal

⚠ COMMON MISTAKES

- Testing `if (s.find(x))`; a match at index 0 is false and `npos` is true. Compare against `npos`
- Storing the result in `int`; `npos` won't fit. Use `std::size_t` or `auto`
- Expecting `find_first_of("abc")` to find the substring "abc"; it finds any single char a, b, or c

✓ TIPS & BEST PRACTICES

- Use `contains` / `starts_with` for readability when you don't need the index
- For case-insensitive search, lowercase copies first or use `std::ranges::search` with a custom predicate
- Use `std::regex` or `std::boyer_moore_searcher` for complex or repeated searches

QUICK SUMMARY

- `find / rfind` — first / last index or `npos`
- `find_first_of(set)` — any char from a set
- `starts_with / ends_with` — prefix/suffix test (C++20)
- `contains` — substring test (C++23)

std::string — Modify & Substring

substr · insert · erase · replace · append · resize · std::transform + toupper · trimming

EDITING STRINGS

`std::string` is mutable, and its editing members change the string **in place**. Most take a position and a count, and many return `*this` so calls can chain. `substr(pos, len)` returns a **new** string. There's no built-in `trim`, `split`, or case conversion, but each is a one- or two-liner with `find_first_not_of`, `std::views::split`, or `std::ranges::transform`.

NOTES

- Positions past `size()` throw `std::out_of_range`
- A count past the end is clamped (`substr(2, 100)` is fine)
- `erase` / `insert` shift every following char: O(n)
- `resize(n, c)` grows with `c` or truncates

MODIFY, SUBSTRING, TRANSFORM

```cpp
std::string s = "Hello World";

s.substr(6);                    // "World"
s.substr(0, 5);                 // "Hello"
s.insert(5, ",");               // "Hello, World"
s.erase(5, 1);                  // "Hello World"
s.replace(6, 5, "C++");         // "Hello C++"
s.append(" 23");                // "Hello C++ 23"
s.resize(5);                    // "Hello"
s.resize(8, '.');               // "Hello..."

std::ranges::transform(s, s.begin(),
    [](unsigned char c) { return std::toupper(c); }); // "HELLO..."

// trim whitespace
std::string t = "  padded \n";
t.erase(0, t.find_first_not_of(" \t\n"));
t.erase(t.find_last_not_of(" \t\n") + 1);            // "padded"
```

CODE EXPLAINED

- → `substr(pos, n)` — copy of up to `n` chars from `pos`
- → `insert` / `erase` / `replace` — edit in place
- → `resize` — truncate or pad
- → `ranges::transform` with `toupper` — in-place uppercase
- → trim — erase leading and trailing runs of whitespace

MUTATORS

MEMBER	EFFECT
`append(x) / +=`	add to end
`insert(pos, x)`	insert before pos
`erase(pos, n)`	remove n chars
`replace(pos, n, x)`	swap a range for x
`resize(n[, c])`	truncate / pad
`clear()`	empty the string
`swap(other)`	O(1) exchange

COMMON RECIPES

TASK	ONE-LINER
`split on ','`	`for (auto w : s | std::views::split(',')) …`
`join`	`s | std::views::join_with(',')` (C++23)
`reverse`	`std::ranges::reverse(s);`
`remove char`	`std::erase(s, ' ');` (C++20)
`repeat`	`std::string(3, '-')`
`lowercase`	`ranges::transform` + `tolower`

⚠ COMMON MISTAKES

- Calling `s.substr(pos)` with `pos = npos` from a failed `find`; it throws. Check `find` first
- Passing plain `char` to `std::toupper`; negative values are UB. Take `unsigned char` in the lambda
- Erasing inside an index loop and skipping the char that shifted into the current position

✓ TIPS & BEST PRACTICES

- Use `std::erase` / `std::erase_if` (C++20) to remove characters in one call
- For many edits, build a new string with `reserve` + `append` instead of repeated `insert`
- Return `std::string_view` slices instead of `substr` copies when the source outlives the view

QUICK SUMMARY

- `substr(pos, n)` — new string slice
- `insert / erase / replace` — in-place edits
- `ranges::transform` — case conversion
- `find_first_not_of` — trimming building block

std::string_view

NON-OWNING STRING REFERENCE

`std::string_view` (C++17, `<string_view>`) is a lightweight **pointer + length** pair that refers to characters owned by someone else. It can view a `std::string`, a literal, or part of a buffer without copying, and slicing with `substr` is O(1). It's the best type for **read-only string parameters**. Because it doesn't own the data, it must never outlive the string it points into.

RULES

- Pass by value: it's only 16 bytes
- Not guaranteed null-terminated, so don't pass `.data()` to C APIs
- Converting to `std::string` needs an explicit `std::string{sv}`
- `"text"sv` creates a `string_view` literal

USING STRING_VIEW

```
#include <string_view>
using namespace std::literals;

void log(std::string_view msg) {          // accepts anything string-ish
    std::println("[log] {}", msg);
}
log("literal");                           // no allocation
log(std::string{"owned"});                // OK during the call

std::string_view path = "/usr/local/bin";
path.substr(5);                           // "local/bin"  O(1)
path.remove_prefix(1);                    // "usr/local/bin"
path.remove_suffix(4);                    // "usr/local"
path.find('/');                           // 3

std::string owned{path};                  // explicit copy
std::string_view bad() {
    std::string tmp = "temporary";
    return tmp;                           // DANGLING: tmp destroyed
}
std::string_view oops = std::string{"x"}; // dangles immediately
```

CODE EXPLAINED

- → `std::string_view msg` — one signature handles literals, strings, and slices
- → `substr` — returns another view, no allocation
- → `remove_prefix` / `remove_suffix` — shrink the view in place
- → `std::string{path}` — explicit conversion when you need ownership
- → returning a view of a local — classic dangling bug

STRING VS STRING_VIEW

ASPECT	STD::STRING	STD::STRING_VIEW
owns data	yes	no
mutable	yes	no (view only)
substr cost	O(n) copy	O(1)
null-terminated	yes	not guaranteed
size	~32 bytes	16 bytes
safe to store	yes	only if source outlives it

WHEN TO USE

USE STRING_VIEW FOR	USE STD::STRING FOR
read-only params	storing in a class member
parsing / tokenizing	returning built strings
constexpr tables	values that outlive input
literals	calling C APIs (c_str)

⚠ COMMON MISTAKES

- Storing a `string_view` member that points into a temporary `std::string`; it dangles as soon as the temporary dies
- Passing `sv.data()` to `printf("%s")` or `fopen`; there may be no terminator
- Returning `std::string_view` from a function that builds a local string

✓ TIPS & BEST PRACTICES

- Default read-only string parameters to `std::string_view`
- Compile with `-Wdangling` (GCC 14+) / Clang's lifetime warnings to catch view misuse
- Use `std::string_view` in `constexpr` lookup tables; it's a literal type

QUICK SUMMARY

- `std::string_view` — non-owning (ptr, len) view
- pass by value — cheap read-only parameter
- `substr / remove_prefix` — O(1) slicing
- lifetime — never outlive the source string

String Conversions

NUMBERS ↔ TEXT

C++ offers three generations of conversion. `std::to_string` / `std::stoi` are simple but allocate, and `stoi` throws on bad input. `std::stringstream` handles mixed parsing with stream syntax. `std::from_chars` / `std::to_chars` (C++17, <charconv>) are the fastest: no allocation, no locale, no exceptions; they report errors through a result struct. `std::format` is the easiest way to produce text.

CHOOSING

- Quick script code: `std::stoi`, `std::to_string`
- Hot paths / parsers: `std::from_chars`, `std::to_chars`
- Formatting: `std::format("{:.2f}", x)`
- `to_string(double)` always prints 6 decimals: `"3.140000"`

CONVERSION EXAMPLES

```cpp
#include <charconv>
#include <sstream>
#include <string>

std::to_string(42);                 // "42"
std::to_string(3.14);               // "3.140000"
std::format("{}", 3.14);            // "3.14"

std::stoi("123");                   // 123
std::stoi("12abc");                 // 12 (stops at 'a')
std::stoi("abc");                   // throws std::invalid_argument

std::string_view in = "8080";
int port{};
auto [ptr, ec] = std::from_chars(in.data(), in.data() + in.size(), port);
if (ec == std::errc{}) {}           // success, port == 8080
std::istringstream iss("7 2.5 abc");
int i; double d; std::string w;
iss >> i >> d >> w;                 // 7, 2.5, "abc"
```

CODE EXPLAINED

- → `stoi` — parses a prefix; throws on no digits or overflow
- → `stoi(s, nullptr, 16)` — optional base argument
- → `from_chars` — returns `{ptr, errc}`; check `ec == std::errc{}`
- → `std::format("{}", x)` — shortest round-trip text for numbers
- → `istringstream` — tokenize mixed-type text with `>>`

TEXT → NUMBER

FUNCTION	ON ERROR
`std::stoi / stol / stoll`	throws invalid_argument / out_of_range
`std::stoul / stoull`	throws
`std::stof / stod / stold`	throws
`std::from_chars`	errc::invalid_argument / result_out_of_range
`iss >> x`	sets failbit
`std::atoi`	returns 0 (can't tell)

NUMBER → TEXT

FUNCTION	NOTES
`std::to_string(x)`	fixed 6 decimals for floats
`std::format("{}", x)`	shortest round-trip
`std::to_chars`	no alloc, fastest
`std::ostringstream`	stream formatting

⚠ COMMON MISTAKES

- Assuming `std::stoi("12abc")` fails; it returns 12. Use the `pos` out-param or `from_chars` and check `ptr == end`
- Calling `stoi` on user input without `try` / `catch`
- Using `std::atoi`; it can't report errors and has UB on overflow

✓ TIPS & BEST PRACTICES

- Wrap `from_chars` in a helper returning `std::optional<int>`
- Prefer `std::format` over `to_string` for floats to avoid trailing zeros
- Reuse one `std::ostringstream` with `.str("")` instead of creating one per value

QUICK SUMMARY

- `std::to_string / std::stoi` — simple, allocating, throwing
- `std::from_chars / to_chars` — fast, no-throw
- `std::istringstream` — parse mixed tokens
- `std::format` — best way to produce text

Sheet 024

Arithmetic Operators

+ - * / % · integer division & truncation · ++ / -- prefix vs postfix · mixed-type arithmetic · overflow

MATH OPERATORS

Arithmetic operators work on all numeric types after the **usual arithmetic conversions**: operands smaller than `int` promote to `int`, and mixed types convert to the wider one. **Integer division truncates toward zero**, and `%` takes the sign of the dividend. Dividing an integer by zero is undefined behavior, and so is signed overflow.

PRECEDENCE

- `*` `/` `%` bind tighter than `+` `-`
- Unary `-` and `++` / `--` bind tighter still
- Same precedence groups left-to-right: `8 / 4 / 2` = 1
- Parenthesize when in doubt

ARITHMETIC IN ACTION

```
int a = 17, b = 5;
a + b;            // 22
a - b;            // 12
a * b;            // 85
a / b;            // 3     (truncated)
a % b;            // 2
-17 / 5;          // -3    (toward zero)
-17 % 5;          // -2    (sign of dividend)
17.0 / 5;         // 3.4   (double wins)
static_cast<double>(a) / b;   // 3.4

int i = 5;
int x = i++;      // x = 5, i = 6   (postfix: old value)
int y = ++i;      // y = 7, i = 7   (prefix: new value)
char c = 'a';
auto z = c + 1;   // int 98 (char promoted)
unsigned u = 3;
u - 5;            // 4294967294 (wraps)
1 / 0;            // UB: integer divide by zero
1.0 / 0.0;        // inf
```

CODE EXPLAINED

→ `/` on ints — fractional part discarded

→ `%` — remainder; only for integer types (use `std::fmod` for floats)

→ `i++` vs `++i` — postfix yields old value, prefix yields new

→ `c + 1` — promotion makes the result `int`

→ unsigned subtraction below zero wraps around

OPERATORS

OP	NAME	EXAMPLE	RESULT
`+`	addition	7 + 2	9
`-`	subtraction	7 − 2	5
`*`	multiplication	7 * 2	14
`/`	division	7 / 2	3
`%`	modulo	7 % 2	1
`-x`	negation	−7	-7
`++ / --`	increment / decrement	++n	n + 1

DIVISION RULES

EXPRESSION	RESULT
`7 / 2`	3
`-7 / 2`	-3
`7 % -2`	1
`-7 % 2`	-1
`7.0 / 2`	3.5
`std::fmod(7.5, 2)`	1.5

⚠ COMMON MISTAKES

- `double avg = sum / count;` with two ints gives a truncated result; cast one operand first
- `i = i++ + ++i;` — modifying a variable twice without sequencing is UB (or unspecified); split the statement
- Using `%` to test oddness with negative numbers: `-3 % 2 == -1`, so test `x % 2 != 0`

✓ TIPS & BEST PRACTICES

- Prefer `++i` in loops and with iterators; it never needs a temporary copy
- Use `std::midpoint(a, b)` (C++20) to avoid overflow in `(a + b) / 2`
- Check for overflow with `__builtin_add_overflow` (GCC/Clang) or C++26 `std::add_sat`

QUICK SUMMARY

- `+ - * /` — standard arithmetic after promotion
- `int / int` — truncates toward zero
- `%` — remainder, sign follows dividend
- `i++ / ++i` — old value vs new value

Sheet 025

Comparison & Three-Way (<=>)

== != < > <= >= · <=> · std::strong_ordering · std::partial_ordering · = default comparisons · std::cmp_less

COMPARING VALUES

The six relational operators return `bool`. C++20 added the **three-way comparison** operator <=> ("spaceship"), which returns an **ordering** object that says whether the left side is less, equal, or greater. Writing `auto operator<=>(const T&) const = default;` gives a class all six comparisons, memberwise, in one line. A defaulted `operator==` is generated alongside it.

ORDERING TYPES

- `std::strong_ordering` — equal means interchangeable (ints)
- `std::weak_ordering` — equivalent but distinguishable (case-insensitive strings)
- `std::partial_ordering` — some values are unordered (`NaN`)
- Compare the result to 0: `(a <=> b) < 0`

RELATIONAL & SPACESHIP

```cpp
#include <compare>

3 == 3;              // true
3 != 4;              // true
2 < 5;  5 >= 5;      // true, true

auto r = 3 <=> 5;    // std::strong_ordering::less
r < 0;               // true

struct Version {
    int major, minor, patch;
    auto operator<=>(const Version&) const = default; // all 6 ops
};
Version a{1, 4, 0}, b{1, 10, 2};
a < b;               // true  (compares major, then minor, ...)
a == b;              // false
-1 < 0u;             // false!  -1 converts to unsigned
std::cmp_less(-1, 0u);         // true (C++20, <utility>)
```

CODE EXPLAINED

- → `<=>` — one comparison yields less / equal / greater
- → `= default` — memberwise comparison in declaration order
- → `partial_ordering::unordered` — result when NaN is involved
- → `-1 < 0u` — mixed signedness converts -1 to a huge unsigned value
- → `std::cmp_less` — mathematically correct mixed-sign comparison

OPERATORS

OP	NAME	EXAMPLE	RESULT
==	equal	4 == 4	true
!=	not equal	4 != 5	true
<	less	3 < 2	false
>	greater	3 > 2	true
<=	less or equal	2 <= 2	true
>=	greater or equal	1 >= 2	false
<=>	three-way	1 <=> 2	less

SAFE INTEGER COMPARES (C++20)

FUNCTION	MEANING
`std::cmp_equal(a, b)`	a == b
`std::cmp_less(a, b)`	a < b
`std::cmp_greater(a, b)`	a > b
`std::in_range<T>(x)`	x fits in T

⚠ COMMON MISTAKES

- Writing `if (x = 5)` ; that assigns and is always true. Use `==` , and enable `-Wparentheses`
- Chaining comparisons like math: `0 < x < 10` means `(0 < x) < 10` , always true. Write `0 < x && x < 10`
- Comparing floats with `==` after computation; use a tolerance

✓ TIPS & BEST PRACTICES

- Default `operator<=>` on value types; reorder members to change comparison priority
- Use `std::tie(a.x, a.y) < std::tie(b.x, b.y)` for ad-hoc lexicographic compares in older code
- Turn on `-Wsign-compare` to catch signed/unsigned comparisons

QUICK SUMMARY

- `== != < > <= >=` — return bool
- `<=>` — three-way comparison (C++20)
- `= default` — generate all comparisons
- `std::cmp_less` — safe mixed-sign compare

Logical Operators

BOOLEAN LOGIC

`&&` (and), `||` (or), and `!` (not) operate on `bool`, converting other operands first: zero, `nullptr`, and `false` are false; everything else is true. `&&` and `||` **short-circuit**: the right side is evaluated only if it can change the result. That makes guard patterns like `ptr && ptr->ok()` safe. C++ also accepts the alternative tokens `and`, `or`, `not`.

PRECEDENCE

- `!` binds tightest, then `&&`, then `||`
- `a || b && c` means `a || (b && c)`
- Overloaded `&&` / `||` lose short-circuiting, so avoid overloading them
- `&&` and `||` are sequence points: left is fully evaluated first

LOGIC & SHORT-CIRCUIT

```cpp
bool a = true, b = false;
a && b;            // false
a || b;            // true
!a;                // false
a and not b;       // true  (alternative tokens)

int* p = nullptr;
if (p && *p > 0) {}            // safe: *p never evaluated
if (!p || p->empty()) {}       // safe guard with ||

std::vector<int> v;
if (!v.empty() && v[0] == 42) {}   // check before index

if (std::ifstream f{"cfg.txt"}; f && f.peek() != EOF) {}

int n = 7;
bool inRange = n >= 1 && n <= 10;  // true
bool isEdge  = n == 0 || n == 10;  // false
bool odd     = n % 2;              // true (1 → true)
auto expensive = [] { return true; };
if (cached || expensive()) {}      // expensive() skipped if cached
```

CODE EXPLAINED

- → `p && *p > 0` — right side only runs when `p` is non-null
- → `!p || …` — the "null or empty" guard
- → streams convert to `bool`: true when no error
- → `n % 2` — integer converted to bool
- → put cheap or guarding checks on the left

TRUTH TABLE

A	B	A && B	A || B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

TRUTHINESS

VALUE	CONVERTS TO
`0, 0.0`	false
`non-zero number`	true
`nullptr`	false
`non-null pointer`	true
`stream in error`	false
`std::optional`	true if it has a value

⚠ COMMON MISTAKES

- Using bitwise `&` or `|` instead of `&&` / `||`; there's no short-circuit and both sides always run
- Writing `if (x == 1 || 2)`; that's always true. Write `x == 1 || x == 2`
- Putting the guard on the right: `*p > 0 && p` dereferences null first

✓ TIPS & BEST PRACTICES

- Name complex conditions: `const bool canRetry = attempts < 3 && !fatal;`
- Use `std::ranges::any_of` / `all_of` / `none_of` instead of long `||` chains over a container
- Apply De Morgan to simplify: `!(a && b)` == `!a || !b`

QUICK SUMMARY

- `&& / || / !` — and, or, not
- short-circuit — right side may not run
- guard pattern — `ptr && ptr->x`
- truthiness — zero/null are false

Bitwise Operators

WORKING WITH BITS

Bitwise operators work on the individual bits of integer values. They're used for flags, masks, hashing, compression, and hardware registers. Use **unsigned** types: shifting a negative value or shifting into the sign bit has historically been a trap. `<bit>` (C++20) provides portable helpers such as `std::popcount` and `std::has_single_bit` that compile to single CPU instructions. (For depth, see the Bitwise set.)

SHIFT RULES

- Shifting by ≥ the type's width is UB
- `x << n` multiplies by 2^n (unsigned)
- Right shift of negative signed values is arithmetic since C++20
- Operands smaller than `int` promote first: `~uint8_t{0}` is an `int`

BIT MANIPULATION

```cpp
#include <bit>
#include <bitset>

unsigned a = 0b1100, b = 0b1010;
a & b;             // 0b1000 = 8    AND
a | b;             // 0b1110 = 14   OR
a ^ b;             // 0b0110 = 6    XOR
~a;                // 0xFFFFFFF3    NOT (all 32 bits)
1u << 4;           // 16

unsigned flags = 0;
flags |=  (1u << 2);           // set bit 2
flags &= ~(1u << 2);           // clear bit 2
flags ^=  (1u << 3);           // toggle bit 3
bool on = flags & (1u << 3);   // test bit 3 → true
std::popcount(0b1011u);        // 3   set bits
std::countr_zero(8u);          // 3   trailing zeros
std::bit_width(255u);          // 8   bits needed
std::has_single_bit(64u);      // true (power of 2)
std::bitset<8>(37).to_string();// "00100101"
```

CODE EXPLAINED

→ `&` / `|` / `^` / `~` — and, or, xor, not on each bit

→ `|= (1u << n)` — set; `&= ~(1u << n)` — clear; `^=` — toggle

→ `std::popcount` — count 1 bits

→ `std::bit_width` — position of highest set bit + 1

→ `std::bitset<N>` — fixed-size bit array with printing

OPERATORS

OP	NAME	1100 OP 1010
`&`	AND	1000
`|`	OR	1110
`^`	XOR	0110
`~`	NOT	…0011
`<< 1`	left shift	11000
`>> 1`	right shift	0110

<BIT> (C++20)

FUNCTION	RETURNS
`std::popcount(x)`	number of 1 bits
`std::countl_zero(x)`	leading zeros
`std::countr_zero(x)`	trailing zeros
`std::bit_ceil(x)`	next power of 2 ≥ x
`std::bit_floor(x)`	prev power of 2 ≤ x
`std::byteswap(x)`	reverse bytes (C++23)

⚠ COMMON MISTAKES

- Writing `x & 1 == 0`; `==` binds tighter, so it's `x & (1 == 0)`. Parenthesize: `(x & 1) == 0`
- Shifting `1 << 31` with signed `int`; use `1u << 31`
- Applying `~` to small unsigned types and comparing; the result is a promoted `int` with high bits set

✓ TIPS & BEST PRACTICES

- Use `enum class` flags with overloaded `|` and `&` for type-safe bit sets
- Use `std::bitset::test`, `set`, `reset`, `count` for readable flag code
- Check endianness with `std::endian::native == std::endian::little`

QUICK SUMMARY

- `& | ^ ~` — bitwise and, or, xor, not
- `<< >>` — shifts; use unsigned types
- set/clear/toggle — `|=`, `&= ~`, `^=` with masks
- `<bit>` — `popcount`, `bit_width`, `rotl`

Sheet 028

Assignment & Other Operators

= · += -= *= /= · comma operator · :: · . and -> · sizeof · precedence table

ASSIGNMENT & THE REST

Assignment = stores a value and returns a reference to the left operand, so `a = b = 0` works right to left. Every binary arithmetic and bitwise operator has a **compound** form that combines the operation with assignment. This sheet also collects the other everyday operators: scope resolution `::`, member access `.` and `->`, `sizeof`, the comma operator, and the full precedence table.

NOTES

- `x += y` evaluates `x` once; `x = x + y` evaluates it twice
- Assignment is right-associative
- `::name` refers to the **global** `name`
- `->` is shorthand for `(*ptr).member`

ASSIGNMENT & MISC OPERATORS

```
int x = 10;
x += 5;        // 15
x -= 3;        // 12
x *= 2;        // 24
x /= 5;        // 4
x %= 3;        // 1
x <<= 4;       // 16
x |= 0b11;     // 19
int a, b, c;
a = b = c = 0; // all 0 (right to left)
std::string s = "hi";
std::string* ps = &s;
s.size();      // .  member of object
ps->size();    // -> member through pointer
std::string::npos;   // :: scope resolution
::globalCount; // global namespace
sizeof(int);   // 4    (bytes, compile-time)
sizeof x;      // 4    (parens optional for expressions)
int r = (a++, b++, a + b);   // comma: evaluates left, yields right
```

CODE EXPLAINED

→ compound ops — `op=` exists for `+ - * / % & | ^ << >>`

→ chained `=` — each assignment returns the left operand

→ `.` vs `->` — object vs pointer to object

→ `::` — names inside a namespace or class, or the global scope

→ comma operator — rarely useful outside `for` headers

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	`::`
2	`a++ a-- () [] . ->`
3	`++a --a ! ~ -a *p &x sizeof new delete`
4	`.* ->*`
5	`* / %`
6	`+ -`
7	`<< >>`
8	`<=>`

(CONTINUED)

LEVEL	OPERATORS
9	`< <= > >=`
10	`== !=`
11	& then ^ then |
12	&&
13	||
14	`?: = += -=` … (right to left)
15	,

⚠ COMMON MISTAKES

- Confusing `=` with `==` in conditions; enable warnings and consider `if (5 == x)` style only if your team uses it
- Expecting `a < b < c` style chaining to work like math; it doesn't
- Relying on the order of evaluation of function arguments: `f(i++, i++)` is unspecified

✓ TIPS & BEST PRACTICES

- When mixing `&`, `|`, `<<`, and comparisons, always parenthesize; their precedence surprises people
- Use `std::exchange(x, newVal)` to assign and get the old value in one step
- Prefer `sizeof(var)` over `sizeof(Type)` so it updates if the type changes

QUICK SUMMARY

- `= / op=` — assignment and compound assignment
- `. / ->` — member access via object / pointer
- `::` — scope resolution
- precedence — parenthesize mixed bitwise and comparisons

Sheet 029

if / else & Conditional Operator

if · else if · else · ?: · if with initializer · [[likely]] / [[unlikely]] · if consteval

BRANCHING

`if` runs a statement when its condition converts to `true`; `else if` chains further tests and `else` catches the rest. Since C++17, an `if` can declare a variable in an **initializer** that's scoped to the whole if/else chain. The **conditional operator** `?:` picks between two expressions and yields a value, so it works where a statement can't, such as in initializers.

VARIANTS

- `if (init; cond)` — C++17 initializer
- `[[likely]]` / `[[unlikely]]` — branch hints (C++20)
- `if constexpr` — compile-time branch (templates)
- `if consteval` — true during constant evaluation (C++23)

IF / ELSE / ?:

```
int score = 83;
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';                        // runs: 'B'
} else {
    grade = 'C';
}

if (auto it = users.find(id); it != users.end()) {
    std::println("found {}", it->second);   // it scoped to if/else
} else {
    std::println("no user {}", id);
}

if (err) [[unlikely]] { return handleError(err); }

const auto label = score >= 50 ? "pass" : "fail";  // "pass"
int clamped = v < 0 ? 0 : (v > 100 ? 100 : v);
```

CODE EXPLAINED

- → `else if` — just an `else` whose body is another `if`
- → `if (init; cond)` — keeps `it` out of the surrounding scope
- → `[[unlikely]]` — hints the optimizer about the cold path
- → `?:` — an expression; both branches must have a common type
- → nested ternaries — fine for clamps, otherwise prefer `if`

FORMS

FORM	USE
`if (c) s;`	single statement (braces optional)
`if (c) {…} else {…}`	two-way branch
`if (init; c)`	scoped helper variable
`if constexpr (c)`	discard branch at compile time
`if consteval {…}`	compile-time-only path
`c ? a : b`	value-producing choice

TERNARY RULES

EXPRESSION	RESULT TYPE
`b ? 1 : 2.5`	double
`b ? "x" : std::string{"y"}`	std::string
`b ? obj1 : obj2`	lvalue (assignable)
`b ? throw E{} : v`	type of v

⚠ COMMON MISTAKES

- Writing `if (x = 0)`; that assigns 0 and is always false. Use `==`
- A stray semicolon: `if (ok);` makes the body empty and always runs the next line
- Dangling `else`: without braces, `else` binds to the nearest `if`. Always use braces for nested ifs

✓ TIPS & BEST PRACTICES

- Return early for error cases to avoid deep nesting
- Use `std::clamp(v, 0, 100)` instead of nested ternaries
- Mark variables used only in a branch with the if-initializer form

QUICK SUMMARY

- `if / else if / else` — multi-way branching
- `if (init; cond)` — scoped initializer (C++17)
- `c ? a : b` — conditional expression
- `[[likely]] / [[unlikely]]` — branch hints

switch

MULTI-WAY BRANCH ON A VALUE

`switch` jumps to the `case` label matching an **integral or enum** value. Execution **falls through** into the next case unless you `break`, so mark intentional fall-through with `[[fallthrough]]`. `switch` can't test strings or ranges directly. Since C++17 it accepts an initializer like `if`. Compilers warn (`-Wswitch`) when a switch over an enum misses a value and has no `default`.

RULES

- `case` labels must be constant expressions
- Duplicate case values don't compile
- Declaring a variable in a case needs braces `{ }`
- Works on `int`, `char`, `enum`, `bool` — not `std::string`

SWITCH PATTERNS

```cpp
enum class Cmd { Start, Stop, Pause, Resume };
switch (cmd) {
    case Cmd::Start:
        launch();
        break;
    case Cmd::Pause:
        save();
        [[fallthrough]];                // intentional
    case Cmd::Stop:
        halt();                         // runs for Pause and Stop
        break;
    case Cmd::Resume: {                 // braces: local variable
        resume(now());
        break;
    }
}                                       // no default → -Wswitch helps
switch (char c = next(); c) {           // initializer (C++17)
    case 'y': case 'Y': return true;    // multiple labels
    case 'n': case 'N': return false;
    default:            return ask();
}
```

CODE EXPLAINED

- → `break` — leave the switch; omitting it falls through
- → `[[fallthrough]]` — silences the fall-through warning on purpose
- → `{ }` around a case — required when you declare variables
- → stacked labels — several values share one body
- → `return` inside a case — also ends the switch

PIECES

ELEMENT	PURPOSE
`switch (expr)`	integral / enum to match
`case K:`	jump target for value K
`default:`	no case matched
`break;`	exit the switch
`[[fallthrough]];`	explicit fall-through
`switch (init; v)`	scoped initializer

ALTERNATIVES

NEED	USE INSTEAD
`match strings`	`if` chain or `unordered_map`
`ranges (1–10)`	`if` with comparisons
`type-based dispatch`	`std::visit` on `std::variant`
`value → action table`	map of lambdas

⚠ COMMON MISTAKES

- Forgetting `break`, so one case runs into the next
- Adding `default:` to enum switches; it hides the warning when someone adds a new enumerator
- Jumping past an initialized variable: `case 1: int x = 5; break; case 2:` doesn't compile without braces

✓ TIPS & BEST PRACTICES

- Turn on `-Wimplicit-fallthrough` so missing `break`s are reported
- For enum switches, handle every value and put the "impossible" path after the switch
- Return directly from cases in small mapping functions to avoid `break` noise

QUICK SUMMARY

- `switch (v)` — jump to matching `case`
- `break` — stop fall-through
- `[[fallthrough]]` — intentional fall-through
- enums + no default — compiler checks completeness

for Loops

COUNTED LOOPS

The classic `for` loop has three parts: an **init** statement, a **condition** checked before each iteration, and an **increment** run after each iteration. Any part may be empty. Variables declared in the init are scoped to the loop. C++20's `std::views::iota` offers a range-based alternative for counting that avoids off-by-one mistakes.

NOTES

- `for (;;)` is an infinite loop
- The condition is re-evaluated every iteration
- Use `std::size_t` (or `std::ssize`) when indexing containers
- Prefer range-for when you don't need the index

FOR LOOP FORMS

```cpp
for (int i = 0; i < 5; ++i) {          // 0 1 2 3 4
    std::print("{} ", i);
}
for (int i = 10; i > 0; i -= 3) {}     // 10 7 4 1

for (int i = 0, j = 9; i < j; ++i, --j) {
    std::swap(arr[i], arr[j]);         // reverse in place
}

std::vector<int> v{4, 8, 15};
for (std::size_t i = 0; i < v.size(); ++i) {}  // index loop

for (auto i = v.size(); i-- > 0;) {}   // safe unsigned countdown

for (int i : std::views::iota(1, 4)) {}        // 1 2 3
for (int i : std::views::iota(0) | std::views::take(3)) {}

for (int r = 0; r < 3; ++r)            // nested: 3×3 grid
    for (int c = 0; c < 3; ++c)
        grid[r][c] = r * 3 + c;
```

CODE EXPLAINED

- → `i -= 3` — increment can be any expression
- → `i = 0, j = 9` — comma declares two variables of the same type
- → `i-- > 0` — counts down an unsigned index without wrapping
- → `views::iota(1, 4)` — half-open range 1, 2, 3
- → nested loops — inner variable restarts each outer iteration

PARTS

PART	RUNS	EXAMPLE
init	once, before the loop	`int i = 0`
condition	before each pass	`i < n`
increment	after each pass	`++i`
body	while condition true	`{ … }`

CHOOSING A LOOP

NEED	USE
index + element	`views::enumerate` / index loop
just elements	range-based `for`
count n times	`for (int i = 0; i < n; ++i)`
unknown iterations	`while`
at least once	`do-while`

⚠ COMMON MISTAKES

- `for (unsigned i = n - 1; i >= 0; --i)` never ends: unsigned values are always ≥ 0
- Comparing `int i` with `v.size()` gives sign-compare warnings and bugs for huge sizes; use `std::size_t` or `std::ssize(v)`
- Modifying the container (push_back/erase) while index-looping without adjusting the index or bound

✓ TIPS & BEST PRACTICES

- Cache expensive conditions: `for (auto i = 0uz, n = v.size(); i < n; ++i)`
- Prefer algorithms (`std::ranges::fill`, `transform`) over hand-written loops for common tasks
- Use `views::iota` with `views::reverse` for clear countdowns: `iota(0, n) | views::reverse`

QUICK SUMMARY

- `for (init; cond; inc)` — classic counted loop
- `std::size_t` — index type for containers
- `i-- > 0` — safe unsigned countdown
- `views::iota(a, b)` — counting range (C++20)

Range-Based for

ITERATING A RANGE

The **range-based for** loop visits every element of anything with `begin()` and `end()`: containers, arrays, strings, initializer lists, and views. The loop variable's declaration decides whether you **copy** each element, **modify** it in place, or read it through a const reference. Combine it with **structured bindings** for maps and pairs, and with views for indexes, reversal, or filtering.

PICKING THE VARIABLE

- `auto x` — copy (cheap types only)
- `auto& x` — modify elements in place
- `const auto& x` — read-only, no copies (default choice)
- `auto&& x` — works with proxies and views (generic code)

RANGE-FOR VARIANTS

```
std::vector<std::string> names{"Ada", "Bo", "Cy"};

for (const auto& n : names) std::print("{} ", n);    // Ada Bo Cy
for (auto& n : names) n += "!";                      // modify in place
for (int x : {1, 2, 3}) {}                           // initializer list
for (char c : std::string{"hey"}) {}                 // characters

std::map<std::string, int> ages{{"Ada", 36}, {"Bo", 25}};
for (const auto& [name, age] : ages)                 // structured binding
    std::println("{} is {}", name, age);

for (auto [i, n] : std::views::enumerate(names))     // C++23
    std::println("{}: {}", i, n);                    // 0: Ada! …
for (const auto& n : names | std::views::reverse) {}
for (auto x : nums | std::views::filter([](int v) { return v > 0; })) {}

for (auto v = makeVec(); auto x : v) {}              // init (C++20)
```

CODE EXPLAINED

- → `const auto&` — no copy, can't modify
- → `auto&` — writes go to the container
- → `[name, age]` — unpacks each `std::pair` in the map
- → `views::enumerate` — yields `(index, element)` tuples
- → `views::reverse` / `filter` — lazy adaptors, no copies
- → `for (init; x : range)` — keeps a temporary alive (C++20)

LOOP VARIABLE

DECLARATION	COPIES?	CAN MODIFY?
`auto x`	yes	copy only
`auto& x`	no	yes
`const auto& x`	no	no
`auto&& x`	no	yes (if non-const)

HANDY VIEWS

VIEW	YIELDS
`views::enumerate(r)`	(index, elem) — C++23
`views::reverse`	back to front
`views::zip(a, b)`	pairs from two ranges — C++23
`views::keys / values`	map keys / values
`views::filter(p)`	elements matching p

⚠ COMMON MISTAKES

- `for (auto s : names)` copies every string; use `const auto&`
- Calling `push_back` or `erase` on the container inside its own range-for; iterators are invalidated (UB)
- `for (auto x : getObj().items())` before C++23 could dangle if `getObj()` returns a temporary; use the init form

✓ TIPS & BEST PRACTICES

- Default to `const auto&`; drop to `auto` only for small types like `int` or `char`
- Use `std::views::keys(m)` when you only need map keys
- Can't get an index? Use `views::enumerate` instead of a separate counter

QUICK SUMMARY

- `for (decl : range)` — visit every element
- `const auto&` — default, no copies
- `[k, v]` — structured bindings for pairs/maps
- `views::enumerate / reverse` — index and order helpers

while & do-while

CONDITION-DRIVEN LOOPS

A `while` loop checks its condition **before** each iteration, so the body may run zero times. A `do-while` checks **after** each iteration, so the body runs at least once, which suits menus and "try then check" logic. Use these loops when the number of iterations isn't known up front: reading input until EOF, waiting for a state, or processing a queue until it's empty.

NOTES

- `do { … } while (cond);` needs the trailing semicolon
- `while (true)` with `break` is a common "loop and a half"
- The condition can declare a variable: `while (auto x = next())`
- Streams in conditions are true while reads succeed

WHILE & DO-WHILE

```
int n = 27, steps = 0;
while (n != 1) {                        // Collatz
    n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
    ++steps;
}                                       // steps == 111

int value;
while (std::cin >> value) {             // read until EOF / bad input
    total += value;
}

std::string line;
while (std::getline(file, line)) {}     // line by line
while (auto job = queue.try_pop()) {    // optional-like condition
    process(*job);
}
int choice;
do {
    showMenu();
    std::cin >> choice;
} while (choice != 0);                  // runs at least once
```

CODE EXPLAINED

- → `while (n != 1)` — may run zero times
- → `while (cin >> value)` — stops on EOF or a non-number
- → `while (getline(…))` — the idiomatic line-reading loop
- → `while (auto job = …)` — condition variable, tested for truthiness
- → `do … while` — menu loop runs before checking

WHILE VS DO-WHILE

ASPECT	WHILE	DO-WHILE
check	before body	after body
min. runs	0	1
syntax end	}	} while (c);
typical use	input, queues	menus, retries

LOOP IDIOMS

PATTERN	PURPOSE
`while (cin >> x)`	consume all input
`while (getline(f, s))`	read file lines
`while (!q.empty())`	drain a queue
`while (true) {… break;}`	exit mid-body
`do {…} while (retry());`	retry at least once

⚠ COMMON MISTAKES

- Forgetting to change the loop variable, causing an infinite loop
- `while (!file.eof())` processes the last line twice or reads garbage; loop on the read itself
- Missing `;` after `while (cond)` in a do-while, which is a compile error

✓ TIPS & BEST PRACTICES

- If you can express the loop as a `for` with a clear counter, do that instead
- Add a maximum-iteration guard to loops that wait on external state
- Put the exit condition in the loop header when possible; `break` in the middle is harder to read

QUICK SUMMARY

- `while (c)` — test first, 0+ runs
- `do {} while (c);` — test last, 1+ runs
- `while (cin >> x)` — read until failure
- `while (auto v = f())` — declare in the condition

Loop Controls & goto

CHANGING LOOP FLOW

`break` exits the innermost loop (or `switch`) immediately. `continue` skips the rest of the current iteration and goes to the next one; in a `for` loop, the increment still runs. C++ has **no labeled break**, so leaving nested loops takes a flag, a `return` from a helper function or lambda, or a `goto`. `goto` is legal but rarely the clearest choice.

NOTES

- `break` / `continue` affect only the innermost loop
- `break` inside a `switch` inside a loop leaves the switch, not the loop
- `return` exits the whole function, ending all loops
- `goto` can't jump forward past variable initializations

BREAK, CONTINUE, GOTO

```
for (int x : data) {
    if (x < 0) continue;              // skip negatives
    if (x == 0) break;                // stop at first zero
    sum += x;
}

// Find in a 2D grid: return from a lambda to exit both loops
auto find = [&](int target) -> std::optional<std::pair<int,int>> {
    for (int r = 0; r < rows; ++r)
        for (int c = 0; c < cols; ++c)
            if (grid[r][c] == target) return std::pair{r, c};
    return std::nullopt;
};

for (int i = 0; i < n; ++i) {         // goto version
    for (int j = 0; j < m; ++j)
        if (bad(i, j)) goto done;
}
done:
    cleanup();
```

CODE EXPLAINED

- → `continue` — jump to the next iteration
- → `break` — leave the innermost loop
- → returning from a lambda — clean exit from nested loops
- → `std::optional` — signals "not found" without sentinels
- → `goto done;` — jumps to a label in the same function

STATEMENTS

STATEMENT	EFFECT
`break;`	exit innermost loop / switch
`continue;`	next iteration (for: runs increment)
`return v;`	exit the function
`goto label;`	jump to label in same function
`throw e;`	unwind to a matching catch

EXITING NESTED LOOPS

TECHNIQUE	TRADE-OFF
`extract function + return`	clearest; preferred
`immediately-invoked lambda`	same, inline
`bool flag`	extra checks in each loop
`goto`	concise, but unstructured
`exception`	wrong tool for normal flow

⚠ COMMON MISTAKES

- Expecting `break` inside a `switch` to exit the enclosing loop; it only leaves the switch
- Using `continue` in a `while` loop before the counter update, causing an infinite loop
- Using exceptions to leave loops for normal control flow; they're slow and obscure intent

✓ TIPS & BEST PRACTICES

- Turn the nested loop into a function that returns the result, then `return` early
- Use algorithms like `std::ranges::find_if` to remove many manual `break` loops
- Keep `continue` guard clauses at the top of the loop body for readability

QUICK SUMMARY

- `break` — exit innermost loop
- `continue` — skip to next iteration
- nested exit — return from a function or lambda
- `goto` — allowed, rarely the best option

Exception Basics

EXCEPTIONS

An **exception** reports an error that the current code can't handle. `throw` creates an exception object and starts **stack unwinding**: the runtime leaves functions one by one, running destructors for every local, until it finds a matching `catch`. If none matches, `std::terminate` ends the program. Unwinding plus destructors (RAII) is what makes exceptions safe for resource cleanup.

NOTES

- Throw by value, catch by `const&`
- `catch` clauses are tried in order; put derived types first
- `catch (...)` catches anything
- C++ has no `finally`; destructors do the cleanup

TRY / CATCH / THROW

```cpp
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("divide by zero");
    return a / b;
}
try {
    auto r = divide(10, 0);                  // throws; rest skipped
} catch (const std::invalid_argument& e) {
    std::println("bad input: {}", e.what()); // bad input: divide by zero
} catch (const std::exception& e) {          // any std exception
    std::println("error: {}", e.what());
} catch (...) {                              // anything else
    std::println("unknown error");
    throw;                                   // rethrow the same object
}
void load() {
    std::ifstream f{"data.txt"};             // closed by destructor
    parse(f);                                // even if this throws
}
```

CODE EXPLAINED

- → `throw std::invalid_argument(...)` — create and throw an exception object
- → `catch (const T& e)` — matches T and anything derived from T
- → `e.what()` — the message passed to the constructor
- → `throw;` — rethrows the current exception, preserving its type
- → `std::ifstream` destructor — runs during unwinding (RAII)

SYNTAX

CONSTRUCT	MEANING
`throw expr;`	raise an exception
`throw;`	rethrow current exception
`try { … }`	protected block
`catch (const T& e)`	handle T or derived
`catch (...)`	catch-all
`std::exception_ptr`	store / transfer an exception

UNWINDING FACTS

SITUATION	WHAT HAPPENS
`no matching catch`	`std::terminate()`
`destructor throws during unwind`	`std::terminate()`
`throw in noexcept function`	`std::terminate()`
`local objects`	destroyed in reverse order

⚠ COMMON MISTAKES

- Catching by value: `catch (std::exception e)` copies and **slices** derived exceptions. Catch by `const&`
- Ordering `catch (const std::exception&)` before a more specific handler, so the specific one never runs
- Using `throw e;` to rethrow; it copies and may slice. Use plain `throw;`

✓ TIPS & BEST PRACTICES

- Throw types derived from `std::exception` so callers can catch one base type
- Catch at the level that can actually do something: retry, report to the user, or abort cleanly
- Wrap `main`'s body in `try { … } catch (const std::exception& e)` to log unexpected errors

QUICK SUMMARY

- `throw` — raise; starts stack unwinding
- `catch (const T&)` — handle by reference
- `throw;` — rethrow unchanged
- RAII — destructors replace `finally`

Sheet 036

Standard Exception Hierarchy

std::exception · what() · std::logic_error · std::runtime_error · std::out_of_range · std::bad_alloc · <stdexcept>

BUILT-IN EXCEPTION TYPES

The standard library's exceptions all derive from `std::exception`, which has one virtual function, `what()`. Two main branches exist: `std::logic_error` for **bugs that could have been prevented** (bad arguments, out-of-range indexes) and `std::runtime_error` for **conditions only detectable at run time** (I/O failures, overflow). Most live in `<stdexcept>`; a few have their own headers.

HEADERS

- `<exception>` — `std::exception`, `std::terminate`
- `<stdexcept>` — logic_error, runtime_error families
- `<new>` — `std::bad_alloc`
- `<system_error>` — `std::system_error` with an `error_code`

HIERARCHY IN PRACTICE

```
#include <stdexcept>
#include <vector>

std::vector<int> v{1, 2, 3};
try {
    v.at(10);                              // throws std::out_of_range
} catch (const std::out_of_range& e) {
    std::println("{}", e.what());          // vector::_M_range_check: ...
}

try { std::stoi("abc"); }
catch (const std::invalid_argument&) {}    // from stoi

std::size_t huge = 1ull << 62;
try { auto p = new int[huge]; }
catch (const std::bad_alloc&) {}           // bad_array_new_length
try { std::optional<int> o; o.value(); }
catch (const std::bad_optional_access&) {}
throw std::runtime_error(std::format("cannot open {}", path));
```

CODE EXPLAINED

- → `v.at(10)` — checked access throws `out_of_range`
- → `std::stoi` — throws `invalid_argument` or `out_of_range`
- → `new` failure — throws `bad_alloc` (unless `nothrow`)
- → `bad_optional_access` / `bad_any_cast` — misuse of vocabulary types
- → `std::format` in the message — include the failing value

LOGIC_ERROR FAMILY

TYPE	TYPICAL SOURCE
`std::invalid_argument`	`std::stoi("abc")`
`std::domain_error`	math outside domain
`std::length_error`	exceeding `max_size()`
`std::out_of_range`	`.at()`, `stoi` overflow
`std::future_error`	misuse of futures

RUNTIME_ERROR & OTHERS

TYPE	TYPICAL SOURCE
`std::runtime_error`	general run-time failure
`std::range_error`	result can't be represented
`std::overflow_error`	arithmetic overflow (`bitset`)
`std::system_error`	OS errors (threads, filesystem)
`std::bad_alloc`	`new` out of memory
`std::bad_cast`	failed `dynamic_cast<T&>`

⚠ COMMON MISTAKES

- Expecting `v[10]` to throw; only `.at()` checks bounds. `operator[]` out of range is UB
- Catching `std::exception` everywhere and swallowing errors silently; log or rethrow
- Throwing raw strings or ints (`throw "oops";`); they don't derive from `std::exception`

✓ TIPS & BEST PRACTICES

- Throw `std::invalid_argument` for bad caller input and `std::runtime_error` for environment failures
- `std::filesystem` throws `filesystem_error`; overloads taking `std::error_code&` don't throw
- Put context in the message: what failed and with which value

QUICK SUMMARY

- `std::exception` — root; `what()` returns the message
- `logic_error` — preventable bugs
- `runtime_error` — run-time failures
- `bad_alloc` — memory exhaustion

Custom Exceptions & noexcept

YOUR OWN EXCEPTIONS & NOEXCEPT

Define custom exceptions by deriving from a standard type, usually `std::runtime_error`, and add fields that callers need, like an error code or file name. Mark functions that will never throw `noexcept`: it documents the promise and lets the library choose faster paths (e.g., `std::vector` moves elements only if the move constructor is `noexcept`). If a `noexcept` function does throw, `std::terminate` is called.

GUARANTEES

- **Nothrow**: never throws (destructors, swaps, moves)
- **Strong**: on failure, state is unchanged (commit-or-rollback)
- **Basic**: on failure, no leaks and invariants hold
- Destructors are implicitly `noexcept`

CUSTOM EXCEPTION + NOEXCEPT

```
class HttpError : public std::runtime_error {
public:
    HttpError(int status, const std::string& url)
        : std::runtime_error(std::format("HTTP {} for {}", status, url)),
          status_(status) {}
    int status() const noexcept { return status_; }
private:
    int status_;
};
try { throw HttpError(404, "/api/users"); }
catch (const HttpError& e) {
    std::println("{}", e.what());   // HTTP 404 for /api/users
    if (e.status() == 404) retryLater();
}
struct Buffer {
    Buffer(Buffer&& other) noexcept;           // enables fast vector growth
    void swap(Buffer& o) noexcept;
};
template <class T>
void relocate(T& t) noexcept(std::is_nothrow_move_constructible_v<T>);
static_assert(std::is_nothrow_move_constructible_v<Buffer>);
```

CODE EXPLAINED

- → derive from `std::runtime_error` — inherits `what()` and storage for the message
- → extra fields — give handlers structured data
- → `noexcept` on getters and moves — documents and optimizes
- → `noexcept(expr)` — conditional, based on a compile-time bool
- → `noexcept(…)` operator — asks whether an expression can throw

NOEXCEPT FORMS

FORM	MEANING
`void f() noexcept;`	promises never to throw
`noexcept(true / false)`	conditional specifier
`noexcept(expr)`	operator: can expr throw?
`~T()`	implicitly noexcept
`throw in noexcept`	std::terminate()

DESIGN GUIDELINES

DO	WHY
`derive from std::exception`	one catch for everything
`keep the hierarchy shallow`	easier to handle
`make moves noexcept`	vector uses them on growth
`never throw from destructors`	terminate during unwinding

⚠ COMMON MISTAKES

- Forgetting `noexcept` on move constructors, so `std::vector` copies instead of moving when it grows
- Marking a function `noexcept` that calls code that can throw; the program terminates instead of propagating the error
- Overriding `what()` and returning a pointer to a local string that dies on return

✓ TIPS & BEST PRACTICES

- Let `std::runtime_error` store the message; don't keep your own `std::string` just for `what()`
- Use `= default` special members; they deduce `noexcept` automatically
- Verify with `std::is_nothrow_move_constructible_v` for types stored in vectors

QUICK SUMMARY

- `class E : public std::runtime_error` — custom exception
- `noexcept` — promise not to throw
- `noexcept(expr)` — conditional / query
- basic / strong / nothrow — exception-safety levels

std::optional & std::expected

ERRORS AS VALUES

Not every failure deserves an exception. `std::optional<T>` (C++17) holds either a T or nothing, good for "not found". `std::expected<T, E>` (C++23) holds either a value or an **error** describing what went wrong, like Rust's `Result`. Both support monadic chaining with `and_then`, `transform`, and `or_else`, so you can build pipelines without nested `if`s.

ACCESS

- `if (opt)` / `opt.has_value()` — check first
- `*opt` — unchecked; `opt.value()` — throws if empty
- `opt.value_or(def)` — fallback value
- `exp.error()` — the error when `!exp`

OPTIONAL & EXPECTED

```
#include <optional>
#include <expected>

std::optional<User> findUser(int id) {
    if (auto it = db.find(id); it != db.end()) return it->second;
    return std::nullopt;                        // "no value"
}
auto name = findUser(7).transform(&User::name).value_or("guest");

enum class ParseErr { Empty, NotNumber };
std::expected<int, ParseErr> parsePort(std::string_view s) {
    if (s.empty()) return std::unexpected(ParseErr::Empty);
    int v{};
    auto [p, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{}) return std::unexpected(ParseErr::NotNumber);
    return v;
}
auto r = parsePort("8080");
if (r) std::println("port {}", *r);            // port 8080
auto checked = parsePort(arg)
    .and_then([](int p) -> std::expected<int, ParseErr> { return p; })
    .transform([](int p) { return p + 1; });
```

CODE EXPLAINED

- `std::nullopt` — the empty state of an optional
- `.transform(f)` — apply f if there's a value, keep empty otherwise
- `std::unexpected(e)` — construct the error alternative
- `if (r)` — true when it holds a value
- `.and_then(f)` — chain functions that themselves return expected

MONADIC OPERATIONS

MEMBER	RUNS WHEN	F RETURNS
`and_then(f)`	has value	optional / expected
`transform(f)`	has value	plain value
`or_else(f)`	empty / error	optional / expected
`transform_error(f)`	error (expected)	new error

WHICH TO USE

SITUATION	CHOOSE
`value might be absent`	std::optional
`need to know why it failed`	std::expected
`rare, unrecoverable failure`	exception
`constructor failure`	exception
`hot path / no-exceptions builds`	std::expected

⚠ COMMON MISTAKES

- Dereferencing `*opt` without checking; that's UB when empty
- Using `std::optional<T&>`; it isn't allowed before C++26. Use a pointer or `std::reference_wrapper`
- Returning `std::optional<bool>` and testing `if (opt)`, which checks presence, not the bool

✓ TIPS & BEST PRACTICES

- Return `std::expected` from parsers and I/O helpers; callers see the error type in the signature
- Use `value_or` for defaults instead of `if/else`
- Mark functions returning optional/expected `[[nodiscard]]`

QUICK SUMMARY

- `std::optional<T>` — value or nothing
- `std::expected<T, E>` — value or error (C++23)
- `value_or` — fallback
- `and_then / transform` — chain without nesting

Assertions & Error Codes

CHECKS & CODES

Assertions catch programmer errors during development. `assert(cond)` aborts with a message when the condition is false and disappears when NDEBUG is defined, while `static_assert` checks at compile time. For OS-level errors, C APIs set `errno` and C++ wraps them in `std::error_code`. **Contracts** in C++26 add `pre`, `post`, and `contract_assert` as language features; verify compiler support before relying on them.

NOTES

- Release builds usually pass `-DNDEBUG`, removing `assert`
- Never put side effects inside `assert(...)`
- `std::error_code` = integer value + category (generic, system)
- Contracts (C++26) can be checked, observed, or ignored per build

ASSERT, STATIC_ASSERT, ERROR_CODE

```
#include <cassert>
#include <system_error>
#include <filesystem>

double average(std::span<const double> v) {
    assert(!v.empty() && "average of empty span");   // debug only
    return std::accumulate(v.begin(), v.end(), 0.0) / v.size();
}

static_assert(sizeof(void*) == 8, "64-bit only");
static_assert(std::is_trivially_copyable_v<Packet>);

std::error_code ec;
auto size = std::filesystem::file_size("missing.txt", ec);
if (ec) std::println("{}: {}", ec.value(), ec.message());
                                    // 2: No such file or directory
if (std::fopen("x", "r") == nullptr)
    std::println("{}", std::strerror(errno));
int safe_div(int a, int b)          // C++26 contracts
    pre (b != 0)
    post (r: r * b <= a);
```

CODE EXPLAINED

→ `assert(cond && "msg")` — the string makes the failure message readable

→ `static_assert` — compile-time; message optional since C++17

→ overloads taking `std::error_code&` — report errors instead of throwing

→ `ec.message()` — human-readable description

→ `pre` / `post` — C++26 function contracts; `r` names the return value

TOOLS

TOOL	WHEN CHECKED	DISABLED BY
`assert(c)`	run time	NDEBUG
`static_assert(c)`	compile time	never
`pre / post (C++26)`	run time	evaluation semantic
`contract_assert (C++26)`	run time	evaluation semantic
`std::error_code`	returned value	—

ERROR_CODE API

MEMBER	PURPOSE
`ec.value()`	integer code
`ec.message()`	text description
`ec.category()`	generic / system / custom
`if (ec)`	true when an error is set
`std::errc::…`	portable error enum

⚠ COMMON MISTAKES

- `assert(file.open())` — the call vanishes in release builds. Do the work outside, assert the result
- Using `assert` to validate user input; it's for bugs, not expected failures
- Reading `errno` after another call that may have overwritten it

✓ TIPS & BEST PRACTICES

- Keep assertions on in testing builds and run tests in both debug and release modes
- Compare portably: `ec == std::errc::no_such_file_or_directory`
- Check your compiler's contracts support before adopting `pre` / `post` in shared code

QUICK SUMMARY

- `assert` — debug-only run-time check
- `static_assert` — compile-time check
- `std::error_code` — non-throwing OS errors
- `pre / post` — C++26 contracts

Sheet 040

Defining & Calling Functions

declaration vs definition · prototypes · return · void · trailing return type · auto return · recursion

FUNCTIONS

A function has a **return type**, a **name**, a **parameter list**, and a body. It must be **declared** before it's called; a declaration without a body (a **prototype**) is enough, and the **definition** can come later or live in another `.cpp` file. `void` means no value is returned. The return type can also go after the parameters (**trailing return type**) or be deduced with `auto`.

RULES

- Declarations usually go in headers, definitions in `.cpp` files
- Falling off the end of a non-void function is UB (except `main`)
- `auto f()` deduces the return type from `return` statements
- Recursion: a function may call itself; each call gets its own locals

DECLARE, DEFINE, CALL

```
int add(int a, int b);                  // declaration (prototype)

int main() {
    int s = add(2, 3);                  // 5  — call
    greet("Ada");                       // Hello, Ada
}

int add(int a, int b) { return a + b; } // definition

void greet(std::string_view who) {      // no return value
    std::println("Hello, {}", who);
}
auto area(double w, double h) -> double { return w * h; } // trailing
auto half(int x) { return x / 2.0; }    // deduced: double

unsigned long long fact(unsigned n) {   // recursion
    return n <= 1 ? 1 : n * fact(n - 1);
}
fact(10);                               // 3628800
```

CODE EXPLAINED

- → prototype — lets `main` call `add` before its definition
- → `void` — the function returns nothing; `return;` may end it early
- → `-> double` — trailing return type, useful with templates
- → `auto half(...)` — return type deduced as `double`
- → `fact` — base case `n <= 1`, recursive case `n * fact(n - 1)`

PARTS OF A FUNCTION

PART	EXAMPLE
return type	`int`, `void`, `auto`
name	`add`
parameters	`(int a, int b)`
body	`{ return a + b; }`
specifiers	`inline`, `constexpr`, `noexcept`, `static`
attributes	`[[nodiscard]]`

RETURN FORMS

FORM	NOTES
`int f();`	classic
`auto f() -> int;`	trailing return type
`auto f();`	deduced from body
`decltype(auto) f();`	keeps references
`[[noreturn]] void f();`	never returns
`return {a, b};`	brace-init the return type

⚠ COMMON MISTAKES

- Calling a function before any declaration is visible: "'foo' was not declared in this scope"
- Forgetting `return` in a non-void function; enable `-Wreturn-type` (on with `-Wall`)
- Deep recursion without a base case overflows the stack; C++ doesn't guarantee tail-call optimization

✓ TIPS & BEST PRACTICES

- Keep functions short and single-purpose; name them with verbs (`loadConfig`, `parseLine`)
- Mark functions whose result must be used with `[[nodiscard]]`
- Return multiple values with a `struct` or `std::pair`, then unpack with structured bindings

QUICK SUMMARY

- `T name(params)` — declaration; add `{}` for a definition
- prototype — declare before use
- `auto f() -> T` — trailing / deduced return type
- recursion — base case plus self-call

Parameter Passing

HOW ARGUMENTS ARE PASSED

Parameters are **copies** by default (pass by value). A **reference** parameter (T&) aliases the caller's object so the function can modify it, and a **const reference** (`const T&`) avoids the copy while promising not to modify. **Pointers** (T*) also give access to the caller's object but can be null, which makes them a fit for optional arguments. The choice says a lot about intent.

RULE OF THUMB

- Cheap types (`int` , `double` , `string_view` , `span`): by value
- Read-only large types: `const T&`
- Must modify caller's object: `T&`
- Optional / nullable: `T*` ; taking ownership: by value + `std::move`

PASSING STYLES

```
void byValue(int x)             { x = 99; }   // caller unaffected
void byRef(int& x)              { x = 99; }   // caller changed
void byConstRef(const std::string& s) { std::println("{}", s.size()); }
void byPtr(int* p)              { if (p) *p = 99; }  // may be null

int n = 1;
byValue(n);      // n == 1
byRef(n);        // n == 99
byPtr(&n);       // n == 99
byPtr(nullptr);  // OK, no-op

// "sink": take by value, then move into place
class Person {
    std::string name_;
public:
    explicit Person(std::string name) : name_(std::move(name)) {}
};
Person p{"Ada"};                      // one construction, one move
```

CODE EXPLAINED

→ by value — the function gets its own copy

→ `int&` — the parameter **is** the caller's variable

→ `const std::string&` — no copy, read-only

→ `int*` — explicit `&n` at the call site; check for null

→ sink parameter + `std::move` — efficient ownership transfer

CHOOSING

WANT	PARAMETER
read small value	T
read large object	const T&
read string	std::string_view
read array / vector	std::span<const T>
modify caller's object	T&
optional object	T* (or optional<T>)
store a copy	T, then std::move

CALL SITE LOOKS

SIGNATURE	CALL
f(int)	f(n)
f(int&)	f(n) (can't pass a literal)
f(const int&)	f(5) OK
f(int*)	f(&n)
f(int&&)	f(5) / f(std::move(n))

⚠ COMMON MISTAKES

- Passing `std::vector` or `std::string` by value to read it; each call copies everything
- Trying to pass a literal or temporary to a non-const `T&` parameter; it won't compile
- Returning a reference or pointer to a local variable; it dangles as soon as the function returns

✓ TIPS & BEST PRACTICES

- Prefer return values over out-parameters; copy elision makes returning objects cheap
- Use `std::span` for "any contiguous sequence" parameters (arrays, vectors, `std::array`)
- Pass smart pointers only when the function deals with ownership; otherwise pass `T&` or `T*`

QUICK SUMMARY

- `T` — copy; good for cheap types and sinks
- `const T&` — read-only, no copy
- `T&` — modify the caller's object
- `T*` — nullable access

Overloading & Default Arguments

SAME NAME, DIFFERENT PARAMETERS

Overloading lets several functions share a name when their parameter lists differ in number or types. The compiler picks the best match at each call using **overload resolution**: exact match beats promotion, which beats conversion. If two candidates tie, the call is **ambiguous** and won't compile. **Default arguments** supply values for trailing parameters the caller omits.

RULES

- Return type alone can't distinguish overloads
- Defaults must be trailing: `f(int a, int b = 0)` OK, not `f(int a = 0, int b)`
- Specify defaults in the declaration (header) only
- The compiler encodes parameter types in symbol names (**name mangling**)

OVERLOADS & DEFAULTS

```
void print(int v)              { std::println("int {}", v); }
void print(double v)           { std::println("double {}", v); }
void print(std::string_view v) { std::println("text {}", v); }

print(42);          // int 42
print(3.14);        // double 3.14
print("hi");        // text hi
print(2.5f);        // double 2.5   (float → double promotion)
// print(42L);      // ERROR: ambiguous (long → int or double?)

void connect(std::string_view host, int port = 443, bool tls = true);
connect("example.com");              // port 443, tls true
connect("example.com", 8080);        // tls true
connect("example.com", 80, false);

void f(int);
void f(int*);
f(0);         // calls f(int)
f(nullptr);   // calls f(int*)
```

CODE EXPLAINED

→ three `print`s — chosen by argument type

→ `float → double` — promotion wins over conversion to `int`

→ `42L` — converting `long` to `int` or `double` ties: ambiguous

→ default arguments — filled in from the right

→ `nullptr` — always picks the pointer overload; `0` picks `int`

RESOLUTION RANKING

RANK	EXAMPLE
exact match	int → int
promotion	char/short → int, float → double
standard conversion	int → double, double → int
user-defined conversion	const char* → std::string
ellipsis ...	C varargs, last resort

VALID VS INVALID OVERLOADS

PAIR	VALID?
f(int) / f(double)	✔
f(int) / f(int, int)	✔
f(int&) / f(const int&)	✔
int f() / double f()	✖
f(int) / f(const int)	✖ (same)
f(int a = 1) / f()	✔ but f() is ambiguous

⚠ COMMON MISTAKES

- Overloading on `bool` and `std::string`: `f("text")` picks `bool` because pointer → bool is a standard conversion
- Repeating default arguments in both the declaration and the definition; it's a redefinition error
- Changing a default argument in a header without rebuilding callers; defaults are baked in at the call site

✓ TIPS & BEST PRACTICES

- Prefer distinct names (`parseInt`, `parseFloat`) when overloads would do different things
- Use `= delete` to block unwanted conversions: `void f(double) = delete;`
- Use `std::string_view` instead of separate `const char*` and `std::string` overloads

QUICK SUMMARY

- overloading — same name, different parameters
- best match — exact > promotion > conversion
- default args — trailing parameters only
- `nullptr` vs `0` — pointer vs int overload

Linkage, inline & static

inline · static functions · extern · extern "C" · anonymous namespace · internal vs external linkage

VISIBILITY ACROSS FILES

Linkage decides whether a name in one `.cpp` file refers to the same entity in another. Functions and globals have **external linkage** by default: visible program-wide, so they must be defined exactly once. `static` at namespace scope or an **anonymous namespace** gives **internal linkage**, private to the translation unit. `inline` allows identical definitions in many TUs, which is how header-only functions work.

KEYWORDS

- `extern int x;` — declaration only; defined elsewhere
- `static void helper()` — file-private (old style)
- `namespace { … }` — file-private (preferred)
- `inline` — "may be defined in several TUs", not a speed hint

ACROSS TRANSLATION UNITS

```
// config.h
extern int g_verbosity;                 // declared, not defined
inline int clampLevel(int v) {          // OK to define in a header
    return v < 0 ? 0 : (v > 3 ? 3 : v);
}
inline constexpr int kMaxLevel = 3;     // one shared constant

// config.cpp
int g_verbosity = 1;                    // the single definition

namespace {                             // internal linkage
    int parseLevel(std::string_view s) { return s.size(); }
}
static int s_calls = 0;                 // also internal (C style)
// C interop: no name mangling
extern "C" int sqlite3_open(const char*, sqlite3**);
extern "C" {
    void legacy_init();
}
```

CODE EXPLAINED

- → `extern` — tells the compiler the variable lives in another TU
- → `inline` function in a header — every TU gets the same definition; linker merges
- → `inline constexpr` — one constant instead of a copy per TU
- → anonymous namespace — names are invisible to other files
- → `extern "C"` — use C linkage to call or expose C functions

LINKAGE SUMMARY

DECLARATION	LINKAGE
`int f();`	external
`static int f();`	internal
`namespace { int f(); }`	internal
`inline int f() {…}`	external, multiple defs OK
`const int x = 1; (namespace)`	internal
`extern const int x;`	external

IN HEADERS

PUT IN HEADER	OK?
`declarations`	yes
`inline / constexpr functions`	yes
`templates`	yes
`inline constexpr variables`	yes
`non-inline function bodies`	no — multiple definition
`non-inline global variables`	no — multiple definition

⚠ COMMON MISTAKES

- Defining `int counter = 0;` in a header included by two files, causing "multiple definition of counter"
- Thinking `inline` makes code faster; the optimizer decides inlining on its own
- Using `static` inside a header for functions; each TU gets its own copy, bloating the binary and its static locals

✓ TIPS & BEST PRACTICES

- Put helpers used by one `.cpp` in an anonymous namespace
- Avoid mutable globals entirely; pass state explicitly or wrap it in a class
- Use `inline` variables (C++17) for header-only singletons and constants

QUICK SUMMARY

- external linkage — visible across TUs (default)
- `namespace {}` / `static` — file-private
- `inline` — allows definitions in headers
- `extern "C"` — C linkage, no mangling

constexpr & consteval Functions

COMPILE-TIME FUNCTIONS

A `constexpr` function **can** run at compile time when given constant arguments and is used where a constant is required; otherwise it runs normally at run time. A `consteval` function (C++20) **must** run at compile time, which makes it an "immediate function". Since C++20, `constexpr` code can use `try`, `virtual`, dynamic allocation that's freed before the end, and even `std::vector` and `std::string`.

DETECTING CONTEXT

- `if consteval { … }` (C++23) — branch taken during constant evaluation
- `std::is_constant_evaluated()` (C++20) — older equivalent
- `if constexpr (cond)` — compile-time branch on a constant
- `constexpr` functions are implicitly `inline`

CONSTEXPR & CONSTEVAL

```
constexpr int fib(int n) {
    return n < 2 ? n : fib(n - 1) + fib(n - 2);
}
constexpr int f10 = fib(10);          // 55, computed by compiler
static_assert(fib(10) == 55);

consteval int square(int x) { return x * x; }
constexpr int s = square(8);          // 64
// int bad = square(readInt());       // ERROR: not a constant
constexpr std::size_t countWords(std::string_view s) {
    std::vector<std::string_view> words;   // OK in C++20 constexpr
    for (auto w : s | std::views::split(' ')) words.emplace_back(w);
    return words.size();
}
static_assert(countWords("compile time strings") == 3);
template <class T> auto describe(T v) {
    if constexpr (std::is_integral_v<T>) return "integer";
    else return "other";
}
```

CODE EXPLAINED

- → `constexpr int f10` — forces compile-time evaluation
- → same `fib` at run time — `constexpr` functions are dual-use
- → `consteval` — calling it with a run-time value is an error
- → `std::vector` in `constexpr` — allowed if freed before evaluation ends
- → `if constexpr` — the discarded branch isn't instantiated

KEYWORDS

KEYWORD	EVALUATED	APPLIES TO
`constexpr`	compile or run time	functions, variables
`consteval`	compile time only	functions
`constinit`	static init at compile time	variables
`if constexpr`	compile-time branch	statements
`if consteval`	during const eval?	statements (C++23)

ALLOWED IN CONSTEXPR (C++20+)

FEATURE	SINCE
`loops, local variables`	C++14
`try / catch blocks`	C++20
`virtual calls`	C++20
`new / delete (freed in time)`	C++20
`std::vector / std::string`	C++20
`static / thread_local locals`	C++23

⚠ COMMON MISTAKES

- Assuming a `constexpr` call always happens at compile time; assign it to a `constexpr` variable to force it
- Calling a non-constexpr function inside a constexpr function used in a constant expression
- Returning a `std::vector` from a constexpr function into a `constexpr` variable; allocations can't survive constant evaluation

✓ TIPS & BEST PRACTICES

- Make pure helper functions `constexpr` by default; it costs nothing at run time
- Use `consteval` for compile-time validation helpers such as format-string checking
- Move lookup tables to compile time with `constexpr std::array` built by a constexpr function

QUICK SUMMARY

- `constexpr` function — usable at compile or run time
- `consteval` — compile time only (C++20)
- `if constexpr` — discard branches in templates
- `static_assert(f(x) == y)` — test at compile time

Lambdas — Basics

[](){} · capture by value / reference · [=] [&] · [this] · mutable · explicit return type

ANONYMOUS FUNCTIONS

A **lambda** is an unnamed function object written inline: `[captures](params) { body }`. The **capture list** decides which outside variables the lambda can use and whether it copies them (`[x]`, `[=]`) or refers to them (`[&x]`, `[&]`). Lambdas are the standard way to pass custom behavior to algorithms like `std::sort` or `std::ranges::count_if`.

NOTES

- Each lambda has its own unique, unnamed type: store with `auto`
- Captured copies are `const` inside the body unless `mutable`
- A capture-less lambda converts to a plain function pointer
- Global and `static` variables don't need capturing

LAMBDA BASICS

```
auto add = [](int a, int b) { return a + b; };
add(2, 3);                                      // 5

std::vector<int> v{5, 2, 9, 1};
std::ranges::sort(v, [](int a, int b) { return a > b; }); // 9 5 2 1

int limit = 4;
auto big = std::ranges::count_if(v, [limit](int x) { return x > limit; }); // 2

int total = 0;
std::ranges::for_each(v, [&total](int x) { total += x; }); // total 17

auto counter = [n = 0]() mutable { return ++n; };
counter(); counter();                           // 1, 2

auto toDouble = [](int x) -> double { return x / 2; };  // explicit return
toDouble(5);                                    // 2.0 (int division first)
```

CODE EXPLAINED

- → `[]` — captures nothing
- → `[limit]` — copies `limit` into the lambda
- → `[&total]` — refers to the caller's `total`
- → `mutable` — allows modifying the lambda's own copies
- → `-> double` — explicit return type
- → `[this]` — access members of the enclosing object

CAPTURE FORMS

CAPTURE	MEANING
`[]`	nothing
`[x]`	x by value (copy)
`[&x]`	x by reference
`[=]`	everything used, by value
`[&]`	everything used, by reference
`[this]`	the enclosing object (pointer)

SYNTAX PIECES

PIECE	OPTIONAL?
`[captures]`	required
`(params)`	optional if empty
`mutable`	optional
`noexcept`	optional
`-> ReturnType`	optional (deduced)
`{ body }`	required

⚠ COMMON MISTAKES

- Capturing locals by reference in a lambda that outlives them (stored callback, thread); it dangles
- Expecting `[=]` to copy `*this` members; it captures `this` (deprecated implicitly in C++20). Use `[this]` or `[*this]`
- Forgetting `mutable` and getting "assignment of read-only variable" on a captured copy

✓ TIPS & BEST PRACTICES

- List captures explicitly (`[&total]`) rather than `[&]` in longer lambdas
- Name non-trivial lambdas with `auto` variables so call sites read clearly
- For async callbacks, capture by value or capture a `shared_ptr` to keep data alive

QUICK SUMMARY

- `[cap](args){ body }` — lambda syntax
- `[x] / [&x]` — capture by copy / reference
- `mutable` — modify captured copies
- algorithms — main use: custom predicates and comparators

Lambdas — Advanced

generic lambdas (auto) · init capture [x = std::move(y)] · template lambdas []<typename T> · IIFE · recursive lambdas (this auto&)

BEYOND THE BASICS

Lambdas grew steadily after C++11. **Generic lambdas** take `auto` parameters and act like templates (C++14). **Init captures** create new variables in the capture list, including moving objects in (C++14). **Template lambdas** name their type parameters explicitly (C++20), and **deducing this** (C++23) gives a lambda access to itself, so it can recurse without `std::function`.

NOTES

- `auto` parameters → one template per call type
- `[p = std::move(ptr)]` moves a `unique_ptr` into the lambda
- **IIFE**: define and call immediately for complex initialization
- Lambdas in unevaluated contexts and as default template args since C++20

ADVANCED LAMBDAS

```
auto twice = [](auto x) { return x + x; };      // generic
twice(4); twice(1.5);   // 8, 3.0

auto ptr = std::make_unique<Job>();
auto task = [job = std::move(ptr)] { job->run(); };   // init capture

auto sum = []<typename T>(const std::vector<T>& v) {  // template lambda
    return std::accumulate(v.begin(), v.end(), T{});
};
const auto config = [&] {                       // IIFE
    Config c;
    if (debug) c.level = 3;
    return c;
}();                                            // note the ()
auto fact = [](this auto self, int n) -> long { // deducing this
    return n <= 1 ? 1 : n * self(n - 1);
};
fact(5);           // 120
```

CODE EXPLAINED

- → `[](auto x)` — generic; works for any type with `+`
- → `[job = std::move(ptr)]` — creates a member `job` owning the pointer
- → `[]<typename T>` — explicit template parameter for the vector element
- → `[&] { … }()` — IIFE lets `config` be `const`
- → `this auto self` — the lambda receives itself, enabling recursion

FEATURE TIMELINE

FEATURE	SINCE
basic lambdas	C++11
generic auto params	C++14
init capture [x = expr]	C++14
constexpr lambdas, [*this]	C++17
template lambdas []<class T>	C++20
deducing this	C++23

PATTERNS

PATTERN	USE
IIFE	init a const with complex logic
overloaded{…}	visitor for `std::variant`
move capture	hand ownership to a callback
recursive lambda	tree walks, DFS helpers

⚠ COMMON MISTAKES

- Forgetting the trailing `()` on an IIFE, so `config` becomes the lambda instead of its result
- Copying a lambda that holds a `unique_ptr`; it's move-only, so `std::function` can't store it (use `std::move_only_function`)
- Using `std::function` for recursion; it adds heap allocation and indirect calls. Use deducing this

✓ TIPS & BEST PRACTICES

- Prefer generic lambdas for small predicates used with multiple types
- Use IIFE to keep variables `const` when initialization needs branches or loops
- Combine lambdas with `std::visit` via an `overloaded` helper for variant dispatch

QUICK SUMMARY

- `[](auto x)` — generic lambda
- `[x = std::move(y)]` — init capture
- `[]<typename T>` — template lambda (C++20)
- `this auto self` — recursive lambda (C++23)

Callables & std::function

THINGS YOU CAN CALL

A **callable** is anything you can invoke with `()`: free functions, function pointers, lambdas, objects with `operator()`, and member function pointers. `std::function<R(Args...)>` is a type-erased wrapper that can hold any copyable callable with a matching signature, at the cost of a possible heap allocation and an indirect call. `std::invoke` calls any callable uniformly, including member pointers.

CHOOSING A TYPE

- Template parameter (`auto f` / `F&&`) — fastest, inlinable
- `std::function` — store heterogeneous callbacks
- `std::move_only_function` (C++23) — for move-only lambdas
- `std::function_ref` (C++26) — non-owning, no allocation

CALLABLES

```cpp
int add(int a, int b) { return a + b; }

int (*fp)(int, int) = &add;           // function pointer
fp(2, 3);                             // 5

std::function<int(int, int)> op = add;
op = [](int a, int b) { return a * b; };
op(2, 3);                             // 6

struct Account { double balance() const { return 42; } };
double (Account::*mp)() const = &Account::balance;  // member fn pointer
Account acct;
(acct.*mp)();                         // 42
std::invoke(&Account::balance, acct); // 42, uniform syntax

auto addTen = std::bind_front(add, 10);
addTen(5);                            // 15
```

CODE EXPLAINED

- → `int (*fp)(int, int)` — pointer to a function taking two ints
- → `std::function` — holds any matching callable; check emptiness with `if`
- → `.*` — call a member function through a member pointer
- → `std::invoke` — works for functions, lambdas, and member pointers
- → `std::bind_front` — fix leading arguments (C++20)

CALLABLE KINDS

KIND	EXAMPLE
function	`int f(int);`
function pointer	`int (*p)(int) = f;`
lambda	`[](int x){ return x; }`
functor	`struct F { int operator()(int); };`
member fn pointer	`&Account::balance`
std::function	`std::function<int(int)>`

WRAPPERS

WRAPPER	OWNS?	COPYABLE TARGET REQUIRED?
std::function	yes	yes
std::move_only_function	yes	no (C++23)
std::function_ref	no	no (C++26)
template param	by value	no

⚠ COMMON MISTAKES

- Calling an empty `std::function` ; it throws `std::bad_function_call`
- Using `std::function` parameters in hot loops when a template parameter would allow inlining
- Forgetting the parentheses in `(obj.*mp)()` ; `obj.*mp()` parses differently

✓ TIPS & BEST PRACTICES

- Accept callables as `auto&& f` or a constrained template (`std::invocable<int> auto f`) in library code
- Use `std::bind_front` or a lambda instead of the older `std::bind`
- Store callbacks as `std::move_only_function` when they capture move-only state

QUICK SUMMARY

- callable — anything invocable with `()`
- `std::function<R(A...)>` — type-erased callback holder
- `std::invoke` — call any callable uniformly
- `std::bind_front` — partial application

Attributes & Specifiers

[[nodiscard]] · [[maybe_unused]] · [[deprecated]] · [[noreturn]] · [[assume]] · = delete on functions

TELLING THE COMPILER MORE

Attributes (`[[...]]`) attach standard hints to declarations: they let the compiler warn about misuse, suppress false-positive warnings, or optimize. **Deleted functions** (`= delete`) make a call a compile error, which is useful to block dangerous conversions or forbid copying. Unknown attributes are ignored, so vendor-specific ones such as `[[gnu::always_inline]]` are safe to write.

NOTES

- Attributes go before the declaration or after the name for variables
- `[[nodiscard("reason")]]` takes a message (C++20)
- `= delete` works on any function, not just special members
- `[[assume(expr)]]` (C++23) — UB if `expr` is false

ATTRIBUTES & = DELETE

```cpp
[[nodiscard]] std::expected<int, Err> parse(std::string_view);
parse("42");                        // warning: ignoring return value

[[deprecated("use connect2()")]] void connect();
connect();                          // warning: 'connect' is deprecated

[[noreturn]] void fatal(std::string_view msg);   // calls std::exit

void handler([[maybe_unused]] int signal) {}   // no unused warning

int scale(int x) { [[assume(x > 0)]]; return x / 4; }
void setPercent(int);
void setPercent(double) = delete;   // block implicit double → int
// setPercent(0.5);                 // ERROR: use of deleted function
```

CODE EXPLAINED

- → `[[nodiscard]]` — warns when a result is ignored
- → `[[deprecated("…")]]` — warns at every call site with your message
- → `[[noreturn]]` — the function never returns normally
- → `[[maybe_unused]]` — silences unused-variable/parameter warnings
- → `[[assume]]` — optimization hint; wrong assumptions are UB
- → `= delete` — calling the overload is a compile error

STANDARD ATTRIBUTES

ATTRIBUTE	SINCE	EFFECT
`[[noreturn]]`	C++11	never returns
`[[deprecated]]`	C++14	warn on use
`[[fallthrough]]`	C++17	intentional switch fall-through
`[[nodiscard]]`	C++17	warn if result ignored
`[[maybe_unused]]`	C++17	silence unused warnings
`[[assume(e)]]`	C++23	optimizer assumption

USES OF = DELETE

DECLARATION	PREVENTS
`T(const T&) = delete;`	copying
`void f(double) = delete;`	implicit conversions
`template f(T*) = delete`	specific instantiations

⚠ COMMON MISTAKES

- Using `[[assume]]` as a check; it's not verified, and a false assumption is UB. Use `assert` for checks
- Putting `[[nodiscard]]` on functions whose result is commonly ignored, training people to ignore warnings
- Declaring copy operations private-and-unimplemented (C++98 style) instead of `= delete`

✓ TIPS & BEST PRACTICES

- Mark factory functions, error-returning functions, and `empty()`-style queries `[[nodiscard]]`
- Use `[[deprecated]]` for a release before removing an API
- Delete overloads to stop `bool` / `double` from sneaking into integer parameters

QUICK SUMMARY

- `[[nodiscard]]` — don't ignore the result
- `[[deprecated]]` — migration warning
- `[[maybe_unused]]` / `[[noreturn]]` — silence / document
- `= delete` — make specific calls ill-formed

Sheet 049

C-Style Arrays & std::array

int a[5] · array-to-pointer decay · std::array<T, N> · at() · std::size · std::to_array · multidimensional

FIXED-SIZE ARRAYS

A **C-style array** `T a[N]` is a contiguous block of N elements whose size is fixed at compile time. It **decays** to a pointer to its first element in most expressions, losing its size, and can't be copied or returned by value. `std::array<T, N>` (`<array>`) wraps the same storage with zero overhead but behaves like a proper value: it copies, compares, knows its `size()`, and works with algorithms.

NOTES

- Indexes run `0` … `N-1`; out-of-bounds access is UB
- `std::size(a)` works for both kinds (C++17)
- `std::array` can be `constexpr` and returned from functions
- Uninitialized local arrays hold indeterminate values; use `{}`

C ARRAY VS STD::ARRAY

```cpp
int raw[5] = {1, 2, 3};          // 1 2 3 0 0
int zeros[100]{};                // all 0
std::size(raw);                  // 5
int* p = raw;                    // decays: size info lost
void takes(int a[]);             // really: void takes(int* a)

std::array<int, 4> arr{10, 20, 30, 40};
arr.size();                      // 4
arr[1];                          // 20 (unchecked)
arr.at(9);                       // throws std::out_of_range
arr.front(); arr.back();         // 10, 40
arr.fill(0);                     // 0 0 0 0
auto copy = arr;                 // real copy
auto [a, b, c, d] = arr;         // structured bindings
auto deduced = std::to_array({1, 2, 3});   // std::array<int, 3>
int grid[3][4]{};                // 3 rows × 4 cols
std::array<std::array<int, 4>, 3> grid2{};
```

CODE EXPLAINED

- → `{1, 2, 3}` — remaining elements are zero-initialized
- → `int* p = raw` — array-to-pointer decay; `sizeof(p)` is pointer size
- → `int a[]` parameter — just a pointer; pass `std::span` instead
- → `arr.at()` — bounds-checked access
- → `std::to_array` — deduces element type and size (C++20)

C ARRAY VS STD::ARRAY

FEATURE	T[N]	STD::ARRAY
knows size	only in scope	✔ size()
copy / assign	✖	✔
compare with ==	✖ (pointers)	✔
return by value	✖	✔
bounds-checked at()	✖	✔
overhead	none	none

STD::ARRAY MEMBERS

MEMBER	PURPOSE
size() / empty()	element count
[] / at()	access
front() / back()	ends
data()	raw pointer
fill(v)	set every element
swap(o)	exchange contents

⚠ COMMON MISTAKES

- Using `sizeof(arr) / sizeof(arr[0])` on a parameter; it's a pointer there, so the result is wrong. Use `std::size` or `std::span`
- Writing past the end of an array; the compiler won't stop you and it corrupts memory
- Forgetting `std::array` needs its size as a template argument: `std::array<int> a;` doesn't compile

✓ TIPS & BEST PRACTICES

- Default to `std::array` for fixed-size data and `std::vector` for everything else
- Pass arrays to functions as `std::span<const T>` to keep the size
- Use `std::array` with `constexpr` for compile-time lookup tables

QUICK SUMMARY

- `T a[N]` — C array; decays to pointer
- `std::array<T, N>` — fixed size, value semantics
- `at()` — checked access
- `std::to_array` — deduce type and size

std::vector — Basics

THE DEFAULT CONTAINER

`std::vector<T>` is a **dynamic array**: contiguous memory that grows as needed. Appending is amortized O(1) because the vector reserves extra **capacity** and reallocates (usually doubling) only when it's full. Contiguous storage makes it cache-friendly and compatible with C APIs via `data()`. When in doubt, use a vector.

SIZE VS CAPACITY

- `size()` — elements stored
- `capacity()` — elements that fit before reallocating
- `reserve(n)` — grow capacity, size unchanged
- Reallocation invalidates all pointers, references, and iterators

BUILDING & READING A VECTOR

```cpp
#include <vector>

std::vector<int> a;                  // empty
std::vector<int> b{1, 2, 3};         // 1 2 3
std::vector<int> c(4, 7);            // 7 7 7 7
std::vector<std::string> names(3);   // 3 empty strings

a.push_back(10);                     // copy/move in
a.emplace_back(20);                  // construct in place
a.reserve(100);                      // capacity >= 100, size 2

b[0];                                // 1 (unchecked)
b.at(5);                             // throws std::out_of_range
b.front(); b.back();                 // 1, 3
b.size(); b.empty();                 // 3, false

for (const auto& x : b) std::print("{} ", x);   // 1 2 3
for (std::size_t i = 0; i < b.size(); ++i) {}
int* raw = b.data();                 // pointer to contiguous storage
```

CODE EXPLAINED

- → `c(4, 7)` vs `b{4, 7}` — parentheses: count + value; braces: elements
- → `emplace_back` — forwards arguments to the element constructor
- → `reserve` — avoids repeated reallocation when the final size is known
- → `at()` — checked; `[]` — unchecked
- → `data()` — interop with C functions expecting `T*`

CONSTRUCTION

EXPRESSION	RESULT
`vector<int> v;`	empty
`vector<int> v(5);`	five 0s
`vector<int> v(5, 2);`	five 2s
`vector<int> v{5, 2};`	5, 2
`vector v{1.5, 2.5};`	CTAD → vector<double>
`vector<int> v(it1, it2);`	copy a range

ACCESS & SIZE

MEMBER	COMPLEXITY
`[] / at()`	O(1)
`push_back / emplace_back`	amortized O(1)
`size() / empty()`	O(1)
`front() / back()`	O(1)
`reserve(n)`	O(n) once

⚠ COMMON MISTAKES

- Holding a pointer or reference to an element, then calling `push_back`; reallocation leaves it dangling
- Calling `v.reserve(10); v[3] = 1;` — reserve doesn't create elements, so this is UB. Use `resize`
- Using `std::vector<bool>`; it's a packed specialization whose elements are proxies, not `bool&`

✓ TIPS & BEST PRACTICES

- Call `reserve()` before a loop of `push_back` when you know roughly how many elements are coming
- Prefer `emplace_back(args...)` for types with multi-argument constructors
- Use `std::vector<char>` or `std::bitset` / `std::vector<std::uint8_t>` instead of `vector<bool>` when you need real references

QUICK SUMMARY

- `std::vector<T>` — contiguous, growable array
- `push_back / emplace_back` — append
- `size / capacity / reserve` — growth control
- reallocation — invalidates pointers and iterators

std::vector — Operations

CHANGING A VECTOR

Inserting or erasing anywhere except the end shifts every following element, so those operations are **O(n)**. The **erase-remove idiom**, now wrapped by `std::erase` / `std::erase_if` (C++20), deletes all matching elements in one pass. Any operation that changes size can invalidate iterators: knowing which ones prevents the most common vector bug.

INVALIDATION

- Reallocation (growth past capacity) → **all** iterators invalid
- `insert` / `erase` → iterators at or after the point invalid
- `pop_back` → only the last element's iterators
- `clear()` keeps capacity; `shrink_to_fit()` may release it

INSERT, ERASE, RESIZE

```
std::vector<int> v{1, 2, 3, 4, 5, 6};

v.insert(v.begin() + 1, 99);          // 1 99 2 3 4 5 6
v.insert(v.end(), {7, 8});            // ... 6 7 8
v.erase(v.begin());                   // 99 2 3 4 5 6 7 8
v.erase(v.begin(), v.begin() + 2);    // 3 4 5 6 7 8
v.pop_back();                         // 3 4 5 6 7

std::erase(v, 5);                     // 3 4 6 7     (C++20)
std::erase_if(v, [](int x) { return x % 2; });   // 4 6
v.resize(4);                          // 4 6 0 0
v.resize(2);                          // 4 6
v.assign(3, 9);                       // 9 9 9
v.clear();                            // size 0, capacity kept
v.shrink_to_fit();                    // non-binding request
for (auto it = v.begin(); it != v.end(); )       // erase in a loop
    it = (*it < 0) ? v.erase(it) : it + 1;
```

CODE EXPLAINED

- → `insert(pos, val)` — inserts before `pos`
- → `erase(first, last)` — removes the half-open range
- → `std::erase_if` — erase-remove in one call, returns count removed
- → `resize(n)` — grows with value-initialized elements or truncates
- → erase loop — use the iterator `erase` returns

MUTATORS

MEMBER	COST
`push_back / pop_back`	O(1) amortized
`insert(pos, x)`	O(n)
`erase(pos)`	O(n)
`resize(n)`	O(n)
`clear()`	O(n) destructors
`swap(other)`	O(1)

OLD VS NEW REMOVAL

BEFORE C++20	C++20
`v.erase(remove(…), v.end())`	std::erase(v, val)
`v.erase(remove_if(…), v.end())`	std::erase_if(v, pred)
`manual loop`	either of the above

⚠ COMMON MISTAKES

- `for (auto it = v.begin(); it != v.end(); ++it) if (bad) v.erase(it);` — `it` is invalid after erase. Use the returned iterator
- Calling `std::remove` without `erase`; elements are only moved, the size doesn't change
- Inserting at the front of a large vector in a loop, which is O(n²); use `std::deque` or build reversed

✓ TIPS & BEST PRACTICES

- Use `std::erase_if` for every "remove all matching" task
- If order doesn't matter, erase by swapping with the last element then `pop_back` (O(1))
- Use `v.insert(v.end(), other.begin(), other.end())` or `append_range` (C++23) to concatenate

QUICK SUMMARY

- `insert / erase` — O(n) middle edits
- `std::erase_if` — remove matching elements (C++20)
- `resize / clear / shrink_to_fit` — size control
- invalidation — use the iterator `erase` returns

std::pair & std::tuple

GROUPING VALUES

`std::pair<A, B>` holds exactly two values in `.first` and `.second`; maps use it for key/value entries. `std::tuple<Ts...>` (`<tuple>`) holds any fixed number of values, accessed by index or type with `std::get`. **Structured bindings** (C++17) unpack both into named variables. For anything with meaning, a small named `struct` is usually clearer than a tuple.

NOTES

- Both compare lexicographically with `==`, `<`, `<=>`
- CTAD: `std::pair p{1, 2.5};` deduces `pair<int, double>`
- `std::tie(a, b) = f();` assigns into existing variables
- `std::apply(f, tup)` calls `f` with the tuple's elements as arguments

PAIR, TUPLE, STRUCTURED BINDINGS

```cpp
#include <tuple>
#include <utility>

std::pair<std::string, int> p{"Ada", 36};
p.first;  p.second;                    // "Ada", 36
auto q = std::make_pair(1, 2.5);       // pair<int, double>

std::tuple<int, std::string, double> t{7, "seven", 7.0};
std::get<0>(t);                        // 7
std::get<std::string>(t);              // "seven" (by type)

auto [name, age] = p;                  // structured bindings
auto& [id, label, val] = t;            // references into t

std::tuple<int, int> minmax(std::span<const int> v);
auto [lo, hi] = minmax(data);
int a, b;
std::tie(a, std::ignore) = minmax(data);   // skip one value
std::apply([](int x, auto& s, double d) { /* ... */ }, t);
```

CODE EXPLAINED

→ `.first` / `.second` — pair members

→ `std::get<0>` — compile-time index; `std::get<T>` works if T is unique

→ `auto [name, age]` — copies; `auto&` binds references

→ `std::tie` + `std::ignore` — assign into existing variables

→ `std::apply` — unpack a tuple into a function call

HELPERS

HELPER	PURPOSE
`std::make_pair / make_tuple`	build with deduction
`std::get<I> / get<T>`	access element
`std::tie(a, b)`	tuple of references
`std::ignore`	placeholder in tie
`std::tuple_cat`	concatenate tuples
`std::apply(f, t)`	call f with elements

TUPLE VS STRUCT

ASPECT	TUPLE	STRUCT
`field names`	no (indexes)	yes
`quick ad-hoc returns`	good	more typing
`readability later`	poor	good
`structured bindings`	✔	✔

⚠ COMMON MISTAKES

- Returning `std::tuple<int, int, int>` from public APIs; callers can't tell which int is which. Return a named struct
- `std::get<int>(t)` when the tuple has two `int` elements; it doesn't compile
- Expecting `auto [x, y] = p;` to alias `p`; it's a copy unless you write `auto&`

✓ TIPS & BEST PRACTICES

- Use structured bindings with maps: `for (const auto& [k, v] : m)`
- Use `std::tie(a.x, a.y) < std::tie(b.x, b.y)` for quick lexicographic comparisons
- Aggregates (plain structs) also support structured bindings with no extra code

QUICK SUMMARY

- `std::pair` — two values, `.first` / `.second`
- `std::tuple` — N values, `std::get<I>`
- `auto [a, b] = …` — structured bindings
- `std::tie / std::apply` — assign / unpack

std::map & std::multimap

operator[] · insert · try_emplace · insert_or_assign · find · contains · erase · ordered iteration

ORDERED KEY/VALUE STORE

`std::map<K, V>` stores unique keys in **sorted order**, usually as a red-black tree, so lookup, insert, and erase are **O(log n)** and iteration visits keys in order. `std::multimap` allows duplicate keys. Keys are compared with `std::less<K>` (<) by default; pass a custom comparator for other orderings. Elements are `std::pair<const K, V>`.

LOOKUP CAVEAT

- `m[key]` **inserts** a default value if the key is missing
- `m.at(key)` throws `std::out_of_range` if missing
- `m.contains(key)` (C++20) checks without inserting
- `m.find(key)` returns an iterator or `m.end()`

MAP OPERATIONS

```cpp
#include <map>

std::map<std::string, int> stock{{"apple", 5}, {"pear", 2}};

stock["kiwi"] = 7;                      // insert or overwrite
stock["plum"];                          // inserts plum → 0 (!)
stock.at("apple");                      // 5
stock.contains("fig");                  // false

auto [it, inserted] = stock.insert({"apple", 9});    // no overwrite
inserted;                               // false; apple still 5
stock.try_emplace("fig", 3);            // inserts only if absent
stock.insert_or_assign("pear", 4);      // overwrite
if (auto f = stock.find("kiwi"); f != stock.end())
    std::println("{} → {}", f->first, f->second);    // kiwi → 7
stock.erase("plum");                    // by key
for (const auto& [fruit, n] : stock)    // sorted: apple fig kiwi pear
    std::print("{}:{} ", fruit, n);
auto lo = stock.lower_bound("b");       // first key >= "b" → fig
```

CODE EXPLAINED

- → `operator[]` — creates a default value when the key is absent
- → `insert` — returns `{iterator, bool}`; never overwrites
- → `try_emplace` — constructs the value only if the key is new
- → `insert_or_assign` — always ends with your value stored
- → `lower_bound` — ordered search, handy for ranges of keys

MEMBERS

MEMBER	PURPOSE
`m[k]`	access / insert default
`at(k)`	access, throws if missing
`find(k)`	iterator or end()
`contains(k)`	bool (C++20)
`erase(k)`	remove, returns count
`lower_bound / upper_bound`	ordered range queries

MULTIMAP & CUSTOMIZING

CODE	EFFECT
`std::multimap<K, V>`	duplicate keys allowed
`mm.equal_range(k)`	all values for k
`std::map<K, V, std::greater<>>`	descending order
`std::map<std::string, V, std::less<>>`	heterogeneous lookup
`m.extract(k)`	move node out (C++17)

⚠ COMMON MISTAKES

- Using `m[key]` to check whether a key exists; it inserts it. Use `contains` or `find`
- Calling `m[key]` on a `const std::map`; there's no const `operator[]`. Use `at` or `find`
- Modifying a key through an iterator; keys are `const` because changing them would break ordering

✓ TIPS & BEST PRACTICES

- Use `std::less<>` so `find("literal")` doesn't build a temporary `std::string`
- Prefer `try_emplace` for "insert if missing" with expensive values
- Reach for `std::unordered_map` when you don't need ordering and want O(1) average lookup

QUICK SUMMARY

- `std::map` — sorted, unique keys, O(log n)
- `m[k]` — inserts default if missing
- `contains / find / at` — non-inserting lookup
- `try_emplace / insert_or_assign` — explicit insert intent

std::unordered_map

HASH TABLE

`std::unordered_map<K, V>` stores key/value pairs in a **hash table**: average **O(1)** lookup, insert, and erase, with no ordering. Keys need a hash function (`std::hash<K>`) and ==. The table is an array of **buckets**; when the **load factor** (elements ÷ buckets) passes `max_load_factor()`, it **rehashes**, which invalidates iterators. The interface mirrors `std::map`.

NOTES

- Iteration order is unspecified and changes after rehash
- Worst case is O(n) when many keys collide
- `reserve(n)` pre-sizes buckets to avoid rehashing
- Standard `std::hash` exists for integers, pointers, strings, `string_view`

UNORDERED_MAP + CUSTOM HASH

```cpp
#include <unordered_map>

std::unordered_map<std::string, int> wordCount;
for (const auto& w : words) ++wordCount[w];     // count occurrences
wordCount.reserve(10'000);                       // avoid rehash
wordCount.contains("the");                       // true / false
wordCount.load_factor();                         // e.g. 0.73

struct Point { int x, y; bool operator==(const Point&) const = default; };
struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        auto h = std::hash<int>{}(p.x);
        auto g = std::hash<int>{}(p.y);
        return h ^ (g + 0x9e3779b9 + (h << 6) + (h >> 2));
    }
};
std::unordered_map<Point, std::string, PointHash> labels;
labels[{1, 2}] = "A";
template <> struct std::hash<Point> : PointHash {};  // or specialize
std::unordered_set<Point> pts;                       // no 2nd arg now
```

CODE EXPLAINED

- → `++wordCount[w]` — missing keys start at 0, so counting is one line
- → `reserve` — sets bucket count for the expected size
- → `operator== = default` — needed alongside the hash
- → `PointHash` — combines member hashes (boost-style mixing)
- → specializing `std::hash` — then no third template argument is needed

MAP VS UNORDERED_MAP

ASPECT	MAP	UNORDERED_MAP
structure	balanced tree	hash table
lookup	O(log n)	O(1) avg, O(n) worst
ordered iteration	✔	✖
key needs	<	hash + ==
memory	node per element	buckets + nodes

BUCKET INTERFACE

MEMBER	PURPOSE
bucket_count()	number of buckets
load_factor()	size / bucket_count
max_load_factor(f)	rehash threshold
rehash(n)	at least n buckets
reserve(n)	room for n elements

⚠ COMMON MISTAKES

- Using a type with no `std::hash` specialization as a key, which causes a long template error
- Writing a hash that ignores some fields used by `==`; equal keys must have equal hashes
- Keeping iterators across inserts; a rehash invalidates them all

✓ TIPS & BEST PRACTICES

- Call `reserve()` up front when you know the approximate element count
- Use transparent hashing (`is_transparent` + `std::equal_to<>`) to look up `string_view` keys without allocating
- For hot paths, consider open-addressing tables like `absl::flat_hash_map` or `boost::unordered_flat_map`

QUICK SUMMARY

- `std::unordered_map` — hash table, O(1) average
- `std::hash<K>` + `==` — key requirements
- `reserve` — pre-size to avoid rehash
- no ordering — use `std::map` when order matters

Sets

UNIQUE COLLECTIONS

A **set** stores unique keys with no associated value. `std::set` keeps them **sorted** (tree, O(log n)); `std::unordered_set` uses a **hash table** (O(1) average, no order). `std::multiset` / `unordered_multiset` allow duplicates. Sets are the right tool for membership tests, deduplication, and ordered unique collections. Elements are `const` once inserted.

NOTES

- `insert` returns `{iterator, bool}`; `false` means it was already there
- `contains(x)` (C++20) is the clearest membership test
- Change an element by `extract`-ing it, editing, and reinserting
- Set operations need **sorted** ranges: `std::set_union`, etc.

SET & UNORDERED_SET

```cpp
#include <set>
#include <unordered_set>

std::set<int> s{5, 1, 3, 1};           // 1 3 5  (dup dropped)
auto [it, added] = s.insert(3);        // added == false
s.insert(4);                           // 1 3 4 5
s.erase(1);                            // 3 4 5
s.contains(4);                         // true
*s.begin(); *s.rbegin();               // 3, 5 (min, max)
s.lower_bound(4);                      // iterator to 4

std::unordered_set<std::string> seen;
for (const auto& w : words)
    if (!seen.insert(w).second) std::println("dup: {}", w);
std::set<std::string, std::greater<>> desc{"b", "a", "c"};  // c b a
auto byLen = [](const std::string& a, const std::string& b) {
    return a.size() < b.size(); };
std::set<std::string, decltype(byLen)> lens(byLen);  // custom compare
std::vector<int> v{3, 1, 3, 2};
std::set<int> uniq(v.begin(), v.end()); // dedupe + sort: 1 2 3
std::multiset<int> ms{1, 1, 2};  ms.count(1);   // 2
```

CODE EXPLAINED

- → duplicates in the initializer are ignored
- → `insert(...).second` — tells you whether the value was new
- → `*begin()` / `*rbegin()` — smallest and largest in a sorted set
- → `std::greater<>` — reverse order
- → custom comparator — elements equal under it count as duplicates

WHICH SET

TYPE	ORDER	DUPES	LOOKUP
`std::set`	sorted	no	O(log n)
`std::multiset`	sorted	yes	O(log n)
`std::unordered_set`	none	no	O(1) avg
`std::flat_set`	sorted	no	O(log n), contiguous

SET ALGORITHMS (SORTED RANGES)

ALGORITHM	RESULT
`std::ranges::set_union`	A ∪ B
`std::ranges::set_intersection`	A ∩ B
`std::ranges::set_difference`	A − B
`std::ranges::includes`	A ⊇ B ?

⚠ COMMON MISTAKES

- A comparator that isn't a strict weak ordering (e.g. using `<=`) gives undefined behavior
- Trying to modify `*it` in a set; elements are const. Erase and reinsert, or use `extract`
- Running `std::set_union` on unsorted vectors; the output is wrong

✓ TIPS & BEST PRACTICES

- Use `std::unordered_set` for fast "have I seen this?" checks
- To dedupe a vector in place, `std::ranges::sort` then `v.erase(std::ranges::unique(v).begin(), v.end())`
- Use `std::flat_set` (C++23) for small, read-mostly sets for better cache behavior

QUICK SUMMARY

- `std::set` — sorted unique keys
- `std::unordered_set` — hashed unique keys
- `contains / insert().second` — membership tests
- `std::multiset` — duplicates allowed

std::deque & std::list

OTHER SEQUENCES

`std::deque` (double-ended queue) supports O(1) insertion at **both ends** and O(1) random access, storing elements in fixed-size chunks rather than one block. `std::list` is a **doubly linked list**: O(1) insert/erase anywhere given an iterator, and iterators stay valid, but no random access and poor cache locality. `std::forward_list` is singly linked and smaller still.

WHEN TO USE

- `deque` — queue/sliding window needing `push_front`
- `list` — frequent middle insert/erase with stable iterators
- `list::splice` moves nodes between lists in O(1)
- In practice, `vector` usually beats `list` even for middle inserts on small data

DEQUE, LIST, FORWARD_LIST

```cpp
#include <deque>
#include <list>
#include <forward_list>

std::deque<int> dq{2, 3};
dq.push_front(1);                  // 1 2 3
dq.push_back(4);                   // 1 2 3 4
dq.pop_front();                    // 2 3 4
dq[1];                             // 3 (random access)
std::list<std::string> todo{"wash", "cook"};
auto it = std::next(todo.begin());
todo.insert(it, "shop");           // wash shop cook  (it still valid)
todo.remove("wash");               // shop cook
todo.sort();                       // member sort: cook shop
std::list<std::string> later{"nap"};
todo.splice(todo.end(), later);    // cook shop nap; later empty
std::forward_list<int> fl{3, 1, 2};
fl.push_front(0);                  // 0 3 1 2
fl.insert_after(fl.begin(), 9);    // 0 9 3 1 2
```

CODE EXPLAINED

→ `push_front` / `pop_front` — O(1) on deque and lists

→ `dq[1]` — deque allows indexing; lists don't

→ `list::insert` — doesn't invalidate other iterators

→ `list::sort` / `remove` — members, because `std::sort` needs random access

→ `splice` — relinks nodes, no copying

→ `forward_list` — insert/erase *after* a position

COMPARISON

OP	VECTOR	DEQUE	LIST
`push_back`	O(1)*	O(1)	O(1)
`push_front`	O(n)	O(1)	O(1)
`random access`	O(1)	O(1)	O(n)
`middle insert`	O(n)	O(n)	O(1)
`contiguous`	yes	no	no

LIST-ONLY MEMBERS

MEMBER	EFFECT
`splice(pos, other)`	move nodes in O(1)
`merge(other)`	merge sorted lists
`remove(v) / remove_if(p)`	erase matching
`unique()`	drop consecutive dups
`sort()`	stable merge sort
`reverse()`	reverse links

⚠ COMMON MISTAKES

- Choosing `std::list` for "fast insertion" without measuring; pointer chasing often makes it slower than `vector`
- Calling `std::sort` on a list; it needs random-access iterators. Use `list.sort()`
- Assuming `deque` is contiguous and passing `&dq[0]` to a C API

✓ TIPS & BEST PRACTICES

- Use `std::deque` as the underlying container for queues and sliding windows
- Use `std::list` when you need iterators that survive insertions and erasures elsewhere
- Profile before replacing `vector`; its cache behavior wins surprisingly often

QUICK SUMMARY

- `std::deque` — O(1) at both ends, indexable
- `std::list` — linked list, stable iterators
- `splice` — O(1) node transfer
- `std::forward_list` — singly linked, minimal

Stacks, Queues & Priority Queues

CONTAINER ADAPTORS

Container adaptors wrap another container and expose a restricted interface. `std::stack` is **LIFO** (last in, first out), `std::queue` is **FIFO** (first in, first out), and `std::priority_queue` always gives you the **largest** element first (a binary heap). None of them support iteration. Choose the underlying container with a template argument.

DEFAULTS

- `stack` / `queue` default to `std::deque`
- `priority_queue` defaults to `std::vector` + `std::less` (max-heap)
- `pop()` returns `void`; read with `top()` / `front()` first
- Calling `top()` / `pop()` on an empty adaptor is UB

STACK, QUEUE, PRIORITY_QUEUE

```cpp
#include <stack>
#include <queue>

std::stack<int> st;
st.push(1); st.push(2); st.push(3);
st.top();                  // 3
st.pop();                  // removes 3

std::queue<std::string> q;
q.push("a"); q.push("b");
q.front(); q.back();       // "a", "b"
q.pop();                   // removes "a"

std::priority_queue<int> maxHeap;
for (int x : {4, 9, 1}) maxHeap.push(x);
maxHeap.top();             // 9

std::priority_queue<int, std::vector<int>, std::greater<>> minHeap{
    std::greater<>{}, {4, 9, 1}};
minHeap.top();             // 1
using Job = std::pair<int, std::string>;          // (priority, name)
std::priority_queue<Job> jobs;                    // highest priority first
jobs.emplace(2, "email"); jobs.emplace(5, "deploy");  // top: deploy
```

CODE EXPLAINED

- → `push` / `pop` / `top` — stack interface
- → `front` / `back` — queue reads; `pop` removes the front
- → default `priority_queue` — max-heap
- → `std::greater<>` — turns it into a min-heap
- → pairs compare by first element, then second

INTERFACES

ADAPTOR	INSERT	READ	REMOVE
`stack`	push / emplace	top()	pop()
`queue`	push / emplace	front() / back()	pop()
`priority_queue`	push / emplace	top()	pop()

COMPLEXITY

OPERATION	STACK / QUEUE	PRIORITY_QUEUE
`push`	O(1)	O(log n)
`pop`	O(1)	O(log n)
`top / front`	O(1)	O(1)
`size / empty`	O(1)	O(1)

⚠ COMMON MISTAKES

- Writing `int x = st.pop();`; `pop` returns `void`. Read `top()` first, then `pop()`
- Expecting a min-heap from the default `priority_queue`; it's a max-heap
- Needing to iterate or update priorities; adaptors can't. Use a `std::set` or `std::ranges::make_heap` on a vector

✓ TIPS & BEST PRACTICES

- Use a min-heap of `{distance, node}` pairs for Dijkstra's algorithm
- Store `std::pair<int, T>` with negated priorities as a quick min-heap alternative
- Check `empty()` before every `top()` / `front()`

QUICK SUMMARY

- `std::stack` — LIFO
- `std::queue` — FIFO
- `std::priority_queue` — max-heap by default
- `std::greater<>` — min-heap

std::span & std::mdspan

VIEWS OVER CONTIGUOUS DATA

`std::span<T>` (C++20, `<span>`) is a non-owning **pointer + size** view over any contiguous sequence: a C array, `std::array`, `std::vector`, or part of one. It replaces the classic `(T* ptr, size_t n)` parameter pair. `std::mdspan` (C++23, `<mdspan>`) extends the idea to **multidimensional** views over flat storage, such as matrices and images.

NOTES

- `span<const T>` for read-only access, `span<T>` to modify
- **Dynamic extent** `span<T>` stores size; **static** `span<T, N>` fixes it at compile time
- Pass by value; it's two words
- Like `string_view`, it must not outlive the data

SPAN & MDSPAN

```
#include <span>
#include <mdspan>

double average(std::span<const double> xs) {
    return std::accumulate(xs.begin(), xs.end(), 0.0) / xs.size();
}
double raw[] = {1, 2, 3};
std::vector<double> vec{4, 5, 6, 7};
average(raw);                       // 2
average(vec);                       // 5.5
average({vec.data() + 1, 2});       // 5.5  (5, 6)

std::span<int> s{ints};             // writable view
s.first(2);  s.last(2);             // first / last 2 elements
s.subspan(1, 3);                    // 3 elements from index 1
s[0] = 42;                          // modifies ints[0]
std::span<int, 4> fixed{arr4};      // static extent
std::vector<float> pixels(3 * 4);
std::mdspan img{pixels.data(), 3, 4};   // 3 rows × 4 cols
img[1, 2] = 0.5f;                   // C++23 multi-index operator[]
img.extent(0); img.extent(1);       // 3, 4
```

CODE EXPLAINED

- → `span<const double>` — accepts arrays, vectors, `std::array`
- → `{ptr, count}` — build a span over part of a buffer
- → `first` / `last` / `subspan` — O(1) slicing
- → writes through `span<int>` modify the original data
- → `mdspan img{ptr, 3, 4}` — 2D view; `img[r, c]` indexing

SPAN MEMBERS

MEMBER	RETURNS
`size() / size_bytes()`	count / bytes
`data()`	pointer to first element
`first(n) / last(n)`	sub-span
`subspan(off, n)`	sub-span
`front() / back()`	ends
`as_bytes(s)`	span<const std::byte>

REPLACES

OLD SIGNATURE	MODERN
`f(int* p, size_t n)`	f(std::span<int>)
`f(const int* p, size_t n)`	f(std::span<const int>)
`f(const vector<int>&)`	f(std::span<const int>)
`float* + width, height`	std::mdspan

⚠ COMMON MISTAKES

- Returning a `span` over a local vector; it dangles
- Keeping a span to a vector, then calling `push_back`; reallocation invalidates it
- Passing a `std::list` to a span parameter; spans need contiguous storage

✓ TIPS & BEST PRACTICES

- Make span the default parameter type for "a bunch of T" when you don't need ownership
- Use `std::as_bytes(span)` for serialization and checksums
- Library support for `std::mdspan` is newer (libstdc++ 14+, libc++ 17+, MSVC 17.9+); check before relying on it

QUICK SUMMARY

- `std::span<T>` — non-owning contiguous view
- `span<const T>` — read-only parameter type
- `first / last / subspan` — slicing
- `std::mdspan` — multidimensional view (C++23)

Choosing a Container

Big-O table · contiguous vs node-based · std::flat_map / flat_set · std::inplace_vector (C++26) · invalidation rules · decision chart

PICK BY ACCESS PATTERN

Choose a container by how you'll **access** the data, not by what sounds right. Contiguous containers (`vector`, `array`, `flat_map`) win most real-world benchmarks thanks to CPU caches, even when Big-O suggests otherwise. Node-based containers (`list`, `map`, `set`, `unordered_*`) pay a pointer chase per element but keep references stable. C++23 adds the flat containers, and C++26 adds `std::inplace_vector`, a fixed-capacity vector with no heap allocation.

DEFAULTS

- Sequence → `std::vector`
- Fixed size → `std::array`
- Key lookup → `std::unordered_map` (or `flat_map` if read-mostly)
- Need sorted keys → `std::map` / `std::flat_map`

DECISION GUIDE

```
// Need indexing + append?            → std::vector<T>
// Size known at compile time?        → std::array<T, N>
// Bounded size, no heap (C++26)?     → std::inplace_vector<T, N>
// Push/pop at both ends?             → std::deque<T>
// Stable iterators on middle edits?  → std::list<T>
// Key → value, any order, fast?      → std::unordered_map<K, V>
// Key → value, sorted / ranges?      → std::map<K, V>
// Mostly lookups, rare inserts?      → std::flat_map<K, V> (C++23)
// Unique membership?                 → std::unordered_set / std::set
// LIFO / FIFO / best-first?          → stack / queue / priority_queue
// View someone else's data?          → std::span / std::string_view

std::flat_map<std::string, int> fm{{"b", 2}, {"a", 1}};  // sorted vectors
std::inplace_vector<int, 8> small{1, 2, 3};              // capacity 8
```

CODE EXPLAINED

- → start with `vector`; switch only for a specific reason
- → `inplace_vector` — embedded/real-time code that can't allocate
- → `flat_map` — two sorted vectors (keys, values): fast lookup, slow insert
- → views (`span`, `string_view`) — for parameters, not storage

BIG-O CHEAT SHEET

CONTAINER	ACCESS	SEARCH	INSERT	ERASE
`vector`	O(1)	O(n)	O(1)* end / O(n)	O(n)
`deque`	O(1)	O(n)	O(1) ends	O(n)
`list`	O(n)	O(n)	O(1)	O(1)
`map / set`	O(log n)	O(log n)	O(log n)	O(log n)
`unordered_*`	O(1) avg	O(1) avg	O(1) avg	O(1) avg
`flat_map`	O(log n)	O(log n)	O(n)	O(n)

INVALIDATION ON INSERT

CONTAINER	WHAT BREAKS
`vector`	all, if it reallocates
`deque`	all iterators; refs OK at ends
`list / map / set`	nothing
`unordered_*`	iterators on rehash
`flat_map`	everything

⚠ COMMON MISTAKES

- Using `std::map` by habit when you never need ordering; `unordered_map` or `flat_map` is usually faster
- Choosing `std::list` for performance without measuring
- Storing pointers to `vector` elements while the vector still grows

✓ TIPS & BEST PRACTICES

- Benchmark with realistic sizes; for under ~100 elements a linear search in a `vector` often wins
- Reserve capacity up front (`vector::reserve`, `unordered_map::reserve`)
- Prefer `std::flat_map` for configuration-style data built once and read often

QUICK SUMMARY

- `std::vector` — default choice
- contiguous — cache-friendly, usually fastest
- node-based — stable references, slower traversal
- `flat_map` (C++23) / `inplace_vector` (C++26) — newer options

Classes & Objects

USER-DEFINED TYPES

A **class** bundles data (**data members**) with the functions that operate on it (**member functions**). An **object** is an instance of a class. `class` and `struct` are identical except for default access: `struct` members are `public` by default, `class` members `private`. Inside a member function, `this` points to the current object. Large classes usually declare members in a header and define them in a `.cpp` using `ClassName::`.

CONVENTIONS

- `struct` for plain data bundles; `class` when there are invariants to protect
- End the class definition with `};`
- Member functions defined inside the class are implicitly `inline`
- Common member naming: `name_`, `m_name`, or `mName`

A COMPLETE MINIMAL CLASS

```
// counter.h
class Counter {
public:
    void increment() { ++count_; }      // defined inline
    int  value() const;                 // declared only
    void reset(int start = 0);
private:
    int count_ = 0;                     // default member initializer
};
// counter.cpp
int  Counter::value() const { return count_; }
void Counter::reset(int start) { this->count_ = start; }
// main.cpp
Counter c;                              // object on the stack
c.increment(); c.increment();
c.value();                              // 2
auto* p = &c;  p->reset(10);            // via pointer: 10
```

CODE EXPLAINED

- → `public:` / `private:` — access sections
- → `int count_ = 0;` — every object starts at 0
- → `Counter::value()` — out-of-class definition
- → `this->count_` — explicit access; usually just `count_`
- → `.` on objects, `->` on pointers

CLASS VS STRUCT

ASPECT	CLASS	STRUCT
default member access	private	public
default base access	private	public
capabilities	identical	identical
typical use	invariants	plain data

MEMBER KINDS

MEMBER	EXAMPLE
data member	`int count_;`
member function	`void increment();`
const member function	`int value() const;`
static member	`static int instances;`
nested type	`using Id = int;`
constructor / destructor	`Counter();` / `~Counter();`

⚠ COMMON MISTAKES

- Forgetting the semicolon after the closing brace: `class X { … }` followed by an error on the next line
- Writing `int value() const` in the header but `int Counter::value()` (no `const`) in the `.cpp`; they're different functions
- Making all data `public` in a class with invariants; anyone can break them

✓ TIPS & BEST PRACTICES

- Give every data member a default initializer so objects are never half-initialized
- Keep the public interface small; add private helpers freely
- Order members: public interface first, then private details

QUICK SUMMARY

- `class` / `struct` — same thing, different default access
- data + member functions — bundled behavior
- `this` — pointer to the current object
- `Class::member` — out-of-class definition

Constructors

INITIALIZING OBJECTS

A **constructor** is a special member function with the class's name and no return type, called automatically when an object is created. The **member initializer list** (`: a(x), b(y)`) initializes members directly, before the constructor body runs. It's required for `const` members, references, and members without default constructors. A constructor can **delegate** to another constructor of the same class.

NOTES

- Members initialize in **declaration order**, not initializer-list order
- If you declare no constructors, the compiler provides a default one
- `explicit` blocks implicit conversions from a single argument
- `= default` asks for the compiler-generated version

CONSTRUCTOR FORMS

```
class Temperature {
public:
    Temperature() = default;                          // default ctor
    explicit Temperature(double c) : celsius_(c) {}   // init list
    Temperature(double v, char unit)                  // delegating
        : Temperature(unit == 'F' ? (v - 32) / 1.8 : v) {}
    double celsius() const { return celsius_; }
private:
    double celsius_ = 0.0;
};
Temperature a;                  // 0.0
Temperature b{21.5};            // 21.5
Temperature c{98.6, 'F'};       // 37.0
// Temperature d = 30.0;        // ERROR: constructor is explicit
void show(Temperature t);
// show(25.0);                  // ERROR: no implicit conversion
show(Temperature{25.0});        // OK
```

CODE EXPLAINED

- → `= default` — compiler-generated default constructor
- → `: celsius_(c)` — direct member initialization
- → `: Temperature(...)` — delegates to another constructor
- → `explicit` — prevents `Temperature t = 30.0;` and silent conversions
- → braces — `Temperature b{21.5}` calls the matching constructor

SPECIAL CONSTRUCTORS

SIGNATURE	PURPOSE
`T()`	default
`T(const T&)`	copy
`T(T&&)`	move
`T(int)`	converting (mark explicit)
`T(std::initializer_list<U>)`	brace list
`T(...) : T(...)`	delegating

MUST USE INIT LIST

MEMBER TYPE	WHY
`const member`	can't assign later
`reference member`	must bind at creation
`no default ctor`	nothing to default-construct
`base class args`	base built before body
`expensive types`	avoids default + assign

⚠ COMMON MISTAKES

- Assigning in the body (`name_ = n;`) instead of initializing; members are default-constructed first, then assigned
- Listing initializers in a different order than declaration and relying on that order; `-Wreorder` warns
- Single-argument constructors without `explicit`, allowing surprising implicit conversions

✓ TIPS & BEST PRACTICES

- Make single-argument constructors `explicit` unless conversion is intended
- Prefer default member initializers for defaults; keep constructors for real parameters
- Use a static factory (`static std::expected<T, E> create(...)`) when construction can fail without exceptions

QUICK SUMMARY

- constructor — same name as class, no return type
- `: m(x)` — member initializer list
- `explicit` — no implicit conversion
- delegating — `T(a) : T(a, default) {}`

Destructors

~Class() · destruction order · virtual destructors · scope exit · = default · never throw from a destructor

CLEANING UP

A **destructor** `~T()` runs automatically when an object's lifetime ends: when it goes out of scope, when `delete` is called, or when its owner is destroyed. It's where a class releases resources it acquired. Members are destroyed **after** the destructor body, in **reverse declaration order**; local objects are destroyed in reverse order of creation. If a class will be deleted through a base pointer, its base needs a **virtual destructor**.

RULES

- One destructor per class, no parameters, no return type
- Implicitly `noexcept`; never let exceptions escape it
- Most classes need no destructor at all (rule of zero)
- Base classes used polymorphically: `virtual ~Base() = default;`

DESTRUCTORS IN ACTION

```
class Logger {
public:
    explicit Logger(std::string name) : name_(std::move(name)) {
        std::println("open {}", name_);
    }
    ~Logger() { std::println("close {}", name_); }   // destructor
private:
    std::string name_;
};
void run() {
    Logger a{"A"};
    {
        Logger b{"B"};
    }                       // "close B" here (end of inner scope)
    Logger c{"C"};
}                           // "close C", then "close A"
struct Shape { virtual ~Shape() = default; };
struct Circle : Shape { ~Circle() override { /* ... */ } };
```

CODE EXPLAINED

- → `~Logger()` — runs automatically, even during exception unwinding
- → inner scope — `b` dies at its closing brace
- → reverse order — `c` is destroyed before `a`
- → `virtual ~Shape()` — makes `delete shapePtr` call `~Circle()`
- → `override` on a destructor — allowed, documents intent

WHEN DESTRUCTORS RUN

OBJECT	DESTROYED AT
`local variable`	end of its scope
`temporary`	end of full expression
`new-ed object`	`delete`
`member`	after owner's destructor body
`static / global`	program exit, reverse order
`container element`	erase / container destruction

DO / DON'T

DO	DON'T
`release resources`	throw exceptions
`log or flush`	call virtual functions expecting derived behavior
`= default when trivial`	write one with nothing to do
`virtual in poly bases`	make it virtual "just in case" in value types

⚠ COMMON MISTAKES

- Deleting a derived object through a base pointer without a virtual destructor; that's UB and usually leaks
- Throwing from a destructor; during unwinding this calls `std::terminate`
- Calling the destructor manually (`obj.~T()`) on a normal object; it runs again at scope end

✓ TIPS & BEST PRACTICES

- Wrap each resource (file, socket, lock) in its own class so destructors clean up automatically
- Use `std::unique_ptr` with a custom deleter for C handles instead of writing a destructor
- Write `virtual ~Base() = default;` in every class meant to be inherited from polymorphically

QUICK SUMMARY

- `~T()` — runs automatically at end of lifetime
- reverse order — locals and members
- `virtual ~Base()` — required for polymorphic deletion
- noexcept — destructors must not throw

Access Control & friend

ENCAPSULATION

Access specifiers control who can use a class member. `public` members form the interface; `private` members are visible only inside the class (and its friends); `protected` members are also visible to derived classes. Access is checked per **class**, not per object, so one `Account` can read another `Account`'s private data. `friend` grants a specific function or class full access.

NOTES

- Access control is compile-time only; no run-time cost
- `friend` is not inherited and not transitive
- Prefer non-member functions that use the public interface over friends
- Getters/setters only where they add value (validation, invariants)

ACCESS LEVELS & FRIEND

```
class Account {
public:
    explicit Account(double opening) : balance_(opening) {}
    double balance() const { return balance_; }        // getter
    void deposit(double amt) {
        if (amt <= 0) throw std::invalid_argument("amount must be > 0");
        balance_ += amt;                               // invariant kept
    }
    bool richerThan(const Account& o) const {
        return balance_ > o.balance_;      // other's private: OK
    }
    friend std::ostream& operator<<(std::ostream& os, const Account& a) {
        return os << "Account(" << a.balance_ << ")";
    }
protected:
    void audit() const {}                              // derived classes only
private:
    double balance_;
};
```

CODE EXPLAINED

- → `public` — callable by anyone
- → validation in `deposit` — the reason to keep `balance_` private
- → `o.balance_` — allowed; access is per class
- → `friend operator<<` — non-member with private access
- → `protected` — visible to subclasses, not to outsiders

WHO CAN ACCESS

SPECIFIER	CLASS	DERIVED	OUTSIDE
`public`	✔	✔	✔
`protected`	✔	✔	✖
`private`	✔	✖	✖
`friend`	✔ (granted)	—	—

FRIEND FORMS

DECLARATION	GRANTS ACCESS TO
`friend void f(T&);`	one function
`friend class Helper;`	all of Helper's members
`friend bool operator==(…)`	hidden friend operator
`friend struct Test;`	unit-test access (use sparingly)

⚠ COMMON MISTAKES

- A getter and setter for every field; that's public data with extra steps. Expose operations instead
- Returning a non-const reference to private data (`std::vector<int>& items()`), which lets callers bypass invariants
- Using `friend` widely to work around design problems

✓ TIPS & BEST PRACTICES

- Make data private by default; start with the smallest public interface that works
- Define operators like `operator<<` and `operator==` as **hidden friends** inside the class
- Return `const T&` or `std::span<const T>` from accessors to large members

QUICK SUMMARY

- `public` — interface for everyone
- `private` — implementation details
- `protected` — open to derived classes
- `friend` — selective access grant

const & Qualified Member Functions

QUALIFYING MEMBER FUNCTIONS

A **`const` member function** promises not to modify the object, so it can be called on `const` objects and through `const&`. Inside it, `this` is a pointer to const. `mutable` members are exempt, for caches or mutexes that don't affect the object's logical state. **Ref-qualifiers** (&, &&) overload on whether the object is an lvalue or a temporary, and C++23's **explicit object parameter** (`this auto&& self`) collapses const/non-const/ref overloads into one template.

NOTES

- Mark every function that doesn't modify state `const`
- `const` objects can only call `const` member functions
- `const` and non-const overloads are often written as a pair
- Deducing `this` removes that duplication (C++23)

CONST, MUTABLE, &/&&, DEDUCING THIS

```
class Document {
public:
    std::size_t wordCount() const {           // callable on const objects
        if (!cache_) cache_ = countWords();   // OK: cache_ is mutable
        return *cache_;
    }
    std::string& title() & { return title_; }           // lvalue object
    std::string  title() && { return std::move(title_); } // temporary
    template <class Self>
    auto& text(this Self&& self) { return self.text_; } // C++23: one for all
private:
    std::size_t countWords() const;
    std::string title_, text_;
    mutable std::optional<std::size_t> cache_;
};
const Document doc = load();
doc.wordCount();                // OK
auto t = load().title();        // && overload: moves the title out
```

CODE EXPLAINED

- → `const` after the parameter list — `this` is `const Document*`
- → `mutable` — lazily computed cache can change inside `const` functions
- → `&` / `&&` qualifiers — choose by value category of the object
- → `this Self&& self` — deduces const and ref-ness from the call
- → `load().title()` — temporary object picks the `&&` version

QUALIFIERS

SIGNATURE	CALLABLE ON
`void f();`	non-const lvalues and rvalues
`void f() const;`	anything
`void f() &;`	non-const lvalues
`void f() const &;`	any lvalue, and rvalues
`void f() &&;`	rvalues (temporaries)
`void f(this auto&& s);`	all, deduced

MUTABLE — GOOD USES

MEMBER	WHY
`cache / memo`	logical state unchanged
`std::mutex`	locking in const getters
`access counters`	diagnostics only
`lazy-loaded data`	invisible to callers

⚠ COMMON MISTAKES

- Forgetting `const` on getters, so `const&` parameters can't call them
- Using `mutable` to dodge `const` for real state changes; that breaks the promise and thread-safety assumptions
- Returning a non-const reference from a `const` function; it won't compile (and shouldn't)

✓ TIPS & BEST PRACTICES

- Write the `const` version first; add non-const only when callers need to modify
- Pair `mutable std::mutex m_;` with `std::scoped_lock` in const accessors for thread-safe reads
- Use deducing `this` to replace duplicated const/non-const accessor pairs

QUICK SUMMARY

- `f() const` — doesn't modify the object
- `mutable` — exempt from const (caches, mutexes)
- `f() &` / `f() &&` — overload on lvalue vs temporary
- `this auto&& self` — explicit object parameter (C++23)

Static Members

CLASS-LEVEL MEMBERS

A **static data member** belongs to the class, not to any object: there's exactly one copy shared by all instances. A **static member function** has no `this` pointer and can only touch static members directly; call it as `ClassName::f()`. Since C++17, `inline static` lets you define a static data member inside the class, so no separate `.cpp` definition is needed.

NOTES

- `static constexpr` members are implicitly `inline` (C++17)
- Access via `Class::member` (preferred) or through an object
- Static members exist even if no object is ever created
- A function-local `static` initializes once, thread-safely

STATIC MEMBERS

```
class Connection {
public:
    Connection()  { ++open_; }
    ~Connection() { --open_; }
    static int openCount() { return open_; }     // no `this`
    static constexpr int kMaxOpen = 64;          // compile-time constant
    static Connection& primary() {               // singleton accessor
        static Connection instance;              // built on first call
        return instance;
    }
private:
    inline static int open_ = 0;                 // one shared counter
};
Connection a, b;
Connection::openCount();                         // 2
Connection::kMaxOpen;                            // 64
Connection::primary();                           // same object every call
```

CODE EXPLAINED

- → `++open_` — every instance updates the shared counter
- → `static int openCount()` — called without an object
- → `static constexpr` — class-wide constant
- → function-local `static` — Meyers singleton, thread-safe init
- → `inline static` — in-class definition (C++17)

INSTANCE VS STATIC

ASPECT	INSTANCE MEMBER	STATIC MEMBER
copies	one per object	one per class
this available	yes	no
call syntax	obj.f()	Class::f()
can access	everything	static members only
lifetime	object's	whole program

DEFINING STATICS

FORM	WHERE DEFINED
`inline static int n = 0;`	in class (C++17)
`static constexpr int k = 5;`	in class
`static const int k = 5;`	in class (integral only)
`static int n; + int T::n = 0;`	one .cpp (pre-17)

⚠ COMMON MISTAKES

- Declaring `static int count;` without a definition (pre-C++17 style), causing an "undefined reference" link error
- Accessing instance members inside a static function; there's no object, so it won't compile
- Overusing singletons; global state makes testing and concurrency harder

✓ TIPS & BEST PRACTICES

- Use `inline static` for in-class definitions
- Prefer a free function or namespace for utilities that don't need class access
- If a singleton is truly needed, use a function-local static, not a global pointer

QUICK SUMMARY

- `static` data — one copy shared by all objects
- `static` function — no `this`, call `Class::f()`
- `inline static` — define in class (C++17)
- local `static` — lazy, thread-safe singleton

Aggregates & Designated Initializers

PLAIN DATA TYPES

An **aggregate** is a class or array with no user-declared constructors, no private non-static data, no virtual functions, and no virtual or private bases. Aggregates can be initialized directly with braces, member by member. C++20's **designated initializers** (`.x = 1`) name the members being set, which makes calls readable, and **default member initializers** fill in anything omitted.

RULES

- Designators must appear in **declaration order**
- Omitted members use their default initializer, or are value-initialized
- Designated init and positional init can't be mixed
- Aggregates support structured bindings automatically

AGGREGATES & DESIGNATED INIT

```
struct Options {
    std::string host = "localhost";
    int  port = 8080;
    bool tls  = false;
    int  timeoutMs = 5000;
};
Options a;                                  // all defaults
Options b{"example.com", 443, true};        // positional; timeout 5000
Options c{.host = "api.dev", .tls = true};  // port 8080, timeout 5000
// Options d{.port = 1, .host = "x"};       // ERROR: out of order
void connect(const Options& o);
connect({.port = 9000, .timeoutMs = 100});  // named-argument style
auto [h, p, t, ms] = c;                     // structured bindings
struct Point { double x, y; };
Point pts[] = {{0, 0}, {1, 2}};             // nested aggregates
static_assert(std::is_aggregate_v<Options>);
```

CODE EXPLAINED

- → default member initializers — used when a member isn't mentioned
- → positional `{…}` — fills members in order
- → `.tls = true` — designated initializer (C++20)
- → `connect({.port = …})` — emulates named arguments
- → `std::is_aggregate_v` — verify a type is an aggregate

AGGREGATE REQUIREMENTS

ALLOWED	AGGREGATE?
public data members	✔
default member initializers	✔ (C++14)
public base classes	✔ (C++17)
user-declared constructor	✖
private / protected data	✖
virtual functions	✖

TYPE TRAITS

TRAIT	MEANING
is_aggregate	brace-init member-wise
is_trivially_copyable	safe to memcpy
is_standard_layout	C-compatible layout
is_trivial	trivially copyable + trivial default ctor

⚠ COMMON MISTAKES

- Adding a constructor to an aggregate (even `= default` declared), which stops brace-init by member
- Writing designators out of order; unlike C, C++ requires declaration order
- Reordering struct members and silently changing the meaning of positional initializers

✓ TIPS & BEST PRACTICES

- Use designated initializers for option/config structs with many fields
- Keep option structs as aggregates with sensible defaults instead of many-parameter constructors
- Add `auto operator<=>(const T&) const = default;` to aggregates for free comparisons

QUICK SUMMARY

- aggregate — public data, no ctors, no virtuals
- `T{a, b}` — positional initialization
- `T{.x = 1}` — designated initializers (C++20)
- default member initializers — fill omitted fields

Copy Semantics

COPYING OBJECTS

Copying is controlled by two special members: the **copy constructor** `T(const T&)` creates a new object from an existing one, and the **copy assignment operator** `T& operator=(const T&)` overwrites an existing object. The compiler generates memberwise versions, which are correct when every member copies itself properly. Classes owning raw resources need a **deep copy**, and the **rule of three** says if you write one of destructor / copy ctor / copy assignment, you probably need all three.

NOTES

- Copy ctor: `T b = a;` or `T b{a};` or pass-by-value
- Copy assign: `b = a;` on an existing object
- `= delete` makes a type non-copyable
- **Copy elision**: returning a local or temporary often skips the copy entirely

DEEP COPY (RULE OF THREE)

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : size_(n), data_(new int[n]{}) {}
    ~Buffer() { delete[] data_; }
    Buffer(const Buffer& o)                          // copy constructor
        : size_(o.size_), data_(new int[o.size_]) {
        std::copy_n(o.data_, size_, data_);          // deep copy
    }
    Buffer& operator=(const Buffer& o) {             // copy assignment
        if (this != &o) { Buffer tmp(o); swap(tmp); }   // copy-and-swap
        return *this;
    }
    void swap(Buffer& o) noexcept {
        std::swap(size_, o.size_); std::swap(data_, o.data_);
    }
private:
    std::size_t size_;
    int* data_;
};
```

CODE EXPLAINED

- → default copy would copy the **pointer** — two objects, one array, double delete
- → copy constructor — allocates its own array and copies elements
- → `if (this != &o)` — self-assignment guard
- → copy-and-swap — strong exception guarantee
- → better: use `std::vector<int>` and write none of this (rule of zero)

SPECIAL MEMBERS

MEMBER	SIGNATURE	TRIGGERED BY
copy ctor	T(const T&)	`T b = a;`
copy assign	T& operator=(const T&)	`b = a;`
destructor	~T()	end of lifetime
non-copyable	T(const T&) = delete;	—

SHALLOW VS DEEP

COPY	RESULT
shallow (default with raw ptr)	shared buffer, double free
deep	independent buffers
member types like vector	deep automatically
unique_ptr member	copying disabled

⚠ COMMON MISTAKES

- Owning a raw pointer and relying on the default copy; both objects delete the same memory
- Forgetting the self-assignment case, so `a = a;` frees data before copying it
- Writing a destructor but not the copy operations (violating the rule of three)

✓ TIPS & BEST PRACTICES

- Prefer the **rule of zero**: hold resources in `std::vector`, `std::string`, smart pointers, and write no special members
- Make resource handles move-only (`= delete` copies) when copying makes no sense
- Pass large objects by `const&` to avoid accidental copies

QUICK SUMMARY

- `T(const T&)` — copy constructor
- `operator=(const T&)` — copy assignment
- rule of three — dtor + copy ctor + copy assign together
- rule of zero — let members manage resources

Move Semantics

MOVING INSTEAD OF COPYING

Move semantics (C++11) let an object **steal** the resources of another object that's about to die, instead of copying them. A move constructor `T(T&&)` and move assignment `T& operator=(T&&)` take an **rvalue reference**. `std::move(x)` doesn't move anything; it's a cast that says "treat x as an rvalue", allowing a move. The moved-from object must stay valid but has an unspecified value.

RULES

- Temporaries and `std::move(x)` bind to `T&&`
- Mark moves `noexcept` so containers use them
- **Rule of five**: dtor, copy ctor, copy assign, move ctor, move assign
- **Rule of zero**: write none; let members handle it

MOVE CONSTRUCTOR & ASSIGNMENT

```
class Buffer {
public:
    Buffer(Buffer&& o) noexcept                          // move ctor
        : size_(std::exchange(o.size_, 0)),
          data_(std::exchange(o.data_, nullptr)) {}      // steal, leave empty
    Buffer& operator=(Buffer&& o) noexcept {             // move assign
        if (this != &o) {
            delete[] data_;
            size_ = std::exchange(o.size_, 0);
            data_ = std::exchange(o.data_, nullptr);
        }
        return *this;
    }
    // ... ctor, dtor, copy ops as on the previous sheet
};
std::vector<std::string> v;
std::string s = "long text that won't fit in SSO";
v.push_back(std::move(s));   // moves; s is now valid but unspecified
Buffer make() { Buffer b(1024); return b; }   // moved or elided
```

CODE EXPLAINED

- → `Buffer&&` — binds to temporaries and `std::move`d objects
- → `std::exchange(o.x, 0)` — take the value, reset the source
- → `noexcept` — lets `std::vector` move during reallocation
- → `std::move(s)` — enables the move; don't use `s`'s value afterwards
- → returning a local — automatically moved (or elided)

RULE OF FIVE / ZERO

WRITE	WHEN
`nothing (rule of zero)`	members manage resources
`all five`	class directly owns a raw resource
`= default all five`	need virtual dtor but default ops
`= delete copies`	move-only types (`unique_ptr`)

GENERATED MOVES SUPPRESSED BY

USER-DECLARED	IMPLICIT MOVE?
`destructor`	✖
`copy ctor / copy assign`	✖
`move ctor`	✔ (only that one)
`nothing`	✔

⚠ COMMON MISTAKES

- `return std::move(local);` — this blocks copy elision. Just `return local;`
- Using an object after `std::move` as if it still held its value
- Declaring a destructor (even empty) and silently losing implicit moves, making everything copy

✓ TIPS & BEST PRACTICES

- Write `= default` for all special members when you need just one non-default one
- Take "sink" parameters by value and `std::move` them into members
- Verify with `std::is_nothrow_move_constructible_v` in a `static_assert`

QUICK SUMMARY

- `T(T&&)` — move constructor
- `std::move` — cast to rvalue; enables moving
- `noexcept` moves — required for fast container growth
- rule of five / zero — all or nothing

Inheritance

: public Base · protected / private inheritance · base ctor calls · using Base::Base · final classes · object slicing

DERIVING CLASSES

Inheritance creates a new class from an existing one: the **derived** class gets the base's members and can add or override behavior. `class Dog : public Animal` models "is-a". Base subobjects are constructed **first** and destroyed **last**. Copying a derived object into a base-typed variable **slices** off the derived part, which is why polymorphic objects are used through references or pointers.

NOTES

- Base constructor args go in the derived initializer list
- `using Base::Base;` inherits the base constructors
- `final` on a class forbids further derivation
- Prefer composition ("has-a") when "is-a" doesn't really hold

BASE & DERIVED

```
class Animal {
public:
    explicit Animal(std::string name) : name_(std::move(name)) {}
    virtual ~Animal() = default;
    const std::string& name() const { return name_; }
protected:
    int age_ = 0;                       // visible to derived classes
private:
    std::string name_;
};
class Dog final : public Animal {
public:
    Dog(std::string name, std::string breed)
        : Animal(std::move(name)), breed_(std::move(breed)) {}
    std::string describe() const { return name() + " the " + breed_; }
private:
    std::string breed_;
};
Dog rex{"Rex", "collie"};  rex.describe();   // "Rex the collie"
Animal a = rex;                             // SLICED: only Animal part
```

CODE EXPLAINED

- → `: public Animal` — public inheritance keeps the base's public interface public
- → `: Animal(std::move(name))` — base constructed before `breed_`
- → `protected age_` — derived classes can use it
- → `final` — nobody can inherit from `Dog`
- → `Animal a = rex;` — copies only the base part (slicing)

INHERITANCE ACCESS

BASE MEMBER	PUBLIC INH.	PROTECTED INH.	PRIVATE INH.
`public`	public	protected	private
`protected`	protected	protected	private
`private`	inaccessible	inaccessible	inaccessible

ORDER OF EVENTS

STEP	CONSTRUCTION / DESTRUCTION
1	base classes (declaration order)
2	members (declaration order)
3	derived constructor body
`destroy`	exact reverse of the above

⚠ COMMON MISTAKES

- Passing derived objects by value as the base type, which slices them; use `const Base&` or pointers
- Forgetting `public` in `class D : B`; the default for `class` is private inheritance
- Using inheritance just to reuse code; composition is usually simpler and less coupled

✓ TIPS & BEST PRACTICES

- Keep hierarchies shallow; two or three levels is plenty
- Use `using Base::Base;` to forward constructors when the derived class adds no state
- Mark leaf classes `final`; it documents intent and enables devirtualization

QUICK SUMMARY

- `class D : public B` — "is-a" inheritance
- base constructed first, destroyed last
- slicing — copying derived into base loses data
- `final` — stop further derivation

Virtual Functions & Polymorphism

DYNAMIC DISPATCH

A `virtual` function is resolved at **run time** based on the actual type of the object, not the static type of the pointer or reference used to call it. That's **polymorphism**: code written against `Shape&` works for every derived shape. Mark overriding functions `override` so the compiler checks the signature. Under the hood, each polymorphic object carries a pointer to a **vtable** of function pointers.

NOTES

- Dispatch works only through references or pointers
- `override` — compile error if nothing is overridden
- `final` on a function — no further overriding
- Virtual calls in constructors/destructors use the **current** class

VIRTUAL, OVERRIDE, DYNAMIC_CAST

```
struct Shape {
    virtual ~Shape() = default;
    virtual double area() const = 0;                 // pure virtual
    virtual std::string name() const { return "shape"; }
};
struct Circle : Shape {
    double r; explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    std::string name() const override { return "circle"; }
};
struct Square : Shape {
    double s; explicit Square(double s) : s(s) {}
    double area() const final { return s * s; }
};
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(1.0));
shapes.push_back(std::make_unique<Square>(2.0));
for (const auto& sh : shapes) std::println("{} {}", sh->name(), sh->area());
if (auto* c = dynamic_cast<Circle*>(shapes[0].get())) c->r *= 2;
```

CODE EXPLAINED

- → `virtual` — enables run-time dispatch
- → `= 0` — pure virtual; `Shape` can't be instantiated
- → `override` — checked override; typos become errors
- → `final` — `Square::area` can't be overridden further
- → `dynamic_cast<Circle*>` — `nullptr` if it isn't a `Circle`
- → prints "circle 3.14159" then "shape 4"

KEYWORDS

KEYWORD	MEANING
`virtual`	dispatch on dynamic type
`override`	must override a base virtual
`final`	no further overriding / deriving
`= 0`	pure virtual (abstract)
`dynamic_cast`	checked downcast
`typeid(obj)`	run-time type info

COST & BEHAVIOR

ASPECT	DETAIL
`object size`	+ one vptr (8 bytes)
`call cost`	one indirect call
`inlining`	blocked unless devirtualized
`by value`	no dispatch (sliced)
`in ctor/dtor`	calls the current class's version

⚠ COMMON MISTAKES

- Signature mismatch (missing `const`) creates a new function instead of overriding; `override` catches it
- Calling virtual functions from a constructor expecting the derived version
- Storing polymorphic objects by value in `std::vector<Shape>`; it slices. Store `unique_ptr<Shape>`

✓ TIPS & BEST PRACTICES

- Always write `override`; enable `-Wsuggest-override` / `-Winconsistent-missing-override`
- Prefer `std::variant` + `std::visit` for closed sets of types with no virtual overhead
- Avoid `dynamic_cast` chains; add a virtual function instead

QUICK SUMMARY

- `virtual` — run-time dispatch
- `override` — checked overriding
- vtable — mechanism behind virtual calls
- `dynamic_cast` — safe downcast, null on failure

Abstract Classes & Interfaces

INTERFACES IN C++

A class with at least one **pure virtual** function (`= 0`) is **abstract**: it can't be instantiated, only derived from. C++ has no `interface` keyword; an interface is an abstract class with only pure virtual functions and a virtual destructor. The **Non-Virtual Interface (NVI)** idiom makes the public functions non-virtual and the customization points private virtual, so the base class controls pre- and post-conditions.

NOTES

- Derived classes must override every pure virtual to be concrete
- A pure virtual function may still have a body (called explicitly)
- A pure virtual destructor still needs a definition
- Interfaces should have no data members

INTERFACE + NVI

```cpp
class Storage {                                  // interface
public:
    virtual ~Storage() = default;
    virtual void put(std::string_view key, std::string value) = 0;
    virtual std::optional<std::string> get(std::string_view key) const = 0;
};
class MemoryStorage final : public Storage {
public:
    void put(std::string_view k, std::string v) override {
        data_[std::string(k)] = std::move(v);
    }
    std::optional<std::string> get(std::string_view k) const override;
private:
    std::unordered_map<std::string, std::string> data_;
};
class Report {                                   // NVI
public:
    std::string render() const { auto s = body(); return header() + s; }
private:
    virtual std::string body() const = 0;        // customization point
    std::string header() const { return "== Report ==\n"; }
};
```

CODE EXPLAINED

- → `= 0` — pure virtual; `Storage` is abstract
- → `virtual ~Storage()` — safe deletion through the interface
- → `MemoryStorage` — overrides every pure virtual, so it's concrete
- → NVI `render()` — public, non-virtual; controls the flow
- → private `body()` — derived classes override; callers can't call it

ABSTRACT VS INTERFACE VS CONCRETE

KIND	PURE VIRTUALS	DATA	INSTANTIABLE
`interface`	all	none	no
`abstract class`	some	allowed	no
`concrete class`	none	allowed	yes

ALTERNATIVES

TECHNIQUE	DISPATCH
`abstract base`	run time (vtable)
`concepts + templates`	compile time
`std::variant`	closed set, visit
`type erasure (std::function)`	run time, no base class
`CRTP`	compile time, static

⚠ COMMON MISTAKES

- Forgetting one pure virtual override, then getting "cannot instantiate abstract class" far from the cause
- Omitting the virtual destructor on an interface, so `delete` through it is UB
- Putting data members in an interface, coupling every implementation to them

✓ TIPS & BEST PRACTICES

- Depend on interfaces in function parameters (`Storage&`) to make code testable with fakes
- Use NVI when every override needs the same logging, locking, or validation around it
- Consider a C++20 `concept` instead when the set of types is known at compile time

QUICK SUMMARY

- `virtual f() = 0` — pure virtual
- abstract class — can't instantiate
- interface — only pure virtuals + virtual dtor
- NVI — public non-virtual wrapper, private virtual hooks

Multiple & Virtual Inheritance

MORE THAN ONE BASE

C++ allows a class to inherit from **multiple** base classes. This works well for combining independent interfaces or **mixins**, but introduces name ambiguity and the **diamond problem**: if two bases share a common base, the derived class contains two copies of it. **Virtual inheritance** (`: virtual public Base`) merges them into one shared subobject, at the cost of extra indirection and special construction rules.

NOTES

- Resolve name clashes with `Base1::f()` qualification
- With virtual bases, the **most derived** class constructs the shared base
- Inheriting several pure interfaces is common and safe
- Virtual inheritance is rare outside framework code

DIAMOND & VIRTUAL INHERITANCE

```
struct Device {
    explicit Device(int id) : id(id) {}
    int id;
};
struct Scanner : virtual Device {                // virtual base
    Scanner() : Device(0) {}
    void scan() {}
};
struct Printer : virtual Device {
    Printer() : Device(0) {}
    void print() {}
};
struct Copier : Scanner, Printer {
    explicit Copier(int id) : Device(id) {}      // most derived builds Device
};
Copier c{42};
c.id;                     // 42 — only one Device subobject
c.scan(); c.print();
struct Loggable { void log() const {} };         // mixins
struct Serializable { std::string toJson() const; };
struct Order : Loggable, Serializable {};        // independent features
```

CODE EXPLAINED

- → `virtual Device` — Scanner and Printer share one `Device`
- → `Copier(int id) : Device(id)` — the most derived class initializes the virtual base
- → `Device(0)` calls in Scanner/Printer — ignored when building a `Copier`
- → without `virtual` — `c.id` would be ambiguous (two copies)
- → mixins — unrelated small bases add capabilities

NAME AMBIGUITY

PROBLEM	FIX
both bases have f()	obj.Base1::f()
bring one into scope	using Base1::f;
diamond, two copies	virtual inheritance
cast to common base	cast via one path first

WHEN TO USE

PATTERN	RECOMMENDED?
several pure interfaces	✔
stateless mixins	✔
diamond with data	avoid if possible
virtual bases everywhere	✖

⚠ COMMON MISTAKES

- Accidentally creating a diamond without `virtual`, then hitting "request for member 'id' is ambiguous"
- Forgetting that the most derived class must initialize virtual bases; intermediate initializers are skipped
- Using multiple inheritance of implementation where composition would be clearer

✓ TIPS & BEST PRACTICES

- Limit multiple inheritance to interfaces and empty mixins
- If you need a diamond, make the shared base an interface with no data
- Consider CRTP mixins (`struct Order : Loggable<Order>`) for compile-time features

QUICK SUMMARY

- `class D : A, B` — multiple bases
- diamond — shared base duplicated
- `virtual` inheritance — one shared base subobject
- mixins — small, independent capability bases

Sheet 073

Operator Overloading — Basics

arithmetic operators · member vs non-member · operator<< / >> · operator== / <=> · operator[] · hidden friends

CUSTOM OPERATORS

Operator overloading lets user-defined types use built-in operator syntax: `a + b`, `std::cout << p`, `v[i]`. An overloaded operator is just a function named `operator@`. Implement **symmetric** binary operators (+, ==) as non-members, often **hidden friends** defined inside the class, so conversions work on both sides. Implement compound assignment (+=) as a member and build + on top of it.

RULES

- You can't invent new operators or change precedence
- `::`, `.`, `.*`, `?:`, `sizeof` can't be overloaded
- At least one operand must be a user-defined type
- Defaulted `<=>` and `==` (C++20) cover all six comparisons

VECTOR2 WITH OPERATORS

```
struct Vec2 {
    double x{}, y{};
    Vec2& operator+=(const Vec2& o) { x += o.x; y += o.y; return *this; }
    friend Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }
    friend Vec2 operator*(Vec2 v, double k) { return {v.x * k, v.y * k}; }
    friend Vec2 operator*(double k, Vec2 v) { return v * k; }
    Vec2 operator-() const { return {-x, -y}; }        // unary
    auto operator<=>(const Vec2&) const = default;     // ==, <, …
    double& operator[](std::size_t i) { return i == 0 ? x : y; }
    friend std::ostream& operator<<(std::ostream& os, const Vec2& v) {
        return os << '(' << v.x << ", " << v.y << ')';
    }
};
Vec2 a{1, 2}, b{3, 4};
a + b;               // (4, 6)
2.0 * a;             // (2, 4)
a == Vec2{1, 2};     // true
a[1] = 9;            // a == (1, 9)
std::cout << -a;     // (-1, -9)
```

CODE EXPLAINED

- `operator+=` — member; modifies and returns `*this`
- `operator+` via `+=` — takes `a` by value, reuses the logic
- hidden friend — found only via argument-dependent lookup; works with conversions on either side
- `operator<=> = default` — memberwise comparisons
- `operator<<` — must be a non-member (left operand is the stream)

MEMBER OR NON-MEMBER?

OPERATOR	IMPLEMENT AS
`= [] () ->`	member (required)
`+= -= *=`	member
`+ - * / == <=>`	non-member / hidden friend
`<< >> (streams)`	non-member
`unary - ! ~`	member

CANONICAL SIGNATURES

OPERATOR	SIGNATURE
`+=`	T& operator+=(const T&)
`+`	T operator+(T, const T&)
`==`	bool operator==(const T&) const
`<=>`	auto operator<=>(const T&) const
`[]`	T& operator[](size_t)

⚠ COMMON MISTAKES

- Overloading operators with surprising meanings (`+` that removes items); readers assume normal semantics
- Making `operator+` a member, so `2.0 * v` or `"a" + str` won't compile
- Returning a reference from `operator+`; it returns a new value

✓ TIPS & BEST PRACTICES

- Implement `op=` first, then `op` in terms of it
- Default `<=>` and `==` instead of writing six comparison operators
- Provide a `const` overload of `operator[]` returning `const T&`

QUICK SUMMARY

- `operator@` — overload by naming a function
- `+=` member, `+` non-member — canonical pattern
- hidden friends — symmetric operators inside the class
- `<=> = default` — all comparisons

Operator Overloading — Advanced

operator() · conversion operators · explicit operator bool · ++ prefix / postfix · copy-and-swap · user-defined literals

SPECIAL-PURPOSE OPERATORS

Some operators unlock idioms beyond arithmetic. `operator()` makes an object callable (a **functor**), the basis of comparators and lambdas. **Conversion operators** let a type convert to another; mark them `explicit` to avoid surprises, especially `explicit operator bool`. Increment comes in **prefix** and **postfix** forms, and **user-defined literals** (`operator""`) let you write values like `10_km`.

NOTES

- Postfix `++` takes a dummy `int` parameter
- `explicit operator bool` still works in `if` and `!` (contextual conversion)
- User-defined literal suffixes must start with `_`
- `operator[]` can take multiple indexes since C++23

FUNCTORS, CONVERSIONS, ++, LITERALS

```
struct ByLength {                                          // functor
    bool operator()(const std::string& a, const std::string& b) const {
        return a.size() < b.size();
    }
};
std::ranges::sort(words, ByLength{});
class Handle {
    int fd_ = -1;                                          // private
public:
    explicit operator bool() const { return fd_ >= 0; }  // if (h) OK
};
struct Counter {
    int n = 0;
    Counter& operator++() { ++n; return *this; }           // prefix
    Counter operator++(int) { auto old = *this; ++n; return old; } // postfix
};
struct Distance { double meters; };
constexpr Distance operator""_km(long double v) {
    return {static_cast<double>(v * 1000)};
}
auto d = 2.5_km;                                           // Distance{2500}
```

CODE EXPLAINED

→ `operator()` — instances can be called like functions

→ `explicit operator bool` — allows `if (h)`, blocks `int x = h;`

→ prefix returns a reference; postfix returns the old value by copy

→ `operator""_km` — user-defined literal suffix

→ `long double` — required parameter type for floating literals

OPERATORS & IDIOMS

OPERATOR	IDIOM
`operator()`	functor / comparator
`explicit operator bool`	validity check
`operator T()`	conversion to T
`operator-> / operator*`	smart pointers, iterators
`operator"" _x`	user-defined literals
`operator new / delete`	custom allocation

STANDARD LITERALS

LITERAL	TYPE (NAMESPACE)
`"hi"s`	std::string (std::literals)
`"hi"sv`	std::string_view
`10ms, 2s, 1h`	std::chrono durations
`2i`	std::complex<double>

⚠ COMMON MISTAKES

- Non-explicit `operator bool`, which allows nonsense like `h1 + h2` or `int x = handle;`
- Writing postfix `++` without the `int` parameter, or returning a reference to the old value
- Implicit conversion operators that create ambiguous overloads

✓ TIPS & BEST PRACTICES

- Prefer `++it` over `it++` for iterators; postfix creates a copy
- Put user-defined literals in a namespace (`namespace units::literals`) and import them where needed
- Use copy-and-swap in assignment operators for the strong exception guarantee

QUICK SUMMARY

- `operator()` — callable objects
- `explicit operator bool` — safe truthiness
- `++x` vs `x++` — `operator++()` vs `operator++(int)`
- `operator""_suffix` — user-defined literals

Reading Files

INPUT FILE STREAMS

`std::ifstream` (<fstream>) reads files with the same >> and `std::getline` interface as `std::cin`. Opening happens in the constructor and closing in the destructor (RAII), so you rarely call `close()`. Always check that the stream opened, and loop on the **read operation itself**: the stream converts to `false` once a read fails or hits end of file.

NOTES

- `if (!in)` or `!in.is_open()` — file missing or unreadable
- `while (std::getline(in, line))` — the idiomatic line loop
- `in >> x` skips whitespace; `getline` keeps it
- Read a whole file in one go with `std::istreambuf_iterator`

READING PATTERNS

```cpp
#include <fstream>

std::ifstream in{"scores.txt"};
if (!in) throw std::runtime_error("cannot open scores.txt");

std::string line;
while (std::getline(in, line)) {          // line by line
    if (line.empty() || line.starts_with('#')) continue;
    process(line);
}

std::ifstream nums{"data.txt"};
int x, sum = 0;
while (nums >> x) sum += x;               // whitespace-separated ints

std::ifstream all{"config.json"};
std::string text{std::istreambuf_iterator<char>{all}, {}};  // whole file
```

CODE EXPLAINED

→ constructor opens; destructor closes automatically

→ `if (!in)` — check before reading anything

→ `getline` returns the stream, which is false at EOF

→ `nums >> x` — stops at EOF or the first non-number

→ `istreambuf_iterator` — slurp the entire file into a string

STREAM STATE

CHECK	TRUE WHEN
`if (in)`	last operation succeeded
`in.good()`	no flags set
`in.eof()`	end of file reached
`in.fail()`	parse error or EOF
`in.bad()`	unrecoverable I/O error
`in.clear()`	reset flags to retry

READING FUNCTIONS

CALL	READS
`in >> x`	one token, parsed
`std::getline(in, s)`	one line
`std::getline(in, s, ';')`	up to a delimiter
`in.get(c)`	one char
`in.read(buf, n)`	n raw bytes
`in.ignore(n, '\n')`	skip input

⚠ COMMON MISTAKES

- `while (!in.eof()) { getline(in, line); … }` processes a bogus last iteration; loop on `getline` itself
- Not checking whether the file opened, then silently processing nothing
- Using relative paths and being surprised by the working directory; print `std::filesystem::current_path()`

✓ TIPS & BEST PRACTICES

- Use `std::filesystem::path` for file names; streams accept it directly
- Enable exceptions for strict code: `in.exceptions(std::ios::badbit);`
- Speed up big reads by slurping the file, then parsing the string

QUICK SUMMARY

- `std::ifstream in{path}` — open for reading
- `if (!in)` — check it opened
- `while (getline(in, line))` — read lines
- RAII — closes automatically

Writing & Appending Files

OUTPUT FILE STREAMS

`std::ofstream` writes files with <<, just like `std::cout`. By default it **truncates** an existing file; open with `std::ios::app` to append. `std::fstream` can read and write the same file. Output is buffered and flushed when the stream is destroyed, so data is safe on normal exit. C++23's `std::print` writes to a `FILE*`, and `std::format_to` works with a stream iterator.

NOTES

- Default mode for ofstream: `out | trunc`
- `std::ios::app` — every write goes to the end
- Check `if (!out)` after opening and after writing
- `flush()` forces buffered data to the OS

WRITING & APPENDING

```cpp
#include <fstream>

std::ofstream out{"report.txt"};                  // create / truncate
if (!out) throw std::runtime_error("cannot write report.txt");
out << "Name,Score\n";
for (const auto& [name, score] : results)
    out << name << ',' << score << '\n';

std::ofstream log{"app.log", std::ios::app};      // append
log << std::format("[{}] started\n", timestamp());
log.flush();                                      // push to disk now

std::fstream db{"data.bin", std::ios::in | std::ios::out | std::ios::binary};

std::FILE* f = std::fopen("out.txt", "w");
std::println(f, "pi = {:.3f}", 3.14159);          // C++23 print to FILE*
std::fclose(f);
```

CODE EXPLAINED

- → `ofstream{"report.txt"}` — truncates if the file exists
- → loop with structured bindings — one CSV row per entry
- → `std::ios::app` — preserves existing content
- → `std::fstream` with `in | out` — read and write one file
- → `std::println(FILE*, …)` — formatted output to C files

OPEN MODES

FLAG	MEANING
`std::ios::in`	read
`std::ios::out`	write
`std::ios::app`	append to end
`std::ios::trunc`	discard existing content
`std::ios::ate`	start at end, can seek
`std::ios::binary`	no newline translation

COMMON COMBOS

MODE	EFFECT
`out`	create / truncate
`out | app`	create / append
`in | out`	read+write, must exist
`in | out | trunc`	read+write, truncate
`out | binary`	raw bytes

⚠ COMMON MISTAKES

- Opening a log with plain `ofstream` and wiping it every run; use `std::ios::app`
- Using `std::endl` on every line of a large file; each one flushes. Use `'\n'`
- Assuming data is on disk right after `<<`; it may still be buffered if the program crashes

✓ TIPS & BEST PRACTICES

- Write to a temporary file, then `std::filesystem::rename` it over the original for atomic saves
- Check `out` after the last write to catch disk-full errors
- Use `std::format` to build each line, then write it in one `<<`

QUICK SUMMARY

- `std::ofstream` — write (truncates by default)
- `std::ios::app` — append
- `std::fstream` — read + write
- `flush()` — force buffered data out

Binary Files

RAW BYTES

Open with `std::ios::binary` to read and write raw bytes with no newline translation (which matters on Windows). Use `write(ptr, n)` and `read(ptr, n)`, which take a `char*`, so you `reinterpret_cast` your object's address. That's only safe for **trivially copyable** types, and the bytes depend on the platform's **endianness**, padding, and type sizes. `seekg`/`tellg` move around the file.

NOTES

- `gcount()` — bytes actually read by the last `read`
- `std::endian::native` (C++20) — check byte order
- `std::byteswap` (C++23) — reverse bytes of an integer
- Use fixed-width types (`std::uint32_t`) for file formats

BINARY READ & WRITE

```cpp
#include <fstream>
#include <bit>

struct Header {
    std::uint32_t magic; std::uint16_t version; std::uint16_t count;
};
static_assert(std::is_trivially_copyable_v<Header>);

Header h{0xCAFEBABE, 2, 10};
std::ofstream out{"data.bin", std::ios::binary};
out.write(reinterpret_cast<const char*>(&h), sizeof h);
std::ifstream in{"data.bin", std::ios::binary};
Header r{};
in.read(reinterpret_cast<char*>(&r), sizeof r);
if (in.gcount() != sizeof r) throw std::runtime_error("truncated");
in.seekg(0, std::ios::end);
auto size = in.tellg();                          // file size in bytes
in.seekg(0);                                     // back to start
std::vector<char> bytes(size);
in.read(bytes.data(), size);
std::uint32_t be = std::byteswap(h.magic);       // little ↔ big endian
```

CODE EXPLAINED

- → `std::ios::binary` — bytes in, bytes out
- → `reinterpret_cast<const char*>(&h)` — view the struct as bytes
- → `static_assert(is_trivially_copyable_v)` — guard against unsafe types
- → `seekg(0, end)` + `tellg()` — find the file size
- → `std::byteswap` — convert endianness (C++23)

POSITIONING

CALL	PURPOSE
`seekg(pos)`	move read position
`seekg(off, ios::cur / end)`	relative move
`tellg()`	current read position
`seekp / tellp`	same for write position
`gcount()`	bytes read last time

SAFE VS UNSAFE TO WRITE()

TYPE	SAFE?
`int, double, uint32_t`	✔ (watch endianness)
`struct of those`	✔ (watch padding)
`std::string`	✖ (holds a pointer)
`std::vector`	✖ (write data() + size)
`class with virtuals`	✖ (vptr)

⚠ COMMON MISTAKES

- Writing a `std::string` or `std::vector` object with `write`; you save pointers, not data
- Forgetting `std::ios::binary` on Windows, which corrupts bytes `0x0A` / `0x0D`
- Assuming struct layout matches another compiler or CPU; padding and endianness differ

✓ TIPS & BEST PRACTICES

- Serialize field by field in a defined byte order for portable formats
- Use `std::filesystem::file_size(path)` instead of seek/tell to get the size
- Prefer a format library (Protocol Buffers, FlatBuffers, Cap'n Proto) for anything shared

QUICK SUMMARY

- `std::ios::binary` — raw byte I/O
- `write / read` — with `reinterpret_cast<char*>`
- trivially copyable — only types safe to dump
- `seekg / tellg` — positioning

std::filesystem

PATHS & DIRECTORIES

<filesystem> (C++17) provides portable path manipulation and file system operations. `std::filesystem::path` handles separators and joins with /. Free functions query and change the file system: existence, size, directories, copy, rename, remove. Most functions have two overloads: one that **throws** `filesystem_error` and one that takes a `std::error_code&` and reports errors without throwing.

NOTES

- Alias it: `namespace fs = std::filesystem;`
- `path / "sub" / "file.txt"` joins with the right separator
- `directory_iterator` — one level; `recursive_directory_iterator` — all levels
- Iteration order is unspecified

STD::FILESYSTEM BASICS

```
#include <filesystem>
namespace fs = std::filesystem;

fs::path p = fs::path{"logs"} / "2026" / "app.log";   // logs/2026/app.log
p.filename();        // "app.log"
p.stem();            // "app"
p.extension();       // ".log"
p.parent_path();     // "logs/2026"

fs::create_directories(p.parent_path());   // mkdir -p
fs::exists(p);                             // true / false
if (fs::is_regular_file(p)) fs::file_size(p);   // bytes
fs::copy_file(p, "backup.log", fs::copy_options::overwrite_existing);
fs::rename("backup.log", "old.log");
fs::remove("old.log");                     // true if removed

for (const auto& e : fs::directory_iterator{"logs"})
    std::println("{} {}", e.path().filename().string(), e.is_directory());
std::error_code ec;
fs::remove_all("tmp", ec);                 // no throw; check ec
```

CODE EXPLAINED

- → `/` — joins path components portably
- → `filename` / `stem` / `extension` — decompose a path
- → `create_directories` — creates every missing level
- → `directory_iterator` — entries in one folder
- → `error_code` overload — report errors without exceptions

PATH QUERIES

MEMBER	"A/B/C.TXT" →
`filename()`	c.txt
`stem()`	c
`extension()`	.txt
`parent_path()`	a/b
`replace_extension(".md")`	a/b/c.md
`is_absolute()`	false

OPERATIONS

FUNCTION	EFFECT
`exists(p)`	path exists?
`create_directories(p)`	mkdir -p
`remove(p) / remove_all(p)`	delete file / tree
`copy(src, dst, opts)`	copy files / dirs
`current_path()`	working directory
`temp_directory_path()`	OS temp dir

⚠ COMMON MISTAKES

- Concatenating with `+` and hard-coded `"/"` or `"\\"`; use `operator/`
- Calling `fs::file_size` on a directory, which throws
- Checking `exists(p)` and then opening it; the file can change in between (race). Just open and handle failure

✓ TIPS & BEST PRACTICES

- Use `p.string()` / `p.u8string()` when printing or passing to APIs that need text
- Filter by extension: `if (e.path().extension() == ".cpp")`
- Use `fs::weakly_canonical(p)` to normalize paths that may not exist yet

QUICK SUMMARY

- `fs::path` + `/` — portable paths
- `exists / file_size / create_directories` — queries and setup
- `directory_iterator` — list a folder
- `error_code` overloads — no-throw variants

Parsing Text & CSV

PARSING TEXT FORMATS

The standard library has no CSV or JSON parser, but simple formats are easy with `std::getline` using a **delimiter**, `std::istringstream` for typed fields, and C++20/23 ranges (`views::split`). Use `std::from_chars` to convert fields to numbers quickly and with error reporting. Real CSV (quoted fields with embedded commas) needs a small state machine or a library.

NOTES

- `getline(ss, field, ',')` — split one line on commas
- `std::quoted` reads/writes `"quoted strings"` with escapes
- Trim whitespace before converting numbers
- Libraries: `rapidcsv`, `csv-parser`; JSON → `nlohmann/json`

CSV ROUND TRIP

```cpp
struct Row { std::string name; int qty; double price; };
std::vector<Row> readCsv(const fs::path& file) {
    std::ifstream in{file};
    std::vector<Row> rows;
    std::string line;
    std::getline(in, line);                    // skip header
    while (std::getline(in, line)) {
        std::istringstream ss{line};
        Row r; std::string qty, price;
        std::getline(ss, r.name, ','); std::getline(ss, qty, ',');
        std::getline(ss, price);
        std::from_chars(qty.data(), qty.data() + qty.size(), r.qty);
        r.price = std::stod(price);
        rows.push_back(std::move(r));
    }
    return rows;
}
for (auto f : std::string_view{"a,b,c"} | std::views::split(',')) {}
```

CODE EXPLAINED

- → first `getline` — consume the header row
- → `istringstream ss{line}` — treat one line as a stream
- → `getline(ss, field, ',')` — read up to the next comma
- → `from_chars` / `stod` — convert the text fields
- → `views::split(',')` — lazy split of a `string_view` (C++20)

SPLITTING OPTIONS

TECHNIQUE	NOTES
`getline(ss, f, ',')`	simple, allocates strings
`views::split(',')`	lazy subranges (C++20)
`find + substr loop`	manual control
`std::regex`	flexible, slow
`std::quoted`	handles "a, b" fields

WRITE CSV

CODE	OUTPUT
`out << a << ',' << b << '\n'`	a,b
`std::format("{},{}\n", a, b)`	a,b
`out << std::quoted(s)`	"x, y"
`std::print(f, "{:.2f}", p)`	3.50

⚠ COMMON MISTAKES

- Splitting quoted fields like `"Smith, Jr."` on commas; handle quotes or use a library
- Forgetting Windows line endings: lines keep a trailing `'\r'`; strip it
- Converting with `std::stoi` on untrusted data without catching exceptions

✓ TIPS & BEST PRACTICES

- Read a whole line first, then parse it; error messages can include the line number
- Wrap field conversion in a helper returning `std::optional` or `std::expected`
- Collect split pieces with `std::ranges::to<std::vector>()` (C++23)

QUICK SUMMARY

- `getline(ss, field, ',')` — classic CSV split
- `std::istringstream` — parse one line
- `views::split` — lazy splitting
- `std::from_chars` — fast numeric conversion

Headers & Include Guards

HEADER FILES

A **header** declares the interface that other `.cpp` files need: classes, function prototypes, templates, inline functions, and constants. `#include` pastes the header's text into each translation unit, so a header can end up included several times in one TU. **Include guards** or `#pragma once` stop the second inclusion. **Forward declarations** (`class Widget;`) cut compile times by avoiding includes when only a pointer or reference is needed.

NOTES

- `<header>` — searched in system/include paths
- `"header.h"` — searched next to the including file first
- `#pragma once` is non-standard but supported everywhere
- Each header should compile on its own (include what you use)

HEADER + SOURCE PAIR

```
// inventory.h
#pragma once                        // or classic include guards below
#include <string>                   // needed: used by value
#include <vector>
class Supplier;                     // forward declaration: only a pointer

class Inventory {
public:
    void add(std::string name, int qty);
    int  total() const;
    void setSupplier(Supplier* s) { supplier_ = s; }
private:
    std::vector<std::pair<std::string, int>> items_;
    Supplier* supplier_ = nullptr;
};
// inventory.cpp
#include "inventory.h"              // own header first
#include "supplier.h"               // full definition needed here
```

CODE EXPLAINED

- → `#pragma once` — the compiler skips the file if already included
- → `#include <string>` — the header needs the full type
- → `class Supplier;` — enough for pointers and references
- → definitions go in `inventory.cpp`
- → own header first — proves it's self-contained

INCLUDE GUARDS

CLASSIC FORM	PURPOSE
`#ifndef INVENTORY_H`	skip if already defined
`#define INVENTORY_H`	mark as included
`… contents …`	
`#endif`	end of guarded region

WHAT GOES WHERE

HEADER (.H / .HPP)	SOURCE (.CPP)
`class definitions`	member function bodies
`function declarations`	function definitions
`templates (full)`	static / anonymous helpers
`inline / constexpr functions`	global variable definitions

⚠ COMMON MISTAKES

- `using namespace std;` in a header, which forces it on every file that includes it
- Defining non-inline functions or globals in a header, causing "multiple definition" link errors
- Circular includes between two headers; break the cycle with forward declarations

✓ TIPS & BEST PRACTICES

- Include what you use; don't rely on another header pulling it in transitively
- Forward-declare in headers, include in `.cpp` files, to speed up builds
- Use `.hpp` for C++-only headers if your project mixes C and C++

QUICK SUMMARY

- header — declarations and inline definitions
- `#pragma once` / guards — prevent double inclusion
- forward declaration — `class X;` for pointers/refs
- `.cpp` — definitions

The Preprocessor

TEXT SUBSTITUTION BEFORE COMPILING

The **preprocessor** runs before the compiler and works on text: `#include` pastes files, `#define` creates **macros**, and `#if` / `#ifdef` include or drop code. Macros ignore scope, namespaces, and types, so modern C++ replaces most uses with `constexpr`, `inline` functions, templates, and `std::source_location`. Conditional compilation for platforms and feature flags remains a legitimate use, and C++26 adds `#embed` for including binary files.

NOTES

- Macro names are conventionally `ALL_CAPS`
- Parenthesize macro parameters and the whole body
- `-DNAME=value` defines a macro from the command line
- `__has_include(<header>)` tests whether a header exists

MACROS & CONDITIONAL COMPILATION

```cpp
#define BUFFER_SIZE 4096                  // object-like (prefer constexpr)
#define SQUARE(x) ((x) * (x))             // function-like
SQUARE(3 + 1);                            // 16 (parens matter)
SQUARE(i++);                              // UB: i incremented twice

#ifdef _WIN32
    #include <windows.h>
#elif defined(__linux__) || defined(__APPLE__)
    #include <unistd.h>
#else
    #error "unsupported platform"
#endif

#if __cpp_lib_print >= 202207L            // feature-test macro
    #include <print>
#endif
void trace(std::source_location loc = std::source_location::current()) {
    std::println("{}:{} {}", loc.file_name(), loc.line(), loc.function_name());
}
```

CODE EXPLAINED

- → object-like macro — plain text substitution
- → `SQUARE(i++)` — argument evaluated twice: a classic macro bug
- → `#ifdef _WIN32` — platform-specific code
- → `#error` — stops compilation with a message
- → `__cpp_lib_print` — check library support before using a feature
- → `std::source_location` — replaces `__FILE__` / `__LINE__` macros (C++20)

DIRECTIVES

DIRECTIVE	PURPOSE
`#include`	paste a file
`#define / #undef`	create / remove macro
`#if / #elif / #else / #endif`	conditional code
`#ifdef / #ifndef`	test if defined
`#error / #warning`	diagnostic (#warning: C++23)
`#embed "file.bin"`	binary data (C++26)

PREDEFINED MACROS

MACRO	VALUE
`__FILE__ / __LINE__`	current file / line
`__cplusplus`	202302L for C++23
`_WIN32 / __linux__ / __APPLE__`	platform
`NDEBUG`	disables assert
`__cpp_concepts etc.`	feature-test macros

⚠ COMMON MISTAKES

- Unparenthesized macros: `#define DOUBLE(x) x * 2` makes `DOUBLE(1 + 2)` equal 5
- Macros with side-effect arguments evaluated twice
- Macro names clashing with real identifiers (`min` / `max` from `windows.h`); define `NOMINMAX`

✓ TIPS & BEST PRACTICES

- Replace constant macros with `constexpr` and function macros with `inline` /template functions
- Use feature-test macros (`__cpp_lib_*`) from `<version>` to support several compilers
- Use `std::source_location` in logging helpers instead of `__FILE__` / `__LINE__`

QUICK SUMMARY

- `#define` — text substitution, no scope or types
- `#if / #ifdef` — conditional compilation
- feature-test macros — detect support
- `std::source_location` — modern file/line info

Namespaces

ORGANIZING NAMES

A **namespace** groups related names and prevents collisions between libraries: `std::vector` and `mylib::vector` can coexist. Refer to names with `::`, bring specific ones into scope with a **using-declaration** (`using std::string;`), or import a whole namespace with a **using-directive** (`using namespace std;`), which is best kept to small local scopes. Namespaces can be nested, aliased, reopened, and made `inline` for versioning.

NOTES

- `namespace a::b { }` — nested shorthand (C++17)
- Namespaces are **open**: add to them in multiple files
- `::name` — the global namespace
- **ADL**: functions are also looked up in their arguments' namespaces

NAMESPACES IN PRACTICE

```
namespace acme::geometry {                // nested (C++17)
    struct Point { double x, y; };
    double distance(Point a, Point b);
}
namespace acme::geometry {                // reopened in another file
    double distance(Point a, Point b) {
        return std::hypot(a.x - b.x, a.y - b.y);
    }
}
acme::geometry::Point p{0, 0}, q{3, 4};
distance(p, q);                           // 5 — found by ADL
namespace geo = acme::geometry;           // alias
void report() {
    using std::println;                   // using-declaration
    using namespace std::chrono_literals; // directive, local scope only
    println("{}", 5s);                    // 5s
}
namespace lib {
    inline namespace v2 { int api(); }    // lib::api → lib::v2::api
}
```

CODE EXPLAINED

- → `acme::geometry` — avoids clashes with other `Point` types
- → reopening — split a namespace across headers and sources
- → ADL — `distance(p, q)` works without qualification
- → `namespace geo = …` — shorter alias
- → `inline namespace v2` — default version for `lib::api`

WAYS TO REFER

FORM	SCOPE EFFECT
`std::vector`	fully qualified (clearest)
`using std::vector;`	one name
`using namespace std;`	everything in std
`namespace fs = std::filesystem;`	alias
`::name`	global namespace

SPECIAL NAMESPACES

KIND	USE
`namespace { }`	file-private (internal linkage)
`inline namespace`	API versioning
`std`	standard library only (don't add to it)
`std::literals`	suffixes like s, sv, ms
`detail / impl`	convention for internals

⚠ COMMON MISTAKES

- `using namespace std;` at file scope in headers; it leaks everywhere and causes ambiguities
- Adding your own declarations to `namespace std` (only specializations like `std::hash` are allowed)
- Unqualified calls that unexpectedly pick an ADL overload from another namespace

✓ TIPS & BEST PRACTICES

- Put all your code in a project namespace
- Use `using namespace` only inside functions, and only for literal namespaces or tight scopes
- Name an internal sub-namespace `detail` to signal "not public API"

QUICK SUMMARY

- `namespace a::b {}` — group names
- `a::b::name` — qualified access
- `using X::name;` — import one name
- `namespace x = a::b;` — alias

C++20 Modules

MODULES

C++20 modules replace textual `#include` with compiled interfaces. A module **interface unit** declares `export module name;` and marks what it exports; consumers write `import name;`. Modules are parsed once, don't leak macros, and are isolated from include order. C++23 adds `import std;` for the whole standard library. Tooling support has matured but still varies, so check your compiler and build system versions.

SUPPORT (LATE 2026)

- GCC 15+, Clang 18+, MSVC 17.x support named modules
- CMake 3.28+ builds modules with Ninja / VS generators
- `import std;` needs CMake 3.30+ and a recent standard library
- Macros from a module are **not** visible to importers

A SIMPLE MODULE

```cpp
// math.cppm — module interface unit
module;                          // global module fragment (optional)
#include <cmath>                 // legacy headers go here
export module math;              // declares the module

export namespace math {
    double hypot(double a, double b) { return std::sqrt(a * a + b * b); }
    constexpr double pi = 3.14159265358979;
}
int helper() { return 42; }      // not exported: module-private

// main.cpp
import math;                     // consume the module
import std;                      // C++23: entire standard library
int main() {
    std::println("{}", math::hypot(3, 4));   // 5
}
```

CODE EXPLAINED

- → `module;` — starts the global module fragment for old `#include`s
- → `export module math;` — names the module
- → `export` — only exported names are visible to importers
- → `helper()` — module-private (similar to internal linkage)
- → `import std;` — replaces dozens of standard `#include`s

UNIT TYPES

UNIT	DECLARATION
`interface`	export module m;
`implementation`	module m;
`partition interface`	export module m:part;
`partition impl`	module m:part;
`import partition`	import :part;
`header unit`	import <vector>;

CMAKE SNIPPET

STEP	CMAKE
`minimum version`	`cmake_minimum_required(VERSION 3.28)`
`library target`	`add_library(math)`
`module files`	`target_sources(math PUBLIC FILE_SET CXX_MODULES FILES math.cppm)`
`import std`	`set(CMAKE_CXX_MODULE_STD ON)` (3.30+)

⚠ COMMON MISTAKES

- Expecting macros (like `NDEBUG` checks or config macros) defined in a module to reach importers
- Putting `#include` after `export module`; legacy includes belong in the global module fragment
- Mixing compilers or flags between a module and its consumers; module binaries aren't portable

✓ TIPS & BEST PRACTICES

- Start by using `import std;` in new code if your toolchain supports it; it's a big compile-time win
- Migrate leaf libraries first and keep header shims for old consumers
- Use `.cppm` (Clang/GCC) or `.ixx` (MSVC) consistently and tell CMake via `FILE_SET CXX_MODULES`

QUICK SUMMARY

- `export module m;` — define a module
- `export` — choose what's visible
- `import m;` — use it; no macro leakage
- `import std;` — standard library as a module (C++23)

Package Managers

GETTING DEPENDENCIES

C++ has no single official package manager, but two dominate. **vcpkg** (Microsoft) builds libraries from source using a `vcpkg.json` manifest and integrates with CMake through a toolchain file. **Conan** uses `conanfile.txt`/`conanfile.py`, supports binary caching, and generates CMake config. CMake's own **FetchContent** downloads and builds dependencies as part of your project without any external tool.

NOTES

- All three end with `find_package(...)` + `target_link_libraries(...)`
- vcpkg **manifest mode** pins dependencies per project
- Conan 2.x is current; Conan 1 recipes differ
- Header-only libraries can also be vendored into `third_party/`

VCPKG MANIFEST + CMAKE

```
{
  "name": "inventory",
  "version": "1.0.0",
  "dependencies": [ "fmt", "spdlog", "nlohmann-json", "gtest" ]
}
```

CODE EXPLAINED

- → `vcpkg.json` — lives next to the top-level `CMakeLists.txt`
- → dependencies — installed automatically at configure time
- → configure with the toolchain: `-DCMAKE_TOOLCHAIN_FILE=…/vcpkg.cmake`
- → then `find_package(fmt CONFIG REQUIRED)` and link `fmt::fmt`

FETCHCONTENT (NO EXTERNAL TOOL)

```
include(FetchContent)
FetchContent_Declare(
  json
  URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz
)
FetchContent_MakeAvailable(json)
target_link_libraries(app PRIVATE nlohmann_json::nlohmann_json)
```

CODE EXPLAINED

- → `FetchContent_Declare` — where to get the source (URL or git tag)
- → `FetchContent_MakeAvailable` — downloads and adds it to the build
- → then link the library's exported target as usual

COMPARISON

TOOL	MANIFEST	BINARIES
`vcpkg`	vcpkg.json	source builds + binary cache
`Conan`	conanfile.txt / .py	prebuilt + cache
`FetchContent`	CMakeLists.txt	source, every clean build
`system pkg`	apt / brew	prebuilt, global

COMMON COMMANDS

COMMAND	EFFECT
`vcpkg new --application`	create vcpkg.json
`vcpkg add port fmt`	add a dependency
`vcpkg search json`	find packages
`conan install . --build=missing`	fetch / build deps
`conan profile detect`	create default profile

⚠ COMMON MISTAKES

- Mixing system-installed and vcpkg copies of the same library, which links two versions
- Forgetting the vcpkg toolchain file, so `find_package` can't find anything
- Pinning nothing; builds change when upstream releases a new version

✓ TIPS & BEST PRACTICES

- Commit `vcpkg.json` (plus a `builtin-baseline`) or `conan.lock` for reproducible builds
- Use FetchContent for small, header-only, or test-only dependencies
- Put the toolchain path in `CMakePresets.json` so nobody has to remember it

QUICK SUMMARY

- vcpkg — manifest-based, CMake toolchain
- Conan — profiles, binary packages
- FetchContent — CMake-native download + build
- `find_package` + `target_link_libraries` — the consumer side

Math & Random

NUMBERS & RANDOMNESS

<cmath> provides the usual math functions, overloaded for `float`, `double`, and `long double`, and <numbers> (C++20) provides exact constants like `std::numbers::pi`. <random> separates a random **engine** (the bit source) from a **distribution** (the shape of the output). Seed an engine once and reuse it; never use `rand() % n`, which is low quality and biased.

NOTES

- `std::mt19937` — good general-purpose engine
- `std::random_device` — non-deterministic seed source
- Distributions: `uniform_int`, `uniform_real`, `normal`, `bernoulli`
- Same seed → same sequence (great for tests)

<CMATH>, <NUMBERS>, <RANDOM>

```cpp
#include <cmath>
#include <numbers>
#include <random>

std::sqrt(16.0);            // 4
std::pow(2.0, 10);          // 1024
std::hypot(3.0, 4.0);       // 5
std::abs(-7);               // 7
std::log(std::numbers::e);  // 1
std::sin(std::numbers::pi / 2);   // 1

std::mt19937 rng{std::random_device{}()};          // seed once
std::uniform_int_distribution<int> die{1, 6};
die(rng);                                          // 1..6 inclusive
std::uniform_real_distribution<double> unit{0.0, 1.0};
unit(rng);                                         // [0, 1)
std::normal_distribution<double> height{170.0, 8.0};
std::ranges::shuffle(deck, rng);                   // shuffle a vector
std::mt19937 test_rng{42};                         // reproducible
```

CODE EXPLAINED

- `<cmath>` functions — overloaded for each floating type
- `std::numbers::pi` / `e` — exact constants (C++20)
- `random_device{}()` — one call to seed the engine
- `uniform_int_distribution{1, 6}` — inclusive both ends
- `std::ranges::shuffle` — uniform shuffle with your engine

<CMATH>

FUNCTION	RESULT
`pow(b, e) / sqrt / cbrt`	powers and roots
`exp / log / log10 / log2`	exponentials
`sin / cos / tan / atan2`	trig (radians)
`floor / ceil / round / trunc`	rounding
`fmin / fmax / fma`	min / max / fused mul-add
`std::gcd / lcm (numeric)`	integer helpers

<RANDOM> PIECES

TYPE	ROLE
`std::mt19937 / mt19937_64`	engine (32/64-bit)
`std::random_device`	seed source
`uniform_int_distribution`	ints in [a, b]
`uniform_real_distribution`	reals in [a, b)
`normal_distribution`	bell curve (mean, sd)
`bernoulli_distribution`	true with probability p

⚠ COMMON MISTAKES

- Creating a new engine (and seed) inside a loop or function on every call; seed once and reuse
- Using `rand() % 6 + 1`; it's biased and `rand()` quality is poor
- Passing degrees to `std::sin`; it expects radians. Multiply by `std::numbers::pi / 180`

✓ TIPS & BEST PRACTICES

- Make the engine a function-local `static` or a member so it's created once
- Use a fixed seed in tests for reproducible runs
- Use `std::lerp(a, b, t)` and `std::clamp` (C++20/17) instead of hand-written formulas

QUICK SUMMARY

- `<cmath>` — sqrt, pow, trig, rounding
- `std::numbers::pi` — exact constants
- engine + distribution — the `<random>` model
- `std::mt19937 rng{seed}` — seed once, reuse

Sheet 086

Dates, Times & std::chrono

durations · time_point · steady_clock vs system_clock · chrono literals
5s / 10ms · year_month_day · time zones

TIME IN C++

<chrono> models time with three ideas: a **duration** (an amount of time with a unit), a **clock** (a source of time), and a **time_point** (a duration since a clock's epoch). Units convert automatically when lossless; lossy conversions need `duration_cast`. C++20 added **calendar** types (`year_month_day`) and **time zones** (`zoned_time`), so most date work no longer needs a third-party library.

CLOCKS

- `steady_clock` — monotonic; use for measuring elapsed time
- `system_clock` — wall-clock time; can jump (NTP, DST)
- `high_resolution_clock` — often an alias; prefer `steady_clock`
- Literals: `using namespace std::chrono_literals;` → `5s`, `250ms`, `2h`

DURATIONS, TIMING, DATES

```cpp
#include <chrono>
using namespace std::chrono;
using namespace std::chrono_literals;

auto timeout = 1500ms;
duration_cast<seconds>(timeout).count();   // 1  (truncated)
auto total = 2min + 30s;                   // 150s
std::this_thread::sleep_for(100ms);

auto start = steady_clock::now();
doWork();
auto ms = duration_cast<milliseconds>(steady_clock::now() - start);
std::println("took {}", ms);               // took 42ms

year_month_day today = floor<days>(system_clock::now());
auto due = today + months{1};              // same day next month
auto wd = weekday{sys_days{today}};        // e.g. Sun
zoned_time ny{"America/New_York", system_clock::now()};
std::println("{:%Y-%m-%d %H:%M %Z}", ny);  // 2026-10-11 09:30 EDT
```

CODE EXPLAINED

- → `1500ms` — chrono literal
- → `duration_cast<seconds>` — explicit lossy conversion
- → `steady_clock` for timing — immune to wall-clock changes
- → `floor<days>` — time_point → calendar date
- → `zoned_time` — time in a named IANA zone
- → chrono types format directly with `std::format`

DURATIONS

TYPE	LITERAL
nanoseconds	10ns
microseconds	10us
milliseconds	10ms
seconds	10s
minutes / hours	10min / 10h
days / weeks / years	C++20 types

FORMAT SPECIFIERS

SPEC	OUTPUT
%Y-%m-%d	2026-10-11
%H:%M:%S	14:05:09
%a / %A	Sun / Sunday
%b / %B	Oct / October
%Z / %z	EDT / -0400
%F / %T	ISO date / time

⚠ COMMON MISTAKES

- Measuring elapsed time with `system_clock`; a clock adjustment can make it negative. Use `steady_clock`
- Calling `.count()` and losing the unit; keep values as durations until printing
- Mixing local time and UTC without a time zone; use `zoned_time`

✓ TIPS & BEST PRACTICES

- Write a small `Timer` RAII class that prints elapsed `steady_clock` time in its destructor
- Accept durations in APIs (`void wait(std::chrono::milliseconds)`) instead of raw ints
- Parse dates with `std::chrono::parse` and an `istringstream`

QUICK SUMMARY

- `duration` — amount of time with units
- `steady_clock` — measure elapsed time
- `year_month_day` — calendar dates (C++20)
- `zoned_time` — time zones (C++20)

GoogleTest

GOOGLE'S TEST FRAMEWORK

GoogleTest (gtest) is the most widely used C++ unit-testing framework, and **gMock** (bundled with it) adds mock objects. Tests are written with macros: `TEST(Suite, Name)` for standalone tests and `TEST_F(Fixture, Name)` for tests sharing setup. `EXPECT_*` assertions record a failure and keep going; `ASSERT_*` stop the current test. CMake's `gtest_discover_tests` registers every test with CTest.

SETUP

- vcpkg: `gtest`; CMake: `find_package(GTest CONFIG REQUIRED)`
- Link `GTest::gtest_main` to get a ready-made `main()`
- Run: `ctest --test-dir build` or the test binary directly
- Filter: `./tests --gtest_filter=Cart.*`

TESTS & A FIXTURE

```cpp
#include <gtest/gtest.h>
#include "cart.h"

TEST(PriceTest, AppliesDiscount) {
    EXPECT_DOUBLE_EQ(applyDiscount(100.0, 0.2), 80.0);
    EXPECT_THROW(applyDiscount(100.0, 1.5), std::invalid_argument);
}
class CartTest : public ::testing::Test {
protected:
    void SetUp() override { cart.add("apple", 2); }   // before each test
    Cart cart;
};
TEST_F(CartTest, CountsItems) {
    ASSERT_EQ(cart.size(), 1u);                        // stop if wrong
    EXPECT_EQ(cart.quantity("apple"), 2) << "apples should be 2";
}
```

CODE EXPLAINED

- → `TEST(Suite, Name)` — a standalone test case
- → `EXPECT_DOUBLE_EQ` — floating comparison with ULP tolerance
- → `EXPECT_THROW(stmt, Type)` — checks an exception type
- → fixture `SetUp()` — runs before every `TEST_F`
- → `<< "msg"` — extra context printed on failure

ASSERTIONS

MACRO	CHECKS
`EXPECT_EQ / NE`	== / !=
`EXPECT_LT / LE / GT / GE`	ordering
`EXPECT_TRUE / FALSE`	boolean
`EXPECT_STREQ`	C-string equality
`EXPECT_NEAR(a, b, eps)`	float within eps
`EXPECT_THROW / NO_THROW`	exceptions

CMAKE WIRING

LINE	PURPOSE
`enable_testing()`	turn on CTest
`add_executable(tests t.cpp)`	test binary
`target_link_libraries(tests GTest::gtest_main)`	framework + main
`include(GoogleTest)`	load helpers
`gtest_discover_tests(tests)`	register each TEST

⚠ COMMON MISTAKES

- Using `ASSERT_*` in a helper that returns a value; ASSERT returns `void` from the function. Use EXPECT or return status
- Comparing floats with `EXPECT_EQ`; use `EXPECT_DOUBLE_EQ` or `EXPECT_NEAR`
- Tests that depend on run order or shared global state

✓ TIPS & BEST PRACTICES

- Use `TEST_P` with `INSTANTIATE_TEST_SUITE_P` for table-driven tests
- Mock interfaces with gMock: `MOCK_METHOD(int, get, (std::string_view), (override));`
- Run tests under `-fsanitize=address,undefined` in CI

QUICK SUMMARY

- `TEST / TEST_F` — define tests
- `EXPECT_*` — continue; `ASSERT_*` — stop
- fixtures — shared setup via `SetUp()`
- `gtest_discover_tests` — CTest integration

Catch2

CATCH2

Catch2 (v3) is a popular test framework with a lighter feel than GoogleTest. Tests are named with free-form strings, one assertion macro (`REQUIRE` / `CHECK`) handles every comparison by decomposing the expression to print both sides, and **sections** give each path through a test its own fresh setup. It also includes **generators** for data-driven tests and a micro-**benchmark** facility.

SETUP

- vcpkg: `catch2` ; CMake: `find_package(Catch2 3 REQUIRED)`
- Link `Catch2::Catch2WithMain` for a default `main`
- `catch_discover_tests(tests)` registers with CTest
- Run with tags: `./tests "[cart]"`

TEST_CASE, SECTION, GENERATORS

```cpp
#include <catch2/catch_test_macros.hpp>
#include <catch2/generators/catch_generators.hpp>

TEST_CASE("vector grows when pushing", "[vector]") {
    std::vector<int> v;
    REQUIRE(v.empty());
    SECTION("push_back adds one element") {
        v.push_back(1);
        CHECK(v.size() == 1);          // prints both sides on failure
    }
    SECTION("reserve doesn't change size") {
        v.reserve(10);                 // v is empty again here
        CHECK(v.size() == 0);
        CHECK(v.capacity() >= 10);
    }
}
TEST_CASE("squares", "[math]") {
    auto x = GENERATE(1, 2, 3);
    REQUIRE(square(x) == x * x);
}
```

CODE EXPLAINED

- → `TEST_CASE("name", "[tag]")` — any descriptive string
- → `REQUIRE` — stops this test on failure; `CHECK` continues
- → each `SECTION` re-runs the code above it from scratch
- → `GENERATE(1, 2, 3)` — runs the test once per value
- → expression decomposition — failure shows `2 == 1` with real values

MACROS

MACRO	PURPOSE
`REQUIRE(expr)`	assert, stop on failure
`CHECK(expr)`	assert, continue
`REQUIRE_THROWS_AS(e, T)`	exception type
`REQUIRE_NOTHROW(e)`	no exception
`CHECK_THAT(x, matcher)`	matchers
`INFO / CAPTURE`	extra context

CATCH2 VS GOOGLETEST

FEATURE	CATCH2	GOOGLETEST
`assertion style`	REQUIRE(a == b)	EXPECT_EQ(a, b)
`shared setup`	SECTION	fixtures
`data-driven`	GENERATE	TEST_P
`mocking`	external (trompeloeil)	gMock built in
`benchmarks`	BENCHMARK	separate lib

⚠ COMMON MISTAKES

- Expecting state changed in one `SECTION` to be visible in the next; each section starts over
- Including the old single-header `catch.hpp` (v2) with v3 libraries
- Writing `REQUIRE(a == b && c == d)` ; complex expressions can't be decomposed. Split into two checks

✓ TIPS & BEST PRACTICES

- Use tags to group slow or integration tests and exclude them by default
- Use `Catch::Approx` or `WithinRel` matchers for floating comparisons
- `BENCHMARK("name") { return f(); };` for quick micro-benchmarks

QUICK SUMMARY

- `TEST_CASE("…", "[tag]")` — define a test
- `REQUIRE / CHECK` — stop / continue on failure
- `SECTION` — independent paths with shared setup
- `GENERATE` — data-driven tests

{fmt} & spdlog

FORMATTING & LOGGING LIBRARIES

{fmt} is the library `std::format` and `std::print` were based on. It's still useful on older toolchains and has extras like colored output and compile-time format strings. **spdlog** is a fast, header-friendly logging library built on {fmt}: you log with format strings, choose **sinks** (console, rotating files), set levels, and customize the output **pattern**.

SETUP

- vcpkg: `fmt` , `spdlog` ; link `fmt::fmt` , `spdlog::spdlog`
- spdlog can use `std::format` instead (`SPDLOG_USE_STD_FORMAT`)
- Levels: trace, debug, info, warn, error, critical, off
- Default level is `info`

FMT & SPDLOG

```cpp
#include <fmt/format.h>
#include <fmt/color.h>
#include <spdlog/spdlog.h>
#include <spdlog/sinks/rotating_file_sink.h>

fmt::print("{:>8.2f}\n", 3.14159);                    //     3.14
auto s = fmt::format("{} items", 3);                  // "3 items"
fmt::print(fg(fmt::color::red), "error: {}\n", msg);  // colored

spdlog::info("server listening on port {}", 8080);
spdlog::warn("disk {}% full", 91);
spdlog::set_level(spdlog::level::debug);
spdlog::debug("cache hit ratio {:.2f}", 0.93);
spdlog::set_pattern("[%Y-%m-%d %H:%M:%S.%e] [%^%l%$] %v");

auto file = spdlog::rotating_logger_mt("app", "logs/app.log",
                                       5 * 1024 * 1024, 3);  // 5 MB × 3
file->error("payment failed for order {}", orderId);
```

CODE EXPLAINED

- → `fmt::print` — same mini-language as `std::print`
- → `fg(fmt::color::red)` — terminal colors
- → `spdlog::info(...)` — default console logger
- → `set_level` — hide messages below this level
- → `set_pattern` — timestamp, level (`%l`), message (`%v`)
- → `rotating_logger_mt` — thread-safe file sink with rotation

SPDLOG PATTERN FLAGS

FLAG	OUTPUT
`%Y-%m-%d`	date
`%H:%M:%S.%e`	time with ms
`%l`	level name
`%^ … %$`	color range
`%n`	logger name
`%v`	message
`%t`	thread id

SINKS

SINK	DESTINATION
`stdout_color_mt`	colored console
`basic_logger_mt`	single file
`rotating_logger_mt`	size-rotated files
`daily_logger_mt`	one file per day
`null_sink`	discard (tests)

⚠ COMMON MISTAKES

- Building log messages with string concatenation before calling spdlog; pass format arguments so disabled levels cost nothing
- Using `_st` (single-threaded) loggers from multiple threads; use `_mt`
- Forgetting `spdlog::shutdown()` / flushing before exit with async loggers

✓ TIPS & BEST PRACTICES

- Use `SPDLOG_DEBUG(...)` macros with `SPDLOG_ACTIVE_LEVEL` to compile out debug logs in release
- Call `spdlog::flush_on(spdlog::level::err)` so errors hit disk immediately
- Prefer `std::print` in new code on C++23 toolchains; keep {fmt} for colors or older compilers

QUICK SUMMARY

- `fmt::format / fmt::print` — the original {fmt}
- `spdlog::info(...)` — format-string logging
- levels + `set_level` — filter output
- sinks + patterns — where and how logs appear

nlohmann/json

JSON FOR MODERN C++

nlohmann/json is the most popular C++ JSON library. A `nlohmann::json` value behaves like a dynamic container: index it with strings and numbers, assign from C++ values, and iterate it. `json::parse` reads text and `dump()` writes it. Define `to_json` / `from_json`, or use the `NLOHMANN_DEFINE_TYPE_*` macros, to convert your own structs automatically.

SETUP

- vcpkg: `nlohmann-json`; link `nlohmann_json::nlohmann_json`
- Header-only: `#include <nlohmann/json.hpp>`
- `using json = nlohmann::json;` is the common alias
- Parse errors throw `json::parse_error` (or use `json::accept`)

PARSE, ACCESS, SERIALIZE

```cpp
#include <nlohmann/json.hpp>
using json = nlohmann::json;
json cfg = json::parse(R"({"name":"api", "port":8080, "tags":["a","b"]})");
std::string name = cfg["name"];              // "api"
int port = cfg.at("port").get<int>();        // 8080
cfg.value("debug", false);                   // default if missing
cfg["tags"].push_back("c");
cfg.contains("port");                        // true
struct Server { std::string host; int port; };
NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE(Server, host, port)
Server s = json{{"host", "db"}, {"port", 5432}}.get<Server>();
json out = s;                                // {"host":"db","port":5432}
std::println("{}", out.dump(2));             // pretty, 2-space indent
std::ifstream f{"config.json"};
json fromFile = json::parse(f);
```

CODE EXPLAINED

- → `json::parse` — from string or stream
- → `cfg["name"]` — implicit conversion to `std::string`
- → `.at()` + `.get<int>()` — checked access, explicit type
- → `value(key, default)` — safe lookup with fallback
- → `NLOHMANN_DEFINE_TYPE_NON_INTRUSIVE` — generates to/from_json
- → `dump(2)` — serialize with indentation

ACCESS METHODS

CALL	BEHAVIOR IF MISSING
`j["k"]`	creates null (non-const)
`j.at("k")`	throws out_of_range
`j.value("k", def)`	returns def
`j.contains("k")`	false
`j.find("k")`	end()

TYPE CHECKS

CALL	TRUE WHEN
`is_object()`	{ … }
`is_array()`	[…]
`is_string()`	"…"
`is_number_integer()`	42
`is_boolean() / is_null()`	true / null

⚠ COMMON MISTAKES

- Reading with `j["missing"]` on a non-const json; it silently inserts `null`
- Implicit conversions from a value of the wrong type throw `type_error` at run time
- Using `dump()` output as a map key or for comparisons; key order and spacing can vary

✓ TIPS & BEST PRACTICES

- Use the `…_WITH_DEFAULT` variant of the macro to tolerate missing fields
- Use `json::parse(text, nullptr, false)` to get a discarded value instead of an exception
- For very large or performance-critical JSON, look at simdjson or Glaze

QUICK SUMMARY

- `json::parse / dump` — read and write JSON
- `j["k"] / at / value` — access styles
- `get<T>()` — explicit conversion
- `NLOHMANN_DEFINE_TYPE_*` — struct ↔ JSON

Threads

RUNNING CODE IN PARALLEL

`std::thread` (`<thread>`) starts a new OS thread running any callable. You must `join()` it (wait for it) or `detach()` it before the `std::thread` object is destroyed, or the program calls `std::terminate`. C++20's `std::jthread` joins automatically in its destructor and supports **cooperative cancellation** through a `std::stop_token`, so it's the better default.

NOTES

- Arguments are **copied** into the thread; use `std::ref` to pass references
- `std::thread::hardware_concurrency()` — hint of available cores
- Exceptions escaping a thread function call `std::terminate`
- Threads share memory: unsynchronized writes are data races (UB)

THREAD & JTHREAD

```cpp
#include <thread>

void work(int id, std::string& out) { out = std::format("worker {}", id); }

std::string result;
std::thread t{work, 1, std::ref(result)};   // pass a reference with ref()
t.join();                                   // wait; result == "worker 1"

std::jthread j{[](std::stop_token st) {
    while (!st.stop_requested()) {
        poll();
        std::this_thread::sleep_for(100ms);
    }
}};
j.request_stop();                           // ask it to finish
                                            // destructor joins
unsigned cores = std::thread::hardware_concurrency();   // e.g. 8
std::this_thread::get_id();                 // current thread id
```

CODE EXPLAINED

- → `std::thread{f, args...}` — starts immediately
- → `std::ref(result)` — otherwise the thread gets a copy
- → `join()` — blocks until the thread finishes
- → `std::jthread` — passes a `stop_token` if the callable accepts one
- → `request_stop()` — cooperative; the thread must check the token

THREAD API

MEMBER	PURPOSE
`join()`	wait for completion
`detach()`	let it run independently
`joinable()`	can still join/detach?
`get_id()`	thread id
`request_stop()`	jthread: signal stop
`get_stop_token()`	jthread: token

THIS_THREAD

FUNCTION	PURPOSE
`sleep_for(dur)`	pause for a duration
`sleep_until(tp)`	pause until a time point
`yield()`	hint to reschedule
`get_id()`	current thread id

⚠ COMMON MISTAKES

- Letting a joinable `std::thread` go out of scope; the program terminates. Use `jthread` or join
- Passing a reference argument without `std::ref`, so the thread modifies a copy
- Detaching threads that use local variables of the creating function; they dangle

✓ TIPS & BEST PRACTICES

- Default to `std::jthread` in new code
- Prefer higher-level tools (`std::async`, thread pools, parallel algorithms) to raw threads
- Catch exceptions inside the thread function and hand them back via `std::promise` or a queue

QUICK SUMMARY

- `std::thread` — must join or detach
- `std::jthread` — auto-join + stop token (C++20)
- `std::ref` — pass references to threads
- `stop_token` — cooperative cancellation

Mutexes & Locks

PROTECTING SHARED DATA

A **mutex** ensures only one thread at a time runs a **critical section**. Never call `lock()`/`unlock()` by hand; use RAII lock guards so the mutex is released even if an exception is thrown. `std::scoped_lock` (C++17) locks one or more mutexes and avoids **deadlock** when locking several. `std::shared_mutex` allows many readers or one writer.

NOTES

- Keep critical sections short; never call unknown code while locked
- `std::unique_lock` — movable, can unlock early, needed for condition variables
- `std::shared_lock` — shared (read) ownership
- Mutexes aren't copyable or movable

LOCKS IN PRACTICE

```
class Counter {
    mutable std::mutex m_;
    int value_ = 0;
public:
    void increment() { std::scoped_lock lock{m_}; ++value_; }
    int get() const  { std::scoped_lock lock{m_}; return value_; }
};
void transfer(Account& a, Account& b, double amt) {
    std::scoped_lock lock{a.m, b.m};      // both, deadlock-free
    a.balance -= amt; b.balance += amt;
}
std::shared_mutex cacheMutex;
std::string read(const std::string& k) {
    std::shared_lock lock{cacheMutex};    // many readers at once
    return cache.at(k);
}
void write(std::string k, std::string v) {
    std::unique_lock lock{cacheMutex};    // exclusive writer
    cache[std::move(k)] = std::move(v);
}
```

CODE EXPLAINED

- → `std::scoped_lock lock{m_}` — locks now, unlocks at end of scope
- → `mutable std::mutex` — lock inside `const` member functions
- → `scoped_lock{a.m, b.m}` — locks both with deadlock avoidance
- → `shared_lock` — concurrent readers
- → `unique_lock` on `shared_mutex` — exclusive writer

LOCK TYPES

TYPE	USE
`std::lock_guard`	simple, one mutex (C++11)
`std::scoped_lock`	one or more mutexes (C++17)
`std::unique_lock`	deferred / early unlock / condvars
`std::shared_lock`	shared ownership (readers)

MUTEX TYPES

TYPE	FEATURE
`std::mutex`	basic exclusive
`std::recursive_mutex`	same thread may relock
`std::timed_mutex`	try_lock_for(timeout)
`std::shared_mutex`	readers / writer

⚠ COMMON MISTAKES

- Writing `std::scoped_lock(m_);` without a name; that's a temporary unlocked immediately (or declares a variable named m_)
- Locking two mutexes in different orders in different threads, causing deadlock; use `scoped_lock` with both
- Returning a reference to protected data, letting callers access it unlocked

✓ TIPS & BEST PRACTICES

- Bundle each mutex with the data it protects inside one class
- Use `std::atomic` for single counters or flags instead of a mutex
- Run tests with `-fsanitize=thread` to detect data races

QUICK SUMMARY

- `std::mutex` — mutual exclusion
- `std::scoped_lock` — RAII, multi-mutex safe
- `std::shared_mutex` — readers/writer
- short critical sections — lock only what's needed

Condition Variables & Sync Primitives

WAITING FOR EVENTS

A **condition variable** lets threads sleep until another thread signals that some condition may now be true, which is the basis of producer/consumer queues. Always wait **with a predicate** to handle spurious wakeups. C++20 added simpler one-purpose primitives: `std::latch` (count down once), `std::barrier` (reusable phase sync), and `std::counting_semaphore` (limit concurrent access).

NOTES

- `wait` requires a `std::unique_lock`, not `scoped_lock`
- Modify the shared state **while holding** the mutex, then notify
- `notify_one` wakes one waiter, `notify_all` wakes all
- `std::condition_variable_any` works with `stop_token` (C++20)

PRODUCER / CONSUMER QUEUE

```cpp
#include <condition_variable>
#include <queue>
template <class T> class BlockingQueue {
public:
    void push(T v) {
        { std::scoped_lock lock{m_}; q_.push(std::move(v)); }
        cv_.notify_one();                          // wake one consumer
    }
    T pop() {
        std::unique_lock lock{m_};
        cv_.wait(lock, [&] { return !q_.empty(); });   // predicate wait
        T v = std::move(q_.front()); q_.pop();
        return v;
    }
private:
    std::mutex m_;
    std::condition_variable cv_;
    std::queue<T> q_;
};
std::latch ready{3};           // 3 workers count down
ready.count_down();            // in each worker; main calls ready.wait()
```

CODE EXPLAINED

- → push — modify under the lock, notify after releasing it
- → `cv_.wait(lock, pred)` — sleeps until `pred()` is true
- → the predicate loop handles spurious wakeups
- → `unique_lock` — `wait` unlocks while sleeping and relocks on wake
- → `std::latch` — one-shot countdown (C++20)

PRIMITIVES

TYPE	PURPOSE
`condition_variable`	wait for a condition
`std::latch`	one-time countdown
`std::barrier`	reusable phase sync
`std::counting_semaphore<N>`	limit concurrency to N

CONDITION_VARIABLE API

MEMBER	EFFECT
`wait(lock, pred)`	block until pred true
`wait_for(lock, dur, pred)`	with timeout
`notify_one()`	wake one waiter
`notify_all()`	wake all waiters

⚠ COMMON MISTAKES

- Calling `wait(lock)` without a predicate; spurious wakeups and missed notifications cause bugs
- Changing the condition without holding the mutex, then notifying; the waiter can miss it
- Using `notify_one` when several different conditions share one condition variable

✓ TIPS & BEST PRACTICES

- Prefer `std::latch` / `std::barrier` / semaphores when they fit; they're harder to misuse
- Use `std::counting_semaphore` to cap parallel downloads or DB connections
- Add a "closed" flag to queues so consumers can exit cleanly

QUICK SUMMARY

- `condition_variable` — sleep until signaled
- `wait(lock, pred)` — always use a predicate
- `std::latch / std::barrier` — C++20 sync points
- `counting_semaphore` — limit concurrent access

Sheet 094

Atomics & the Memory Model

std::atomic · load / store · fetch_add · compare_exchange · memory_order · data races = UB

LOCK-FREE BASICS

`std::atomic<T>` (`<atomic>`) makes individual reads, writes, and read-modify-write operations indivisible, so threads can share simple variables without a mutex. The **memory model** defines what other threads observe: the default `memory_order_seq_cst` is easiest to reason about, while acquire/release and relaxed orderings trade simplicity for speed. Any non-atomic data accessed by two threads, with at least one writing and no synchronization, is a **data race**: undefined behavior.

NOTES

- Works for integers, pointers, `bool`, and trivially copyable types
- `is_lock_free()` — whether the hardware supports it without a lock
- `std::atomic_flag` — guaranteed lock-free
- Atomics aren't copyable

ATOMIC OPERATIONS

```
#include <atomic>

std::atomic<int> hits{0};
hits++;                                  // atomic increment
hits.fetch_add(5);                       // returns old value
hits.load();                             // read
hits.store(0);                           // write
int old = hits.exchange(42);             // swap, returns previous

int expected = 42;
bool ok = hits.compare_exchange_strong(expected, 43); // CAS
                                         // ok == true, hits == 43
std::atomic<bool> ready{false};
int payload = 0;
// producer thread
payload = 99;
ready.store(true, std::memory_order_release);
// consumer thread
while (!ready.load(std::memory_order_acquire)) {}
assert(payload == 99);                   // guaranteed by acquire/release
```

CODE EXPLAINED

- → `hits++` — atomic read-modify-write
- → `fetch_add` — returns the value before adding
- → `compare_exchange_strong` — set only if the value equals `expected`
- → release store + acquire load — publish `payload` safely
- → default ordering is `seq_cst` when none is given

OPERATIONS

OPERATION	EFFECT
`load() / store(v)`	read / write
`exchange(v)`	write, return old
`fetch_add / fetch_sub`	arithmetic, return old
`fetch_and / or / xor`	bitwise, return old
`compare_exchange_weak/strong`	CAS loop building block
`wait(old) / notify_one()`	C++20 blocking wait

MEMORY ORDERS

ORDER	GUARANTEE
`relaxed`	atomicity only (counters)
`acquire`	later reads stay after
`release`	earlier writes stay before
`acq_rel`	both (RMW ops)
`seq_cst`	single total order (default)

⚠ COMMON MISTAKES

- `x = x + 1` on an atomic; that's a separate load and store, not atomic. Use `++x` or `fetch_add`
- Mixing atomics and plain variables without proper acquire/release ordering
- Using `memory_order_relaxed` for flags that publish data; readers may see the flag but stale data

✓ TIPS & BEST PRACTICES

- Stick with the default `seq_cst` unless profiling proves it's a bottleneck
- Use `std::atomic<std::shared_ptr<T>>` (C++20) for lock-free pointer swapping
- Reach for a mutex as soon as more than one variable must change together

QUICK SUMMARY

- `std::atomic<T>` — indivisible operations
- `fetch_add / exchange / CAS` — RMW building blocks
- acquire/release — publish data between threads
- data race — UB; use atomics or mutexes

Futures & std::async

GETTING RESULTS BACK

A `std::future<T>` is a handle to a value that will be produced later, usually by another thread. `std::async` runs a callable and returns a future for its result; `get()` waits for the value and rethrows any exception the task threw. `std::promise` lets you set a future's value manually, and `std::packaged_task` wraps a callable so its result goes into a future when it runs.

NOTES

- `get()` can be called **once**; use `shared_future` for many readers
- Pass `std::launch::async` to guarantee a new thread
- The future from `std::async` blocks in its destructor until done
- Exceptions travel through the future to `get()`

ASYNC, PROMISE, PACKAGED_TASK

```cpp
#include <future>

auto f1 = std::async(std::launch::async,
                     [] { return slowSum(1, 1'000'000); });
auto f2 = std::async(std::launch::async, fetchUser, 42);
long total = f1.get();                      // waits, then returns
User u = f2.get();                          // rethrows if fetchUser threw

std::promise<std::string> p;
std::future<std::string> fut = p.get_future();
std::jthread worker{[&p] {
    try { p.set_value(download("https://example.com")); }
    catch (...) { p.set_exception(std::current_exception()); }
}};
std::string body = fut.get();

std::packaged_task<int(int, int)> task{[](int a, int b) { return a + b; }};
auto r = task.get_future();
std::jthread t{std::move(task), 2, 3};
r.get();                                    // 5
if (fut.wait_for(0s) == std::future_status::ready) {}  // poll
```

CODE EXPLAINED

- → `std::launch::async` — run on a new thread now
- → `get()` — blocks for the result, rethrows exceptions
- → `promise::set_value` — fulfill the future from any thread
- → `set_exception` — deliver an error instead
- → `packaged_task` — callable + future, run wherever you like
- → `wait_for(0s)` — non-blocking readiness check

PIECES

TYPE	ROLE
`std::future<T>`	read result once
`std::shared_future<T>`	read result many times
`std::promise<T>`	write side, set manually
`std::packaged_task`	wraps a callable
`std::async`	run + return future

LAUNCH POLICIES

POLICY	BEHAVIOR
`launch::async`	new thread
`launch::deferred`	runs lazily on get()
`async | deferred`	implementation chooses (default)

⚠ COMMON MISTAKES

- Calling `get()` twice on a `std::future`; the second call is UB / throws `future_error`
- Ignoring the future returned by `std::async`; its destructor blocks, making the call synchronous
- Relying on the default launch policy; the task may never run if you never call `get()`

✓ TIPS & BEST PRACTICES

- Always pass `std::launch::async` when you want real parallelism
- Use `wait_for` with a timeout to avoid hanging forever on a stuck task
- For many small tasks, use a thread pool (or C++26 `std::execution`) instead of `std::async` per task

QUICK SUMMARY

- `std::async` — run a task, get a future
- `future::get()` — wait + value or exception
- `std::promise` — set a result manually
- `std::packaged_task` — callable bound to a future

Coroutines

SUSPENDABLE FUNCTIONS

A **coroutine** is a function that can **suspend** and **resume** later, keeping its local state. Any function using `co_await`, `co_yield`, or `co_return` is a coroutine. C++20 provides the low-level machinery (a `promise_type` and `std::coroutine_handle`) but almost no ready-made types; C++23 adds `std::generator` for lazy sequences. For async I/O, use a library such as cppcoro, Boost.Asio, or `std::execution` (C++26).

NOTES

- The return type decides behavior via its nested `promise_type`
- Coroutine state lives on the heap (often optimized away)
- `std::generator<T>` is a range: use it in range-for
- Coroutines can't be `constexpr`, `consteval`, or `main`

STD::GENERATOR & CO_AWAIT

```cpp
#include <generator>                          // C++23
std::generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
        co_yield a;                           // suspend, hand out a
        a = std::exchange(b, a + b);
    }
}
for (int x : fibonacci() | std::views::take(8))
    std::print("{} ", x);                     // 0 1 1 2 3 5 8 13
// Async style (with a library task type, e.g. asio::awaitable):
asio::awaitable<void> session(tcp::socket s) {
    char buf[1024];
    auto n = co_await s.async_read_some(asio::buffer(buf),
                                        asio::use_awaitable);
    co_await asio::async_write(s, asio::buffer(buf, n),
                               asio::use_awaitable);
}
```

CODE EXPLAINED

- → `std::generator<int>` — lazy sequence produced with `co_yield`
- → `co_yield a` — suspend and give the caller a value
- → infinite generator — safe because `take(8)` stops pulling
- → `co_await` — suspend until the awaited operation completes
- → Asio's `awaitable` — a library-provided coroutine task type

KEYWORDS

KEYWORD	EFFECT
`co_yield v`	produce a value, suspend
`co_await expr`	suspend until expr ready
`co_return v`	finish, deliver final value
`promise_type`	customizes coroutine behavior
`std::coroutine_handle`	resume / destroy a frame

LIBRARIES

LIBRARY	PROVIDES
`std::generator (C++23)`	lazy sequences
`Boost.Asio`	awaitable networking
`cppcoro`	task, generator, async primitives
`libcoro / folly::coro`	production task types
`std::execution (C++26)`	senders / receivers

⚠ COMMON MISTAKES

- Capturing references in a coroutine that outlive the caller's data; the coroutine resumes after the referent is gone
- Passing `std::string_view` or references as coroutine parameters for long-lived coroutines; copy instead
- Writing your own `promise_type` for production use instead of using a tested library

✓ TIPS & BEST PRACTICES

- Use `std::generator` for tree walks, file readers, and infinite sequences
- Take coroutine parameters by value so the frame owns its data
- Check your standard library for `std::generator` support (GCC 14+ ships it)

QUICK SUMMARY

- coroutine — any function with `co_*` keywords
- `co_yield / co_await / co_return` — suspend points
- `std::generator<T>` — lazy range (C++23)
- libraries — provide task/awaitable types

std::variant & std::any

TYPE-SAFE UNIONS

`std::variant<Ts...>` (C++17) holds **exactly one** value from a fixed list of types and always knows which. Access it with `std::visit` (handles every alternative, checked at compile time), `std::get_if` (pointer or null), or `std::get` (throws `bad_variant_access` on the wrong type). `std::any` holds a value of **any** copyable type, discovered at run time with `any_cast`.

NOTES

- `v.index()` — which alternative is active (0-based)
- Default-constructs the first alternative; use `std::monostate` for "empty"
- `std::visit` with the **overloaded** lambda pattern is idiomatic
- Prefer `variant` (closed set) over `any` (open set)

VARIANT, VISIT, ANY

```
#include <variant>
#include <any>
using Value = std::variant<int, double, std::string>;
Value v = 3.5;
v.index();                                  // 1 (double)
std::holds_alternative<double>(v);          // true
std::get<double>(v);                        // 3.5
if (auto* s = std::get_if<std::string>(&v)) {}   // nullptr here
v = "text";                                 // now holds std::string
template <class... Fs> struct overloaded : Fs... { using Fs::operator()...; };
std::visit(overloaded{
    [](int i)                { std::println("int {}", i); },
    [](double d)             { std::println("double {}", d); },
    [](const std::string& s) { std::println("string {}", s); },
}, v);                                      // string text
std::any a = std::string{"hi"};             // any copyable type
std::any_cast<std::string>(a);              // "hi"
```

CODE EXPLAINED

- → `index()` / `holds_alternative` — query the active type
- → `std::get<T>` — throws if T isn't active
- → `std::get_if<T>(&v)` — returns pointer or `nullptr`
- → `overloaded` — combines lambdas into one visitor (no deduction guide needed since C++20)
- → `any_cast<T>(&a)` — pointer form doesn't throw

VARIANT API

FUNCTION	PURPOSE
`std::visit(f, v)`	call f with active value
`std::get<T / I>(v)`	access, may throw
`std::get_if<T>(&v)`	pointer or null
`std::holds_alternative<T>`	is T active?
`v.index()`	active index
`v.emplace<T>(args)`	construct in place

VARIANT VS ANY VS VIRTUAL

TOOL	TYPES	DISPATCH
`std::variant`	fixed list	visit, compile-checked
`std::any`	anything	any_cast at run time
`virtual base`	open hierarchy	vtable
`std::optional`	T or nothing	—

⚠ COMMON MISTAKES

- `std::get<int>(v)` without checking; throws `bad_variant_access` when another type is active
- Forgetting an alternative in `visit`; it won't compile, which is a feature, so don't paper over it with an `auto` catch-all too early
- `variant<std::string, bool> v = "x";` — before C++20 this picked `bool`; be explicit with `std::string{"x"}`

✓ TIPS & BEST PRACTICES

- Model state machines and AST nodes as `std::variant` instead of class hierarchies
- Keep `overloaded` in a shared utility header
- Avoid `std::any` in public APIs; it hides the type contract

QUICK SUMMARY

- `std::variant<Ts...>` — one of a fixed set of types
- `std::visit` + `overloaded` — exhaustive handling
- `get_if` — safe checked access
- `std::any` — any type, checked at run time

Regular Expressions

STD::REGEX

<regex> (C++11) provides regular expressions using ECMAScript syntax by default. `std::regex_match` tests whether the **whole** string matches, `std::regex_search` finds the **first** match anywhere, `std::regex_replace` substitutes matches, and `std::sregex_iterator` walks all matches. Results come back in a `std::smatch` with capture groups. The standard implementation is known to be **slow**; see the Regex set for pattern syntax.

NOTES

- Use raw strings `R"(\d+)"` to avoid double escaping
- Constructing a `std::regex` is expensive: build once, reuse
- `std::smatch` for `std::string`, `std::cmatch` for `const char*`
- Flags: `std::regex::icase`, `std::regex::multiline` (C++17)

MATCH, SEARCH, REPLACE, ITERATE

```cpp
#include <regex>

const std::regex date{R"((\d{4})-(\d{2})-(\d{2}))"};   // compile once
std::string s = "Released 2026-10-11, patched 2026-11-02.";

std::regex_match("2026-10-11", date);       // true (whole string)
std::regex_match(s, date);                  // false

std::smatch m;
if (std::regex_search(s, m, date)) {
    m[0];               // "2026-10-11"  whole match
    m[1];               // "2026"        group 1
    m.position(0);      // 9
}
std::regex_replace(s, date, "$3/$2/$1");    // "...11/10/2026, ...02/11/2026."

for (std::sregex_iterator it{s.begin(), s.end(), date}, end; it != end; ++it)
    std::println("{}", it->str());          // each date

std::regex word{R"(\bcolou?r\b)", std::regex::icase};
```

CODE EXPLAINED

- → `R"(…)"` — raw string; backslashes stay literal
- → `regex_match` — must match the entire input
- → `regex_search` — first match anywhere; groups in `m[1]`, `m[2]` …
- → `$1`, `$2` — group references in the replacement
- → `sregex_iterator` — loop over every match
- → `icase` — case-insensitive matching

FUNCTIONS

FUNCTION	PURPOSE
`regex_match(s, re)`	entire string matches?
`regex_search(s, m, re)`	find first match
`regex_replace(s, re, fmt)`	substitute matches
`sregex_iterator`	iterate matches
`sregex_token_iterator`	split / extract groups

SMATCH MEMBERS

MEMBER	RETURNS
`m[0] / m.str()`	whole match
`m[n]`	capture group n
`m.position(n)`	offset of group n
`m.prefix() / suffix()`	text before / after
`m.size()`	groups + 1

⚠ COMMON MISTAKES

- Constructing `std::regex` inside a loop; compilation is costly. Make it `static const`
- Using `regex_match` when you meant "contains"; use `regex_search`
- Running `std::regex` on large inputs in hot paths; it can be orders of magnitude slower than alternatives

✓ TIPS & BEST PRACTICES

- For performance, use CTRE (compile-time regex), RE2, or PCRE2
- For simple checks, `starts_with`, `contains`, and `find` are faster and clearer
- Use named sub-expressions in comments since `std::regex` has no named groups

QUICK SUMMARY

- `regex_match` — whole string
- `regex_search` — first match + groups
- `regex_replace` — substitution with `$n`
- build once — regex construction is expensive

UB, Sanitizers & Debugging

UNDEFINED BEHAVIOR & TOOLS

Undefined behavior (UB) means the standard places no requirements on what happens: the program may crash, appear to work, or be "optimized" into something surprising. Common UB includes out-of-bounds access, signed overflow, use-after-free, data races, and dereferencing null. **Sanitizers** instrument your build to catch UB at run time, and **debuggers** let you inspect a running or crashed program.

NOTES

- UB can appear to work until a compiler upgrade changes optimizations
- Sanitizers slow programs 2–5×; use them in tests and CI
- ASan and TSan can't be combined in one build
- `std::stacktrace` (C++23) prints a call stack from code

CATCHING UB

```
# AddressSanitizer + UndefinedBehaviorSanitizer
g++ -std=c++23 -g -O1 -fsanitize=address,undefined \
    -fno-omit-frame-pointer main.cpp -o app
./app        # reports: heap-use-after-free / signed-integer-overflow ...

# ThreadSanitizer (separate build)
clang++ -std=c++23 -g -fsanitize=thread main.cpp -o app_tsan

# Debug with gdb (or lldb on macOS)
gdb ./app
(gdb) break inventory.cpp:42
(gdb) run --config test.json
(gdb) bt             # backtrace after a crash
(gdb) print items.size()
(gdb) next / step / continue

valgrind --leak-check=full ./app      # leaks & invalid reads (Linux)
```

CODE EXPLAINED

- → `-fsanitize=address` — out-of-bounds, use-after-free, leaks
- → `undefined` — signed overflow, bad shifts, null dereference
- → `-fno-omit-frame-pointer` — better stack traces
- → `thread` — data races (needs its own build)
- → `bt` — show the call stack at the crash
- → `valgrind` — no recompilation needed, but slower

COMMON UB

CODE	PROBLEM
`v[v.size()]`	out of bounds
`INT_MAX + 1`	signed overflow
`delete p; *p`	use after free
`int x; f(x);`	uninitialized read
`1 << 32 (32-bit int)`	shift too far
`two threads, one writes`	data race

TOOLS

TOOL	FINDS
`ASan`	memory errors, leaks
`UBSan`	arithmetic / type UB
`TSan`	data races
`MSan (Clang)`	uninitialized reads
`valgrind`	memory errors (slow)
`clang-tidy`	static bug patterns

⚠ COMMON MISTAKES

- Concluding code is correct because it "works on my machine"; UB often hides until optimization changes
- Debugging release builds without `-g`; you get no line numbers
- Ignoring sanitizer reports in third-party code paths your program triggers

✓ TIPS & BEST PRACTICES

- Add a sanitizer job to CI that runs the whole test suite with ASan + UBSan
- Use `_GLIBCXX_ASSERTIONS` or libc++ hardening modes to bounds-check `operator[]` in debug builds
- Print `std::stacktrace::current()` in fatal error handlers (C++23, link `-lstdc++exp` on GCC)

QUICK SUMMARY

- UB — no guarantees; must be avoided, not tolerated
- `-fsanitize=address,undefined` — catch UB at run time
- `-fsanitize=thread` — catch data races
- gdb / lldb — breakpoints, `bt`, `print`

C++26 Preview

WHAT'S COMING IN C++26

C++26 is the next standard. Its headline features are **static reflection** (inspecting types and generating code at compile time), **contracts** (`pre`, `post`, `contract_assert`), and **`std::execution`** (a senders/receivers framework for async and parallel work). Smaller additions include pack indexing, `#embed`, `std::inplace_vector`, saturation arithmetic, and `std::hive`. Compiler support is still arriving, so check cppreference's support table before relying on any of these.

STATUS

- Feature set frozen in 2025; publication expected around late 2026
- GCC and Clang implement features behind `-std=c++26` / `c++2c`
- Reflection syntax uses `^^T` (reflect) and `[: r :]` (splice)
- Treat examples here as illustrative; details may shift in implementations

C++26 FEATURES (ILLUSTRATIVE)

```cpp
// Static reflection: enum → string
template <typename E> requires std::is_enum_v<E>
constexpr std::string_view to_string(E value) {
    template for (constexpr auto e :
                  std::define_static_array(std::meta::enumerators_of(^^E)))
        if (value == [:e:]) return std::meta::identifier_of(e);
    return "<unknown>";
}
// Contracts
int safe_div(int a, int b) pre (b != 0) post (r: r * b <= a);
// Pack indexing
template <typename... Ts> using First = Ts...[0];
// #embed binary data
constexpr unsigned char icon[] = {
    #embed "icon.png"
};
std::inplace_vector<int, 16> small;        // fixed capacity, no heap
auto s = std::add_sat<std::uint8_t>(200, 100);   // 255 (saturating)
```

CODE EXPLAINED

- → `^^E` — reflect the type into a `std::meta::info` value
- → `[:e:]` — splice a reflection back into code
- → `template for` — expansion statement over compile-time ranges
- → `pre` / `post` — checked function contracts
- → `Ts...[0]` — index directly into a parameter pack
- → `#embed` — include a file's bytes at compile time

HEADLINE FEATURES

FEATURE	WHAT IT GIVES YOU
`static reflection`	serialization, enum names, codegen
`contracts`	checked pre/postconditions
`std::execution`	structured async / parallelism
`pack indexing`	`Ts...[N]`
`#embed`	binary resources in source

LIBRARY ADDITIONS

ADDITION	PURPOSE
`std::inplace_vector`	fixed-capacity vector
`std::hive`	stable-pointer bucket container
`add_sat / sub_sat`	saturating arithmetic
`std::function_ref`	non-owning callable reference
`std::optional<T&>`	optional references

⚠ COMMON MISTAKES

- Writing C++26 code for a project that must build on older compilers; gate it behind feature-test macros
- Assuming reflection replaces all macros today; tooling and debuggers are still catching up
- Copying syntax from early proposals; several features changed spelling during standardization

✓ TIPS & BEST PRACTICES

- Test features on Compiler Explorer with GCC trunk / Clang trunk before adopting
- Use feature-test macros like `__cpp_contracts` and `__cpp_impl_reflection` (check names on cppreference)
- Start with library features (`inplace_vector`, `add_sat`); they're easy to polyfill

QUICK SUMMARY

- reflection — `^^T` and `[: :]` (C++26)
- contracts — `pre` , `post` , `contract_assert`
- `std::execution` — senders / receivers
- `#embed` , pack indexing, `inplace_vector` — smaller wins

Pointer Basics

* and & · dereferencing · nullptr · pointer to pointer · const T* vs T* const · void*

ADDRESSES & INDIRECTION

A **pointer** stores the memory address of an object. &x takes an address, and *p **dereferences** a pointer to reach the object it points to. A pointer can be reassigned to point elsewhere and can be **null** (`nullptr`), meaning it points to nothing. Dereferencing a null, dangling, or uninitialized pointer is undefined behavior. In modern C++, raw pointers are for **non-owning** access; ownership belongs to smart pointers.

READING DECLARATIONS

- `int* p` — pointer to int
- `const int* p` — pointer to const int (can't modify `*p`)
- `int* const p` — const pointer (can't reseat `p`)
- `int** pp` — pointer to pointer to int

POINTER BASICS

```
int x = 10;
int* p = &x;             // p holds x's address
*p = 20;                 // x == 20
std::println("{}", static_cast<void*>(p));   // e.g. 0x7ffd5c3a

int* q = nullptr;        // points to nothing
if (q) *q = 1;           // guard before dereferencing

int** pp = &p;           // pointer to pointer
**pp = 30;               // x == 30

const int* ro = &x;      // *ro = 1;  ERROR   ro = &y;  OK
int* const fixed = &x;   // *fixed = 1; OK    fixed = &y; ERROR

void* any = &x;                       // untyped address
int* back = static_cast<int*>(any);   // must cast back to use
```

CODE EXPLAINED

- → `&x` — address-of operator
- → `*p` — dereference: the object p points to
- → `nullptr` — typed null pointer constant (not `NULL` or `0`)
- → `**pp` — two levels of indirection
- → `void*` — generic address; cast back before use

OPERATORS

SYNTAX	MEANING
`int* p;`	declare pointer
`&x`	address of x
`*p`	object at p
`p->m`	member m of *p
`p == nullptr`	null check
`static_cast<void*>(p)`	print an address

CONST PLACEMENT

DECLARATION	CAN CHANGE *P	CAN CHANGE P
`int* p`	yes	yes
`const int* p`	no	yes
`int* const p`	yes	no
`const int* const p`	no	no

⚠ COMMON MISTAKES

- Declaring `int* a, b;` — only `a` is a pointer; `b` is an `int`. Declare one per line
- Using an uninitialized pointer: `int* p; *p = 5;` is UB. Initialize to `nullptr` or a valid address
- Keeping a pointer to a local after its function returns; it dangles

✓ TIPS & BEST PRACTICES

- Use references when null isn't a valid value; use pointers when "no object" is meaningful
- Use `std::unique_ptr` / `std::shared_ptr` for ownership; raw `T*` means "I'm just looking"
- Enable `-Wnull-dereference` and run ASan to catch pointer bugs early

QUICK SUMMARY

- `T* p = &x;` — pointer to x
- `*p` — dereference
- `nullptr` — null pointer
- `const T*` vs `T* const` — pointee vs pointer constness

Pointer Arithmetic & Arrays

MOVING THROUGH MEMORY

Pointer arithmetic moves a pointer by whole elements, not bytes: `p + 1` points to the next T. It's only valid within one array (plus the **one-past-the-end** position, which you may compute but not dereference). Arrays **decay** to a pointer to their first element, which is why `a[i]` is defined as `*(a + i)`. Subtracting two pointers into the same array gives a `std::ptrdiff_t` element count.

NOTES

- `p + n` advances `n * sizeof(T)` bytes
- Comparing pointers into different arrays with `<` is unspecified
- Iterators generalize pointers; raw pointers *are* random-access iterators
- `std::to_address(it)` gets a raw pointer from an iterator-like object

ARITHMETIC & ARRAYS

```
int a[5] = {10, 20, 30, 40, 50};
int* p = a;               // decay: &a[0]
*(p + 2);                 // 30, same as a[2] and p[2]
p += 3;                   // now points to a[3]
*p;                       // 40
p - a;                    // 3   (std::ptrdiff_t)

int* end = a + 5;         // one past the end: OK to form
for (int* it = a; it != end; ++it) std::print("{} ", *it);   // 10 … 50
// *end;                  // UB: one-past-end can't be dereferenced
// a + 6;                 // UB: beyond one-past-end

std::vector<double> v{1.5, 2.5, 3.5};
double* d = v.data();
d[1];                     // 2.5
std::sort(a, a + 5);      // pointers work as iterators
```

CODE EXPLAINED

- → `a` decays to `&a[0]` in most expressions
- → `p + 2` — two elements forward, not two bytes
- → `p - a` — distance in elements
- → `end = a + 5` — valid sentinel, never dereferenced
- → `v.data()` — raw pointer into a vector's contiguous storage

OPERATIONS

EXPRESSION	MEANING
`p + n / p - n`	move n elements
`++p / --p`	next / previous element
`p[n]`	*(p + n)
`p2 - p1`	element distance (ptrdiff_t)
`p1 == p2 / p1 < p2`	compare (same array)

SAFER ALTERNATIVES

INSTEAD OF	USE
`T* + size_t`	std::span<T>
`pointer loop`	range-based for
`manual end pointer`	std::end(a)
`p[i] unchecked`	span / vector at()

⚠ COMMON MISTAKES

- Going past one-past-the-end, even without dereferencing; just computing it is UB
- Doing byte-level arithmetic on a `T*`; cast to `std::byte*` / `char*` first if you mean bytes
- Assuming `sizeof(arr)` inside a function parameter is the array size; it's the pointer size

✓ TIPS & BEST PRACTICES

- Wrap pointer + length pairs in `std::span` as early as possible
- Use `std::begin(a)` / `std::end(a)` for C arrays in generic code
- Compile debug builds with `-D_GLIBCXX_ASSERTIONS` or ASan to catch out-of-bounds bugs

QUICK SUMMARY

- `p + n` — n elements forward
- `a[i]` — same as `*(a + i)`
- one-past-the-end — may form, may not dereference
- `std::span` — safer pointer + size

References

ALIASES FOR OBJECTS

A **reference** is another name for an existing object. It must be initialized when declared, can't be reseated to refer to something else, and can't be null. Reading or writing through a reference affects the original. `const T&` binds to anything, including temporaries (and extends their lifetime). References make function parameters and range-for loops efficient without pointer syntax.

NOTES

- No "null reference" and no reference reassignment
- `T&` binds to lvalues only; `const T&` also binds to temporaries
- A reference member makes a class non-assignable
- `std::reference_wrapper<T>` is a copyable, reseatable reference

REFERENCES

```
int score = 70;
int& ref = score;           // alias
ref += 5;                   // score == 75
int other = 10;
ref = other;                // assigns 10 to score — does NOT reseat

const std::string& name = std::string{"temp"};  // lifetime extended
// int& bad = 5;            // ERROR: non-const ref to temporary

void bump(int& x) { ++x; }
bump(score);                // score == 11

std::string& first(std::vector<std::string>& v) { return v.front(); }
first(names) = "Zed";       // modifies the vector element

std::vector<std::reference_wrapper<int>> refs{score, other};
refs[0].get() = 1;          // score == 1
int& dangling() { int local = 5; return local; }   // UB on use
```

CODE EXPLAINED

- → `int& ref = score` — `ref` and `score` are the same object
- → `ref = other` — copies the value; references never reseat
- → `const&` to a temporary — temporary lives as long as the reference
- → returning `T&` — lets callers modify the original
- → `reference_wrapper` — store references in containers

REFERENCE VS POINTER

ASPECT	T&	T*
can be null	no	yes
reseatable	no	yes
must initialize	yes	no
syntax	x.m	p->m, *p
arithmetic	no	yes
in containers	via reference_wrapper	yes

BINDING RULES

INITIALIZER	T&	CONST T&
lvalue x	✔	✔
temporary T{}	✖	✔
literal 5	✖	✔
different type (convertible)	✖	✔ (temporary made)

⚠ COMMON MISTAKES

- Returning a reference to a local variable or temporary; it dangles immediately
- Expecting `ref = other;` to make `ref` refer to `other`; it assigns through it
- Storing references to vector elements and then growing the vector; they dangle after reallocation

✓ TIPS & BEST PRACTICES

- Use `const T&` for read-only parameters of non-trivial types
- Prefer pointers for optional or reseatable relationships, references for required ones
- Use `std::ref(x)` to pass references through `std::thread`, `std::bind`, or `std::invoke`

QUICK SUMMARY

- `T& r = x;` — alias for x
- no null, no reseating
- `const T&` — binds to temporaries
- `std::reference_wrapper` — reference in containers

Value Categories & Forwarding

LVALUES, RVALUES & FORWARDING

Every expression has a **value category**. An **lvalue** has identity and can be addressed (a named variable). A **prvalue** is a pure temporary value (42, `f()` returning by value). An **xvalue** is an expiring object, such as the result of `std::move(x)`. In templates, T&& with a deduced T is a **forwarding reference** that binds to anything, and `std::forward<T>` passes the argument on with its original category preserved.

RULES OF THUMB

- Has a name → lvalue (even if its type is `T&&`!)
- `std::move(x)` → xvalue (rvalue)
- `T&&` in a template parameter with deduction → forwarding ref
- `auto&&` is a forwarding reference too

CATEGORIES & PERFECT FORWARDING

```cpp
int x = 5;
int& lr = x;                 // lvalue ref binds to lvalue
int&& rr = 5;                // rvalue ref binds to prvalue
// int&& bad = x;            // ERROR: x is an lvalue
int&& moved = std::move(x);  // OK: std::move makes an xvalue

void take(std::string& s)  { std::println("lvalue"); }
void take(std::string&& s) { std::println("rvalue"); }

template <class T>
void relay(T&& arg) {                  // forwarding reference
    take(std::forward<T>(arg));        // preserves category
}
std::string s = "hi";
relay(s);                    // lvalue   (T = std::string&)
relay(std::string{"tmp"});   // rvalue   (T = std::string)
relay(std::move(s));         // rvalue
```

CODE EXPLAINED

- → `int&&` — binds only to rvalues
- → `std::move` — a cast to rvalue reference; moves nothing itself
- → forwarding reference — `T` deduced as `X&` for lvalues, `X` for rvalues
- → `std::forward<T>` — casts back to rvalue only if the argument was one
- → inside `relay`, `arg` is a named lvalue, hence the need to forward

CATEGORIES

CATEGORY	EXAMPLE	BINDS TO
lvalue	x, *p, v[0]	T&, const T&
prvalue	42, f(), T{}	T&&, const T&
xvalue	std::move(x)	T&&, const T&

REFERENCE COLLAPSING

COMBINATION	RESULT
T& &	T&
T& &&	T&
T&& &	T&
T&& &&	T&&

⚠ COMMON MISTAKES

- Using `std::move` on a forwarding reference; it turns lvalues into rvalues and steals from the caller. Use `std::forward`
- Assuming a parameter declared `T&& x` is an rvalue inside the function; it's a named lvalue
- Calling `std::forward` twice on the same argument; the second use may see a moved-from object

✓ TIPS & BEST PRACTICES

- Remember: `std::move` for rvalue references you own, `std::forward` for forwarding references
- `auto&&` in range-for works for every element type, including proxies
- Use `decltype(auto)` to return a forwarded expression without decaying it

QUICK SUMMARY

- lvalue / prvalue / xvalue — value categories
- `T&&` (deduced) — forwarding reference
- `std::forward<T>` — preserve category
- collapsing — any `&` wins

Sheet 105

Stack, Heap & Storage Duration

automatic · static · thread_local · dynamic · object lifetime · stack overflow · alignas

WHERE OBJECTS LIVE

Every object has a **storage duration** that decides its lifetime. **Automatic** objects (locals) live on the **stack** and die at the end of their scope. **Static** objects (globals, `static` locals) live for the whole program. **Thread** storage (`thread_local`) gives one copy per thread. **Dynamic** objects are created on the **heap** with new (or by containers and smart pointers) and live until they are explicitly destroyed.

STACK VS HEAP

- Stack: fast, automatic, limited size (often 1–8 MB)
- Heap: large, flexible lifetime, slower to allocate
- Deep recursion or big local arrays → **stack overflow**
- `alignas(N)` forces stricter alignment

STORAGE DURATIONS

```
int g_count = 0;                          // static: whole program
thread_local int t_requests = 0;          // one per thread

void handle() {
    int local = 42;                       // automatic: stack
    static int calls = 0;                 // static: initialized once
    ++calls;
    auto buf = std::make_unique<char[]>(1 << 20); // dynamic: 1 MB heap
    std::vector<int> v(1000);             // object on stack, data on heap
}                                         // local, buf, v destroyed here

void bad() { char big[50'000'000]; }      // likely stack overflow

struct alignas(64) CacheLine { int data[16]; };   // 64-byte aligned
static_assert(alignof(CacheLine) == 64);
```

CODE EXPLAINED

- → `g_count` — exists before `main` and after it returns
- → `static int calls` — keeps its value between calls
- → `make_unique<char[]>` — heap buffer owned by a stack object
- → `std::vector` — small handle on the stack, elements on the heap
- → `alignas(64)` — avoid false sharing between threads

STORAGE DURATIONS

DURATION	CREATED	DESTROYED
automatic	at declaration	end of scope
static	before main / first use	after main
thread	thread start / first use	thread exit
dynamic	new / allocator	delete / owner

STACK VS HEAP

ASPECT	STACK	HEAP
allocation	pointer bump	allocator call
size limit	small	large
lifetime	scope	explicit / owned
fragmentation	none	possible

⚠ COMMON MISTAKES

- Returning a pointer or reference to a local; the stack frame is gone
- Huge local arrays (`double m[1000][1000]`) that overflow the stack; use `std::vector`
- Relying on initialization order of globals in different files (static init order fiasco)

✓ TIPS & BEST PRACTICES

- Default to automatic (stack) objects; let containers handle heap memory
- Replace mutable globals with function-local statics or explicit dependencies
- Use `constinit` to guarantee compile-time initialization of globals

QUICK SUMMARY

- automatic — stack, scope-bound
- static — whole program
- `thread_local` — one per thread
- dynamic — heap, owned by smart pointers/containers

new & delete

MANUAL ALLOCATION

new allocates memory on the heap **and** constructs an object; `delete` destroys it **and** frees the memory. Arrays use `new[]` / `delete[]`, and mixing the forms is UB. Every `new` must be matched by exactly one `delete` on every path, including when exceptions are thrown, which is why modern code almost never writes them directly. **Placement new** constructs an object in memory you already own.

FAILURE MODES

- **Leak**: forgetting `delete`
- **Double free**: deleting twice
- **Use after free**: touching memory after `delete`
- `new` throws `std::bad_alloc`; `new (std::nothrow)` returns `nullptr`

NEW / DELETE & FRIENDS

```
int* n = new int{42};            // allocate + construct
delete n;                        // destroy + free
n = nullptr;                     // avoid dangling reuse

int* arr = new int[5]{1, 2, 3};  // 1 2 3 0 0
delete[] arr;                    // must use delete[]

Widget* w = new Widget{"cfg"};
process(*w);                     // if this throws → leak!
delete w;

int* maybe = new (std::nothrow) int[1'000'000];
if (!maybe) { /* handle failure */ }

alignas(Widget) std::byte storage[sizeof(Widget)];
Widget* pw = new (storage) Widget{"inplace"};   // placement new
pw->~Widget();                   // destroy manually, no delete
auto good = std::make_unique<Widget>("cfg");    // what to write instead
```

CODE EXPLAINED

- → `new int{42}` — heap allocation with initialization
- → `delete[]` — required for `new[]`, runs every element's destructor
- → exception between `new` and `delete` — memory leaks
- → `std::nothrow` — returns null instead of throwing
- → placement new — construct in existing storage; call the destructor yourself

PAIRS THAT MUST MATCH

ALLOCATE	RELEASE
`new T`	delete p
`new T[n]`	delete[] p
`new (buf) T`	p->~T()
`std::malloc`	std::free
`std::make_unique`	automatic

BUGS & DETECTORS

BUG	DETECTED BY
`leak`	ASan (LeakSanitizer), valgrind
`double free`	ASan
`use after free`	ASan
`new[] / delete mismatch`	ASan

⚠ COMMON MISTAKES

- `delete` on memory from `new[]` (or vice versa); that's UB
- Mixing `malloc` / `free` with `new` / `delete`; constructors and destructors don't run
- Deleting through a base pointer without a virtual destructor

✓ TIPS & BEST PRACTICES

- Replace `new` with `std::make_unique` / `std::make_shared` and containers
- If you must hold a raw owning pointer temporarily, wrap it in a smart pointer immediately
- Run tests with `-fsanitize=address` to find leaks and double frees

QUICK SUMMARY

- `new` / `delete` — allocate+construct / destroy+free
- `new[]` / `delete[]` — array forms, never mix
- placement new — construct in existing memory
- modern code — use smart pointers instead

std::unique_ptr

EXCLUSIVE OWNERSHIP

`std::unique_ptr<T>` (`<memory>`) owns a heap object and deletes it automatically when the pointer is destroyed. Ownership is **exclusive**: a `unique_ptr` can be moved but not copied, so there's always exactly one owner. It's the same size as a raw pointer and costs nothing extra, making it the default choice for dynamic allocation. Custom deleters let it manage C resources like `FILE*`.

NOTES

- Create with `std::make_unique<T>(args...)`
- Transfer ownership with `std::move`
- `get()` — raw pointer for non-owning use
- `unique_ptr<T[]>` — arrays (prefer `std::vector`)

UNIQUE_PTR IN USE

```
#include <memory>

auto w = std::make_unique<Widget>("main", 3);   // owns a Widget
w->draw();                                      // like a raw pointer
if (w) {}                                       // non-null?

auto w2 = std::move(w);                         // transfer; w is null
// auto w3 = w2;                                // ERROR: no copy

void render(const Widget& wd);                  // non-owning parameter
render(*w2);
void adopt(std::unique_ptr<Widget> owned);      // takes ownership
adopt(std::move(w2));

std::unique_ptr<Shape> s = std::make_unique<Circle>(2.0);   // polymorphic
s.reset();                                      // delete now; s == null
Widget* raw = s.release();                      // give up ownership (rare)

auto file = std::unique_ptr<std::FILE, decltype(&std::fclose)>(
    std::fopen("log.txt", "w"), &std::fclose);  // custom deleter
```

CODE EXPLAINED

- → `make_unique` — allocates and wraps in one exception-safe step
- → `std::move(w)` — ownership moves; `w` becomes null
- → pass `const Widget&` when the callee only uses the object
- → pass `unique_ptr` by value to transfer ownership
- → `reset()` deletes now; `release()` hands back a raw owning pointer
- → custom deleter — `fclose` runs instead of `delete`

MEMBERS

MEMBER	PURPOSE
`get()`	raw pointer (non-owning)
`release()`	give up ownership
`reset(p = nullptr)`	delete current, take p
`operator* / ->`	access the object
`explicit operator bool`	non-null check
`swap(other)`	exchange owned objects

PASSING UNIQUE_PTR

SIGNATURE	MEANING
`f(T&) / f(const T&)`	just use the object
`f(T*)`	use, may be null
`f(unique_ptr<T>)`	take ownership
`f(unique_ptr<T>&)`	may reseat caller's pointer

⚠ COMMON MISTAKES

- Creating two `unique_ptr`s from the same raw pointer; both delete it
- Calling `release()` and forgetting to delete the returned pointer
- Passing `const unique_ptr<T>&` everywhere instead of `T&`; it couples functions to ownership

✓ TIPS & BEST PRACTICES

- Always prefer `std::make_unique` over `unique_ptr<T>(new T)`
- Return `std::unique_ptr` from factory functions to make ownership transfer explicit
- Store polymorphic objects in `std::vector<std::unique_ptr<Base>>`

QUICK SUMMARY

- `std::unique_ptr<T>` — single owner, auto-delete
- `make_unique` — create safely
- move-only — transfer with `std::move`
- custom deleter — manage C handles

std::shared_ptr & std::weak_ptr

SHARED OWNERSHIP

`std::shared_ptr<T>` allows **multiple owners**. A **control block** holds a reference count; the object is deleted when the last `shared_ptr` to it is destroyed. Copying increments the count atomically, so `shared_ptr` costs more than `unique_ptr`. `std::weak_ptr` observes a shared object without owning it, which breaks **reference cycles** and lets you check whether the object still exists.

NOTES

- Create with `std::make_shared<T>(args...)` (one allocation)
- `use_count()` — current number of owners (for debugging)
- `weak_ptr::lock()` — get a `shared_ptr` or null if expired
- The count is thread-safe; the pointed-to object isn't automatically

SHARED_PTR & WEAK_PTR

```
auto cfg = std::make_shared<Config>("prod");   // count 1
auto alias = cfg;                              // count 2
cfg.use_count();                               // 2
alias.reset();                                 // count 1

std::weak_ptr<Config> watcher = cfg;           // doesn't own
if (auto locked = watcher.lock()) {            // shared_ptr or null
    locked->reload();
}
cfg.reset();                                   // count 0 → deleted
watcher.expired();                             // true

struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node>   prev;                // weak: no cycle
};
struct Session : std::enable_shared_from_this<Session> {
    void start() { schedule(shared_from_this()); }   // keep alive
};
```

CODE EXPLAINED

- → copying a `shared_ptr` — increments the count
- → `reset()` — drops this owner
- → `weak_ptr::lock()` — safe temporary access
- → `expired()` — true once the object is gone
- → `prev` as `weak_ptr` — avoids a cycle that would never be freed
- → `shared_from_this()` — get a `shared_ptr` to `this` safely

SHARED VS UNIQUE

ASPECT	UNIQUE_PTR	SHARED_PTR
owners	one	many
copyable	no	yes
size	1 pointer	2 pointers
overhead	none	atomic ref count
default choice	✔	only when needed

WEAK_PTR API

MEMBER	PURPOSE
lock()	shared_ptr or empty
expired()	object destroyed?
use_count()	strong owner count
reset()	stop observing

⚠ COMMON MISTAKES

- Two `shared_ptr`s built from the same raw pointer (`shared_ptr<T>(raw)` twice); each deletes it
- Reference cycles (A → B → A) with `shared_ptr`; neither count reaches zero, so both leak
- Calling `shared_from_this()` in a constructor or on an object not owned by a `shared_ptr`

✓ TIPS & BEST PRACTICES

- Start with `unique_ptr`; convert to `shared_ptr` only when ownership is truly shared (`unique_ptr` converts implicitly)
- Pass `const T&` or `T*` to functions that don't care about ownership
- Use `weak_ptr` for caches, observers, and back-pointers

QUICK SUMMARY

- `std::shared_ptr` — shared ownership, ref-counted
- `make_shared` — one allocation
- `std::weak_ptr` — non-owning observer, breaks cycles
- `enable_shared_from_this` — safe self-reference

RAII & Ownership Guidelines

RESOURCE OWNERSHIP RULES

RAII (Resource Acquisition Is Initialization) ties every resource to an object's lifetime: acquire in the constructor, release in the destructor. Because destructors run on every exit path, including exceptions, RAII makes leaks structurally impossible. Combined with clear ownership types (smart pointers and containers own; raw pointers, references, `span`, and `string_view` observe), it's the core of safe modern C++.

CORE GUIDELINES (R.*)

- R.1: manage resources automatically using RAII
- R.3: a raw pointer `T*` is non-owning
- R.11: avoid calling `new` and `delete` explicitly
- R.20/21: use `unique_ptr` or `shared_ptr` to represent ownership

RAII WRAPPERS

```
class File {                                    // RAII wrapper
public:
    explicit File(const char* path) : f_(std::fopen(path, "r")) {
        if (!f_) throw std::runtime_error("open failed");
    }
    ~File() { std::fclose(f_); }
    File(const File&) = delete;                 // non-copyable
    File& operator=(const File&) = delete;
    std::FILE* get() const { return f_; }
private:
    std::FILE* f_;
};
void work(std::mutex& m) {
    std::scoped_lock lock{m};                   // RAII lock
    File cfg{"app.cfg"};                        // RAII file
    parse(cfg.get());                           // may throw
}                                               // file closed, mutex unlocked
```

CODE EXPLAINED

- → constructor acquires and throws if it can't
- → destructor releases unconditionally
- → deleted copies — one owner per handle
- → `scoped_lock` — the standard library's RAII lock
- → on any exit (return or throw), both resources are released

OWNERSHIP VOCABULARY

TYPE	OWNS?	USE
`T (value)`	yes	default
`std::unique_ptr<T>`	yes, exclusive	heap / polymorphic
`std::shared_ptr<T>`	yes, shared	shared lifetime
`T* / T&`	no	observe / use
`std::span / string_view`	no	view sequences
`std::weak_ptr`	no	observe shared

STANDARD RAII TYPES

RESOURCE	RAII TYPE
`heap memory`	unique_ptr, vector, string
`mutex`	scoped_lock, unique_lock
`file`	ifstream / ofstream
`thread`	std::jthread
`C handle`	unique_ptr + deleter

⚠ COMMON MISTAKES

- Manual `lock()` / `unlock()` or `fopen` / `fclose` pairs; any early return or exception skips the release
- Treating a raw pointer parameter as owning and deleting it
- Writing an RAII class but leaving copying enabled, so two objects release the same handle

✓ TIPS & BEST PRACTICES

- Follow the **rule of zero**: compose RAII members so your class needs no destructor
- Use `std::unique_ptr` with a custom deleter as a one-line RAII wrapper for C APIs
- Make ownership visible in signatures: `unique_ptr` by value means "I take it"

QUICK SUMMARY

- RAII — acquire in ctor, release in dtor
- owning types — values, smart pointers, containers
- non-owning — `T*`, `T&`, `span`, `string_view`
- rule of zero — let members own resources

Function Templates

GENERIC FUNCTIONS

A **function template** is a blueprint for a family of functions: `template <typename T>` introduces a type parameter, and the compiler **instantiates** a concrete function for each set of types it's called with. Template arguments are usually **deduced** from the call's arguments. C++20 **abbreviated templates** let you write `auto` parameters directly, and concepts can constrain them.

NOTES

- `typename` and `class` are interchangeable in parameter lists
- Templates must be visible where used: define them in headers
- Deduction ignores implicit conversions: `max(1, 2.0)` fails
- Specify explicitly when needed: `max<double>(1, 2.0)`

FUNCTION TEMPLATES

```
template <typename T>
T maxOf(T a, T b) { return a < b ? b : a; }

maxOf(3, 7);                  // T = int → 7
maxOf(2.5, 1.5);              // T = double → 2.5
maxOf<std::string>("a", "b");// explicit T → "b"
// maxOf(1, 2.0);             // ERROR: T deduced as int and double

template <typename T, typename U>
auto add(T a, U b) { return a + b; }   // return type deduced
add(1, 2.5);                  // 3.5 (double)

auto printAll(const auto& container) { // abbreviated (C++20)
    for (const auto& x : container) std::print("{} ", x);
}
printAll(std::vector{1, 2, 3});        // 1 2 3

void sink(std::integral auto n);       // constrained auto (C++20)
```

CODE EXPLAINED

- → `template <typename T>` — T is a placeholder type
- → each distinct T creates a separate instantiation
- → mixed argument types — deduction conflict; use two parameters or explicit args
- → `auto` parameter — shorthand for a template parameter
- → `std::integral auto` — only integer types accepted

SYNTAX

FORM	MEANING
`template <typename T>`	type parameter
`template <class T>`	same thing
`f<int>(x)`	explicit argument
`void f(auto x)`	abbreviated template
`void f(Concept auto x)`	constrained
`template <class T = int>`	default argument

DEDUCTION EXAMPLES

CALL TO F(T X)	DEDUCED T
`f(42)`	int
`f("hi")`	const char*
`f(arr) (int[3])`	int*
`f(cx) (const int)`	int (top-level const dropped)
`f<long>(42)`	long

⚠ COMMON MISTAKES

- Defining a template in a `.cpp` file and calling it from another; you get "undefined reference". Put it in the header
- Expecting implicit conversions during deduction (`maxOf(1, 2.0)`)
- Huge error messages from unconstrained templates; add concepts to get clear errors

✓ TIPS & BEST PRACTICES

- Constrain templates with concepts (`std::totally_ordered T`) to document requirements
- Use `std::type_identity_t<T>` on a parameter to exclude it from deduction
- Look at instantiations with C++ Insights (cppinsights.io)

QUICK SUMMARY

- `template <typename T>` — generic function
- deduction — T inferred from arguments
- `f<T>(…)` — explicit template arguments
- `auto` params — abbreviated templates (C++20)

Class Templates

GENERIC TYPES

A **class template** defines a family of types: `std::vector<int>` and `std::vector<std::string>` are different classes instantiated from one template. Member functions are only instantiated when used, so unused members don't need to compile for every T. **CTAD** (class template argument deduction, C++17) deduces template arguments from constructor arguments, and **deduction guides** customize it.

NOTES

- Inside the class, the name alone (`Stack`) means `Stack<T>`
- Out-of-class members repeat `template <class T>` and use `Stack<T>::`
- `std::pair p{1, 2.5};` — CTAD at work
- Default template arguments work like default function arguments

A CLASS TEMPLATE

```
template <typename T, std::size_t Cap = 16>
class Stack {
public:
    void push(T v) { data_.push_back(std::move(v)); }
    T pop();
private:
    std::vector<T> data_;
};
template <typename T, std::size_t Cap>          // out-of-class member
T Stack<T, Cap>::pop() {
    T v = std::move(data_.back()); data_.pop_back(); return v;
}
Stack<int> s; s.push(4); s.pop();   // 4   (Cap = 16 by default)
template <class T> struct Box { T value; };
template <class T> Box(T) -> Box<T>;           // deduction guide
Box b{3.5};                // Box<double> via CTAD
```

CODE EXPLAINED

- → `Cap = 16` — default template argument
- → members defined in the class are implicitly inline
- → out-of-class definition repeats the template header
- → deduction guide — `Box(T) -> Box<T>` (aggregates need it before C++20)
- → `Box b{3.5}` — CTAD deduces `Box<double>`

USING CLASS TEMPLATES

CODE	MEANING
`Stack<int>`	explicit arguments
`std::vector v{1, 2};`	CTAD → vector<int>
`std::pair p{1, 'a'};`	CTAD → pair<int, char>
`Stack<int>::value_type`	nested type (if defined)

MEMBER KINDS

MEMBER	NOTES
`regular member`	instantiated on use
`static member`	one per instantiation
`friend`	hidden friends work well
`nested type`	needs `typename` when dependent

⚠ COMMON MISTAKES

- Splitting template member definitions into a `.cpp` ; link errors follow. Keep them in the header
- Forgetting `typename` before dependent nested types: `typename T::value_type x;`
- Surprises from CTAD with braces: `std::vector v{3, 4};` is two ints, not three fours

✓ TIPS & BEST PRACTICES

- Keep class templates small; move type-independent code into non-template helpers to reduce bloat
- Add deduction guides when constructor parameters don't map directly to template parameters
- Constrain parameters with `requires` clauses to get clear errors

QUICK SUMMARY

- `template <class T> class X` — generic type
- `X<T>::member` — out-of-class definitions
- CTAD — deduce arguments from constructors (C++17)
- deduction guides — steer CTAD

Non-Type Parameters & Specialization

VALUE PARAMETERS & SPECIALIZATION

Template parameters can be **values** as well as types: integers, enums, pointers, and since C++20 floating-point values and literal class types. These **non-type template parameters (NTTPs)** are compile-time constants, like N in `std::array<T, N>`. **Specialization** provides a custom implementation for particular arguments: a **full specialization** for one exact set, or a **partial specialization** (class templates only) for a pattern.

NOTES

- `template <auto N>` (C++17) deduces the NTTP's type
- Function templates can't be partially specialized; overload instead
- Specializations must be declared before first use
- Class-type NTTPs need public members and structural types (C++20)

NTTPS & SPECIALIZATION

```
template <typename T, std::size_t N>
constexpr std::size_t count(const T (&)[N]) { return N; }
int a[7]{};  count(a);                          // 7

template <auto V> constexpr auto constant = V;
constant<42>;  constant<'x'>;                   // int, char

template <class T> struct TypeName     { static constexpr auto value = "?"; };
template <> struct TypeName<int>       { static constexpr auto value = "int"; };
template <class T> struct TypeName<T*> { static constexpr auto value = "ptr"; };
TypeName<int>::value;                           // "int"
TypeName<double*>::value;                       // "ptr"

struct Fixed {
    char s[8]; constexpr Fixed(const char* p) { std::copy_n(p, 8, s); }
};
template <Fixed Name> struct Tag {};            // class-type NTTP (C++20)
Tag<"metrics"> t;
```

CODE EXPLAINED

- → `T (&)[N]` — deduces the array size as a template argument
- → `template <auto V>` — value and its type deduced together
- → `template <>` — full specialization for exactly `int`
- → `TypeName<T*>` — partial specialization for any pointer type
- → class-type NTTP — lets string literals be template arguments

ALLOWED NTTP TYPES

TYPE	SINCE
integers, enums, bool	C++98
pointers / references	C++98
std::nullptr_t	C++11
auto (deduced)	C++17
floating point	C++20
structural class types	C++20

SPECIALIZATION KINDS

KIND	CLASSES	FUNCTIONS
full specialization	✔	✔ (prefer overloads)
partial specialization	✔	✖
variable template spec.	✔	—

⚠ COMMON MISTAKES

- Trying to partially specialize a function template; it's not allowed. Overload or use `if constexpr`
- Specializing after the template was already used with those arguments (ill-formed, no diagnostic)
- Declaring specializations in a different namespace from the primary template

✓ TIPS & BEST PRACTICES

- Prefer `if constexpr` inside one template over many tiny specializations
- Specialize standard traits only where allowed (e.g. `std::hash`, `std::formatter`)
- Use NTTPs for sizes and compile-time configuration, like `Matrix<3, 3>`

QUICK SUMMARY

- NTTP — compile-time value parameter
- `template <auto V>` — deduced NTTP type (C++17)
- full specialization — `template <> struct X<int>`
- partial specialization — patterns like `X<T*>`

Variadic Templates & Fold Expressions

ANY NUMBER OF ARGUMENTS

A **variadic template** takes a **parameter pack** of zero or more types: `template <typename... Ts>`. The pack is expanded with `...` wherever a comma-separated list is valid. **Fold expressions** (C++17) reduce a pack with a binary operator in one expression. Combined with forwarding references and `std::forward`, packs enable **perfect forwarding**, which powers `emplace_back` and `make_unique`. C++26 adds direct **pack indexing** (`Ts...[0]`).

NOTES

- `sizeof...(Ts)` — number of elements in the pack
- `f(args...)` — expand arguments; `f(g(args)...)` — apply g to each
- Unary folds: `(... op pack)` / `(pack op ...)`
- An empty pack is fine for `&&`, `||`, and `,` folds

PACKS, FOLDS, FORWARDING

```
template <typename... Ts>
auto sum(Ts... xs) { return (xs + ... + 0); }       // binary fold
sum(1, 2, 3, 4);                                    // 10
sum();                                              // 0

template <typename... Ts>
void printAll(const Ts&... xs) {
    ((std::cout << xs << ' '), ...);                // comma fold
}
printAll("id", 7, 2.5);                             // id 7 2.5

template <typename... Ts>
constexpr std::size_t count = sizeof...(Ts);
count<int, char, double>;                           // 3

template <typename T, typename... Args>
std::unique_ptr<T> make(Args&&... args) {           // perfect forwarding
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
template <typename... Ts> using First = Ts...[0];   // C++26 pack indexing
```

CODE EXPLAINED

- → `Ts... xs` — function parameter pack
- → `(xs + ... + 0)` — `((x1 + x2) + x3) + 0`; safe when empty
- → `(expr, ...)` — runs `expr` for each element in order
- → `sizeof...` — pack length at compile time
- → `std::forward<Args>(args)...` — forwards each argument with its category
- → `Ts...[0]` — first type in the pack (C++26)

FOLD FORMS

FORM	EXPANDS TO
`(pack op ...)`	p1 op (p2 op p3)
`(... op pack)`	(p1 op p2) op p3
`(pack op ... op init)`	p1 op (p2 op init)
`(init op ... op pack)`	(init op p1) op p2

EXPANSION PATTERNS

PATTERN	RESULT
`f(args...)`	f(a1, a2, a3)
`f(g(args)...)`	f(g(a1), g(a2), g(a3))
`std::tuple<Ts...>`	tuple<T1, T2, T3>
`Base<Ts>...`	multiple bases
`[...xs = xs]`	lambda pack capture (C++20)

⚠ COMMON MISTAKES

- Forgetting the parentheses around a fold; `xs + ...` alone doesn't compile
- Using `std::move` instead of `std::forward` on `Args&&...`, which moves from lvalue arguments
- Recursive variadic templates for simple reductions; a fold expression is shorter and compiles faster

✓ TIPS & BEST PRACTICES

- Use the comma fold `(f(xs), ...)` to call a function on each argument in order
- Constrain packs: `template <std::integral... Ts>`
- Use `std::tuple` and `std::apply` to store a pack and expand it later

QUICK SUMMARY

- `typename... Ts` — parameter pack
- `sizeof...(Ts)` — pack size
- `(xs op ...)` — fold expressions (C++17)
- `std::forward<Args>(args)...` — perfect forwarding

Concepts & requires

CONSTRAINING TEMPLATES

A **concept** (C++20) is a named, compile-time predicate on template arguments. Constraints say what a template needs, produce short readable errors, and let overload resolution pick the most specific candidate (**subsumption**). Write constraints with a `requires` clause, as a type-constraint (`template <std::integral T>`), or on `auto`. **Requires expressions** check that operations compile.

NOTES

- Standard concepts live in `<concepts>`, `<iterator>`, `<ranges>`
- `requires requires (T x) { … }` — ad-hoc constraint (rarely needed)
- More constrained overloads win over less constrained ones
- Concepts are evaluated, never instantiated: no code generated

CONCEPTS IN ACTION

```cpp
#include <concepts>

template <typename T>
concept Shape = requires(const T& s) {
    { s.area() } -> std::convertible_to<double>;   // must return ~double
    { s.name() } -> std::same_as<std::string>;
};
template <Shape S> void report(const S& s) {       // type-constraint
    std::println("{}: {}", s.name(), s.area());
}
template <typename T> requires std::integral<T>    // requires clause
T gcd(T a, T b) { return b == 0 ? a : gcd(b, a % b); }
auto half(std::floating_point auto x) { return x / 2; }   // constrained auto
template <std::integral T>        void describe(T) { std::print("int"); }
template <std::signed_integral T> void describe(T) { std::print("signed"); }
describe(5u);   // int
describe(-5);   // signed (more constrained wins)
```

CODE EXPLAINED

- → `concept Shape = requires …` — define a concept from required expressions
- → `{ expr } -> Concept` — expression must compile and satisfy the concept
- → `template <Shape S>` — shortest constraint syntax
- → `requires std::integral<T>` — clause form, can combine with `&&`
- → subsumption — `signed_integral` refines `integral`, so it's preferred

STANDARD CONCEPTS

CONCEPT	SATISFIED BY
`std::integral`	int, char, bool …
`std::floating_point`	float, double
`std::same_as<U>`	exactly U
`std::convertible_to<U>`	implicitly converts to U
`std::totally_ordered`	< > <= >= ==
`std::invocable<Args…>`	callable with Args
`std::ranges::range`	has begin/end

REQUIRES EXPRESSION PARTS

REQUIREMENT	EXAMPLE
`simple`	`a + b;`
`type`	`typename T::value_type;`
`compound`	`{ x.size() } -> std::integral;`
`nested`	`requires sizeof(T) <= 8;`

⚠ COMMON MISTAKES

- Writing a requirement without braces: `s.area() -> std::convertible_to<double>;` (needs `{ s.area() }`)
- Over-constraining with `std::same_as<int>` when `std::integral` or `std::convertible_to` is what you mean
- Expecting a concept to check semantics (like "sorted"); it only checks that code compiles

✓ TIPS & BEST PRACTICES

- Start with standard concepts before writing your own
- Name concepts after capabilities (`Drawable`, `Serializable`) rather than types
- Use `static_assert(Shape<Circle>);` to test that your types model a concept

QUICK SUMMARY

- `concept C = …` — named compile-time predicate
- `template <C T>` / `requires C<T>` / `C auto` — three spellings
- requires expression — check that operations compile
- subsumption — most constrained overload wins

Type Traits & SFINAE

COMPILE-TIME TYPE LOGIC

<type_traits> provides compile-time queries and transformations on types. **Predicates** like `std::is_integral_v<T>` yield a `bool`; **transformations** like `std::remove_cvref_t<T>` yield a new type. Before concepts, templates were constrained with **SFINAE** ("substitution failure is not an error"): an overload whose signature fails to substitute is silently dropped. You'll still see `std::enable_if` in older code.

NAMING

- `_v` suffix — value shortcut: `is_same_v<A, B>` = `is_same<A, B>::value`
- `_t` suffix — type shortcut: `decay_t<T>` = `typename decay<T>::type`
- `std::declval<T>()` — pretend to have a T in unevaluated contexts
- Prefer `if constexpr` and concepts over SFINAE in new code

TRAITS & SFINAE

```cpp
#include <type_traits>

static_assert(std::is_integral_v<long>);
static_assert(std::is_same_v<std::remove_cvref_t<const int&>, int>);
std::is_trivially_copyable_v<std::string>;     // false

template <typename T>
using Storage = std::conditional_t<(sizeof(T) <= 8), T, const T&>;
template <typename T>
std::string describe(const T& v) {
    if constexpr (std::is_arithmetic_v<T>) return std::to_string(v);
    else if constexpr (std::is_convertible_v<T, std::string>) return v;
    else return "?";
}
// SFINAE (pre-C++20 style):
template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
T twice(T x) { return x * 2; }
using R = decltype(std::declval<Widget&>().size());   // member's return type
```

CODE EXPLAINED

- → `static_assert` + traits — compile-time checks
- → `remove_cvref_t` — strip `const`, `volatile`, and references
- → `conditional_t<cond, A, B>` — pick a type at compile time
- → `if constexpr` + traits — modern replacement for SFINAE overloads
- → `enable_if_t` — removes the overload when the condition is false
- → `declval` — get an expression's type without constructing a Widget

PREDICATES

TRAIT	TRUE FOR
`is_integral_v`	int, char, bool …
`is_floating_point_v`	float, double
`is_pointer_v`	T*
`is_same_v<A, B>`	identical types
`is_base_of_v<B, D>`	D derives from B
`is_trivially_copyable_v`	memcpy-safe

TRANSFORMATIONS

TRAIT	RESULT FOR `CONST INT&`
`remove_reference_t`	const int
`remove_cvref_t`	int
`decay_t`	int
`add_pointer_t`	const int*
`make_unsigned_t<int>`	unsigned int
`underlying_type_t<E>`	enum's integer type

⚠ COMMON MISTAKES

- Forgetting `typename` with the long form: `typename std::decay<T>::type`; use the `_t` alias
- Writing complex SFINAE when a concept or `if constexpr` would be clearer
- Using `std::is_same_v<T, std::string>` in generic code that receives `const std::string&` without removing cvref first

✓ TIPS & BEST PRACTICES

- Reach for `std::remove_cvref_t<T>` before comparing deduced types
- Replace `enable_if` with `requires` clauses when upgrading code to C++20
- Use `static_assert(trait, "message")` to give users clear errors

QUICK SUMMARY

- `is_*_v` — compile-time type predicates
- `*_t` — type transformations
- SFINAE / `enable_if` — legacy constraint technique
- `std::declval` — types of expressions in unevaluated contexts

Template Idioms

GENERIC DESIGN PATTERNS

A handful of template idioms show up constantly in real code. **CRTP** (Curiously Recurring Template Pattern) lets a base class call into its derived class at compile time, giving static polymorphism with no vtable. **Policy-based design** plugs behavior in through template parameters. **Tag dispatch** picks overloads with empty tag types. **Type erasure** hides concrete types behind a uniform interface, as `std::function` and `std::any` do.

NOTES

- CRTP: `class Derived : public Base<Derived>`
- C++23 deducing `this` replaces many CRTP uses
- Tag dispatch is mostly superseded by `if constexpr` and concepts
- **Template template parameters**: `template <template <class> class C>`

CRTP, POLICY, TYPE ERASURE

```
template <class Derived>
struct Printable {                                  // CRTP base
    void print() const { std::println("{}", self().toString()); }
    const Derived& self() const { return static_cast<const Derived&>(*this); }
};
struct Point : Printable<Point> {
    int x, y;
    std::string toString() const { return std::format("({}, {})", x, y); }
};
Point{1, 2}.print();                                // (1, 2)

template <class LockPolicy = std::mutex>            // policy-based
class Registry { LockPolicy lock_; /* ... */ };
Registry<NullMutex> fastSingleThreaded;
template <template <class...> class Container>      // template template param
Container<int> makeInts() { return {1, 2, 3}; }
auto v = makeInts<std::vector>();
std::vector<std::function<void()>> tasks;           // type erasure in std
```

CODE EXPLAINED

- → CRTP — `Printable` knows the derived type at compile time
- → `static_cast<const Derived&>(*this)` — the CRTP downcast
- → policy parameter — swap locking strategy without virtual calls
- → template template parameter — pass `std::vector` itself, not `std::vector<int>`
- → `std::function` — holds any callable behind one type

IDIOMS

IDIOM	SOLVES
CRTP	static polymorphism, mixins
policy-based design	configurable behavior
tag dispatch	overload by category
type erasure	uniform interface, no base class
expression templates	lazy math (Eigen)

MODERN REPLACEMENTS

OLD IDIOM	MODERN TOOL
CRTP for this access	deducing this (C++23)
tag dispatch	if constexpr / concepts
enable_if	requires clauses
traits classes	concepts + reflection (C++26)

⚠ COMMON MISTAKES

- In CRTP, deriving `class B : public Base<A>` by copy-paste mistake; `self()` casts to the wrong type (UB)
- Exposing policy parameters in every public type name, making signatures unreadable; add aliases
- Writing a custom type-erasure class when `std::function`, `std::any`, or `std::move_only_function` already fits

✓ TIPS & BEST PRACTICES

- Make the CRTP base constructor private and `friend Derived` to prevent wrong-type derivation
- Provide defaults for policy parameters so most users never see them
- Study `std::function` and `std::any` implementations to learn type erasure (small-buffer optimization included)

QUICK SUMMARY

- CRTP — base templated on derived
- policies — behavior via template parameters
- tag dispatch — overload on empty types
- type erasure — hide types behind one interface

Iterators

THE GLUE OF THE STL

An **iterator** is a generalized pointer that walks through a container. Every container provides `begin()` (first element) and `end()` (one **past** the last), forming a half-open range `[begin, end)`. Algorithms work on iterator pairs, so one `std::sort` works with any container that offers the right **iterator category**. Inserters like `std::back_inserter` let algorithms append to containers.

NOTES

- `*it` reads/writes, `++it` advances, `it == end` stops
- `cbegin()` / `cend()` — const iterators (read-only)
- `rbegin()` / `rend()` — reverse traversal
- Many operations **invalidate** iterators; see the container sheets

USING ITERATORS

```
std::vector<int> v{10, 20, 30, 40};

for (auto it = v.begin(); it != v.end(); ++it) *it += 1;    // 11 21 31 41
for (auto it = v.cbegin(); it != v.cend(); ++it) {}         // read-only
for (auto it = v.rbegin(); it != v.rend(); ++it) {}   // visits 41 31 21 11

auto it = std::find(v.begin(), v.end(), 31);
std::distance(v.begin(), it);           // 2
auto third = std::next(v.begin(), 2);   // iterator to 31
auto prev  = std::prev(v.end());        // iterator to 41
std::advance(it, -1);                   // now at 21

std::vector<int> out;
std::copy_if(v.begin(), v.end(), std::back_inserter(out),
             [](int x) { return x > 20; });   // out: 21 31 41
std::list<int> l{1, 2, 3};
// l.begin() + 1;                       // ERROR: list isn't random access
```

CODE EXPLAINED

- → half-open range — `end()` is never dereferenced
- → `cbegin` — prevents accidental writes
- → `std::distance` — element count between iterators
- → `std::next` / `std::prev` / `std::advance` — move by n
- → `std::back_inserter` — calls `push_back` for each output
- → list iterators are bidirectional, so no `+ n`

CATEGORIES

CATEGORY	OPERATIONS	EXAMPLE CONTAINER
`input / output`	single pass	istream_iterator
`forward`	++, multi-pass	forward_list
`bidirectional`	++, --	list, map, set
`random access`	+n, -n, [], <	deque
`contiguous`	+ memory is contiguous	vector, array, string

HELPERS

FUNCTION	PURPOSE
`std::begin / std::end`	works on C arrays too
`std::next / std::prev`	copy moved by n
`std::advance(it, n)`	move in place
`std::distance(a, b)`	number of steps
`std::back_inserter(c)`	push_back output
`std::inserter(c, pos)`	insert output

⚠ COMMON MISTAKES

- Dereferencing `end()`; it points past the last element
- Using an iterator after the container reallocated or erased that element
- Comparing iterators from two different containers; it's UB

✓ TIPS & BEST PRACTICES

- Prefer range-for and range algorithms; reach for raw iterators only when you need positions
- Use `std::ranges::begin(r)` in generic code; it handles more range types
- Write `auto it` rather than spelling out `std::vector<int>::iterator`

QUICK SUMMARY

- `[begin, end)` — half-open range
- categories — input → forward → bidirectional → random → contiguous
- `next / prev / distance` — iterator arithmetic helpers
- `back_inserter` — let algorithms append

Algorithms — Non-Modifying

QUERYING RANGES

Non-modifying algorithms in <algorithm> inspect a range without changing it: finding elements, counting, checking predicates, and comparing ranges. Each classic form takes an iterator pair; the C++20 `std::ranges::` versions accept a whole container and optional **projections**. Finding functions return an iterator, equal to `end` when nothing matched.

NOTES

- `_if` variants take a predicate instead of a value
- `find` is O(n); use `set` / `map` / `binary_search` for repeated lookups
- `all_of` on an empty range is `true`; `any_of` is `false`
- `std::ranges::contains` (C++23) — direct yes/no

FIND, COUNT, ALL_OF & FRIENDS

```
std::vector<int> v{3, 8, 1, 8, 5};

auto it = std::ranges::find(v, 8);              // first 8 (index 1)
if (it != v.end()) {}
auto big = std::ranges::find_if(v, [](int x) { return x > 4; });   // 8
std::ranges::count(v, 8);                       // 2
std::ranges::count_if(v, [](int x) { return x % 2; });           // 3

std::ranges::all_of(v,  [](int x) { return x > 0; });   // true
std::ranges::any_of(v,  [](int x) { return x > 7; });   // true
std::ranges::none_of(v, [](int x) { return x < 0; });   // true
std::ranges::contains(v, 5);                    // true   (C++23)

auto [mn, mx] = std::ranges::minmax(v);         // 1, 8
std::ranges::equal(v, std::vector{3, 8, 1, 8, 5});   // true
auto adj = std::ranges::adjacent_find(v);       // first equal neighbors?
auto pos = std::ranges::search(v, std::vector{1, 8}); // subrange at index 2
auto oldest = std::ranges::max_element(people, {}, &Person::age);
```

CODE EXPLAINED

- → `find` / `find_if` — iterator to first match or `end()`
- → `count` / `count_if` — number of matches
- → `all_of` / `any_of` / `none_of` — boolean checks
- → `minmax` — both extremes in one pass
- → `search` — find a sub-sequence
- → projection `&Person::age` — compare by a member without a lambda

ALGORITHMS

ALGORITHM	RETURNS
`find / find_if / find_if_not`	iterator
`count / count_if`	count
`all_of / any_of / none_of`	bool
`min / max / minmax`	value(s)
`min_element / max_element`	iterator
`equal / mismatch`	bool / iterator pair
`search / find_end`	subrange position

CLASSIC VS RANGES

CLASSIC	RANGES (C++20)
`std::find(v.begin(), v.end(), x)`	std::ranges::find(v, x)
`std::count_if(b, e, p)`	std::ranges::count_if(v, p)
`lambda for member`	projection `&T::member`
`returns iterator`	also returns subrange / dangling

⚠ COMMON MISTAKES

- Not checking `find`'s result against `end()` before dereferencing
- Calling `std::max_element` and treating the result as a value; it's an iterator
- Using `count(...) > 0` just to test membership; `any_of` / `contains` stop at the first match

✓ TIPS & BEST PRACTICES

- Use projections to search or compare by a member: `ranges::find(users, id, &User::id)`
- Use `std::ranges::contains` for readable membership tests in C++23
- Switch to a hash set if you call `find` on the same large vector many times

QUICK SUMMARY

- `find / find_if` — locate an element
- `count / count_if` — tally matches
- `all_of / any_of / none_of` — predicate checks
- projections — operate on members directly

Algorithms — Modifying

TRANSFORMING RANGES

Modifying algorithms write to a range: copying, transforming, filling, replacing, removing, and reordering. Output usually goes to a destination iterator that must have enough room, or to an inserter like `std::back_inserter`. `std::remove` / `remove_if` don't change the container's size; they shift kept elements forward and return the new logical end, which is why C++20's `std::erase_if` exists.

NOTES

- Destination ranges must be large enough (or use an inserter)
- `transform` can write in place (`out = in.begin()`)
- `unique` removes **consecutive** duplicates only; sort first
- Many algorithms have `_copy` variants that write elsewhere

COPY, TRANSFORM, REPLACE, UNIQUE

```cpp
std::vector<int> v{1, 2, 3, 4, 5, 6};
std::vector<int> out;

std::ranges::copy(v, std::back_inserter(out));                  // copy all
std::ranges::copy_if(v, std::back_inserter(out),
                     [](int x) { return x % 2 == 0; });         // 2 4 6
std::ranges::transform(v, v.begin(), [](int x) { return x * x; }); // in place
std::ranges::fill(out, 0);                                      // all 0
std::ranges::replace_if(v, [](int x) { return x > 20; }, -1);   // 1 4 9 16 -1 -1

std::erase_if(v, [](int x) { return x < 0; });                  // 1 4 9 16
std::ranges::reverse(v);                                        // 16 9 4 1
std::ranges::rotate(v, v.begin() + 1);                          // 9 4 1 16

std::vector<int> d{3, 1, 3, 2, 1};
std::ranges::sort(d);                                           // 1 1 2 3 3
auto dup = std::ranges::unique(d);          // leftover subrange
d.erase(dup.begin(), dup.end());                                // 1 2 3
```

CODE EXPLAINED

- → `back_inserter` — grows the output as needed
- → `transform(src, dst, f)` — apply f to each element
- → `replace_if(r, pred, value)` — overwrite matches
- → `erase_if` — real removal in one call
- → `rotate` — `v.begin() + 1` becomes the new first element
- → `unique` + `erase` — dedupe a sorted vector

ALGORITHMS

ALGORITHM	EFFECT
`copy / copy_if / copy_n`	copy elements
`move`	move elements
`transform`	map with a function
`fill / fill_n / generate`	assign values
`replace / replace_if`	overwrite matches
`remove / remove_if`	shift kept to front
`reverse / rotate / shuffle`	reorder

REMOVE IDIOM

STEP	CODE
`classic`	`v.erase(remove(b, e, x), e)`
`ranges`	`auto r = ranges::remove(v, x); v.erase(r.begin(), r.end());`
`C++20`	`std::erase(v, x)`
`predicate`	`std::erase_if(v, pred)`

⚠ COMMON MISTAKES

- Copying into an empty vector with `v.begin()` as the destination; that's UB. Use `back_inserter` or `resize` first
- Calling `std::remove` and expecting the size to shrink
- Using `unique` on unsorted data and expecting all duplicates gone

✓ TIPS & BEST PRACTICES

- Prefer `std::erase_if` over the remove-erase idiom
- Reserve the destination when you know the output size: `out.reserve(v.size());`
- Use `std::ranges::to` (C++23) with views to build new containers lazily instead of copying

QUICK SUMMARY

- `copy / transform` — write to an output range
- `fill / replace` — assign values
- `remove` + `erase` / `erase_if` — actually delete
- `unique` — consecutive duplicates; sort first

Sorting & Searching

ORDERING & BINARY SEARCH

`std::sort` sorts random-access ranges in O(n log n), but it isn't **stable**: equal elements may change order. Use `std::stable_sort` when that matters. `partial_sort` and `nth_element` do less work when you only need the top k or the median. On sorted data, **binary search** algorithms find elements in O(log n). All accept a custom comparator, and the ranges versions accept projections.

COMPARATOR RULES

- Must be a **strict weak ordering**: use `<`, never `<=`
- `std::greater<>{}` — descending order
- Binary search requires data sorted by the **same** comparator
- `std::list` has its own `.sort()` member

SORTING & SEARCHING

```
std::vector<int> v{5, 2, 9, 1, 7};
std::ranges::sort(v);                                  // 1 2 5 7 9
std::ranges::sort(v, std::greater<>{});                // 9 7 5 2 1

struct Emp { std::string name; int dept; double pay; };
std::ranges::sort(staff, {}, &Emp::pay);               // by pay (projection)
std::ranges::stable_sort(staff, {}, &Emp::dept);       // keeps pay order
std::ranges::sort(staff, [](const Emp& a, const Emp& b) {
    return std::tie(a.dept, a.name) < std::tie(b.dept, b.name);  // multi-key
});

std::ranges::partial_sort(v, v.begin() + 3);           // smallest 3 first
std::ranges::nth_element(v, v.begin() + v.size() / 2);   // median in place

std::vector<int> s{1, 3, 5, 7, 9};
std::ranges::binary_search(s, 7);                      // true
auto lo = std::ranges::lower_bound(s, 6);              // first >= 6 → 7
auto hi = std::ranges::upper_bound(s, 7);              // first > 7 → 9
auto [b, e] = std::ranges::equal_range(s, 5);          // range of 5s
```

CODE EXPLAINED

- → `std::greater<>{}` — reverse order
- → projection `&Emp::pay` — sort by a member without a lambda
- → `stable_sort` after sorting by pay — groups by dept, keeps pay order
- → `std::tie` — lexicographic multi-key comparison
- → `nth_element` — O(n) average; element at position n is the one a full sort would put there
- → `lower_bound` — insertion point for keeping a vector sorted

SORTING

ALGORITHM	COMPLEXITY	STABLE?
`sort`	O(n log n)	no
`stable_sort`	O(n log n)	yes
`partial_sort`	O(n log k)	no
`nth_element`	O(n) avg	no
`is_sorted`	O(n)	—

BINARY SEARCH (SORTED INPUT)

ALGORITHM	RETURNS
`binary_search`	bool
`lower_bound`	first ≥ value
`upper_bound`	first > value
`equal_range`	[lower, upper)

⚠ COMMON MISTAKES

- A comparator using `<=`; it violates strict weak ordering and can crash `std::sort`
- Running `binary_search` / `lower_bound` on unsorted data
- Sorting by two keys with two `sort` calls instead of one comparator or `stable_sort`

✓ TIPS & BEST PRACTICES

- Insert into a sorted vector with `v.insert(std::ranges::upper_bound(v, x), x);`
- Use `partial_sort` or `nth_element` for top-k problems instead of a full sort
- Projections plus `std::greater<>` sort descending by member: `ranges::sort(v, std::greater<>{}, &T::score)`

QUICK SUMMARY

- `sort` / `stable_sort` — order a range
- comparators — strict weak ordering (`<`)
- `partial_sort` / `nth_element` — partial work
- `lower_bound` / `binary_search` — O(log n) on sorted data

Numeric Algorithms

NUMERIC ALGORITHMS

<numeric> holds algorithms for summing and combining values. `std::accumulate` folds left to right; `std::reduce` (C++17) does the same but may reorder operations, which allows **parallel execution policies** from <execution>. `std::transform_reduce` maps then reduces in one pass. `std::iota` fills a range with increasing values, and `std::partial_sum` produces running totals.

NOTES

- The type of the initial value decides the result type
- `reduce` requires an associative, commutative operation
- Policies: `std::execution::seq`, `par`, `par_unseq`, `unseq`
- C++23 adds `std::ranges::fold_left` / `fold_right`

ACCUMULATE, REDUCE, IOTA

```cpp
#include <numeric>
#include <execution>

std::vector<int> v{1, 2, 3, 4, 5};
std::accumulate(v.begin(), v.end(), 0);           // 15
std::accumulate(v.begin(), v.end(), 1, std::multiplies<>{}); // 120
std::accumulate(prices.begin(), prices.end(), 0.0);  // use 0.0 for doubles!

std::reduce(std::execution::par, big.begin(), big.end(), 0LL);
std::transform_reduce(a.begin(), a.end(), b.begin(), 0.0);   // dot product

std::vector<int> ids(5);
std::iota(ids.begin(), ids.end(), 100);           // 100 101 102 103 104
std::vector<int> run(v.size());
std::partial_sum(v.begin(), v.end(), run.begin()); // 1 3 6 10 15
std::gcd(12, 18);  std::lcm(4, 6);                // 6, 12
std::midpoint(1, 4);                              // 2 (no overflow)
std::ranges::fold_left(v, 0, std::plus<>{});      // 15 (C++23)
```

CODE EXPLAINED

- → `accumulate(b, e, init)` — sum starting from `init`
- → `std::multiplies<>{}` — product instead of sum
- → `0.0` initial value — otherwise doubles are summed as `int`
- → `reduce` with `par` — may run on multiple threads
- → `transform_reduce(a, b, init)` — sum of pairwise products
- → `iota` / `partial_sum` — sequences and running totals

ALGORITHMS

ALGORITHM	RESULT
accumulate	left fold
reduce	unordered fold (parallelizable)
transform_reduce	map + fold
inner_product	dot product (classic)
partial_sum / inclusive_scan	running totals
exclusive_scan	totals excluding current
adjacent_difference	neighbor differences

EXECUTION POLICIES

POLICY	MEANING
seq	sequential
par	parallel threads
par_unseq	parallel + vectorized
unseq	vectorized only (C++20)

⚠ COMMON MISTAKES

- `std::accumulate(d.begin(), d.end(), 0)` on doubles truncates every partial sum to `int`
- Using `reduce` with a non-associative operation like subtraction; results vary between runs
- Expecting parallel policies to speed up small ranges; thread overhead dominates

✓ TIPS & BEST PRACTICES

- Write the initial value with the type you want: `0LL`, `0.0`, `std::string{}`
- On GCC, parallel policies need TBB (`-ltbb`); MSVC has them built in
- Use `std::iota` + `std::ranges::shuffle` to build random permutations

QUICK SUMMARY

- `accumulate` — sequential fold with init
- `reduce` — parallelizable fold
- `transform_reduce` — map + reduce, e.g. dot product
- `iota / partial_sum` — sequences and running totals

Ranges — Basics

RANGES LIBRARY

C++20 **ranges** treat a whole container as one argument: `std::ranges::sort(v)` instead of `std::sort(v.begin(), v.end())`. Range algorithms are constrained with **concepts**, so misuse gives clear errors, and they accept **projections** to operate on a member. Algorithms that return iterators into a temporary return `std::ranges::dangling` instead, preventing use-after-free at compile time.

NOTES

- Headers: `<algorithm>` (range algorithms), `<ranges>` (views, concepts)
- A **range** is anything with `begin()` and `end()`
- **Views** are lazy, non-owning ranges; containers own their elements
- Range algorithms are function objects (niebloids): no ADL surprises

RANGE ALGORITHMS & PROJECTIONS

```cpp
#include <algorithm>
#include <ranges>

std::vector<int> v{4, 1, 3};
std::ranges::sort(v);                          // 1 3 4
std::ranges::reverse(v);                       // 4 3 1

struct User { std::string name; int age; };
std::vector<User> users{{"Bo", 31}, {"Ada", 36}};
std::ranges::sort(users, {}, &User::name);     // by name (projection)
auto it = std::ranges::find(users, 36, &User::age);   // Ada
auto top = std::ranges::max(users, {}, &User::age);   // Ada (copy)

auto bad = std::ranges::find(getUsers(), 36, &User::age);
// bad is std::ranges::dangling — can't be dereferenced (compile error)

static_assert(std::ranges::random_access_range<std::vector<int>>);
static_assert(!std::ranges::random_access_range<std::list<int>>);
```

CODE EXPLAINED

- → `ranges::sort(v)` — whole container, no iterator pair
- → `{}` — default comparator (`std::ranges::less`)
- → `&User::name` — projection applied before comparing
- → `getUsers()` returns a temporary — result becomes `dangling`
- → range concepts — check capabilities at compile time

RANGE CONCEPTS

CONCEPT	REQUIRES
range	begin / end
sized_range	O(1) size()
forward_range	multi-pass
bidirectional_range	-- on iterators
random_access_range	+n, []
contiguous_range	contiguous memory
view	cheap to copy/move

VIEWS VS CONTAINERS

ASPECT	CONTAINER	VIEW
owns elements	yes	no
evaluation	eager	lazy
copy cost	O(n)	O(1)
example	vector, map	filter_view, span
materialize	—	std::ranges::to (C++23)

⚠ COMMON MISTAKES

- Mixing `std::sort` (needs iterators) and range syntax: `std::sort(v)` doesn't compile; use `std::ranges::sort`
- Keeping a view after its underlying container is destroyed
- Assuming every view is cheap to iterate twice; some (like `filter`) cache and some recompute

✓ TIPS & BEST PRACTICES

- Default to `std::ranges::` algorithms in C++20+ code
- Use projections instead of writing comparator lambdas for single-member keys
- Check concept satisfaction with `static_assert` when writing generic code

QUICK SUMMARY

- `std::ranges::alg(container)` — whole-range algorithms
- projections — operate on members
- `dangling` — compile-time safety for temporaries
- range concepts — describe iterator capabilities

Range Views & Adaptors

LAZY PIPELINES

Range adaptors in `std::views` build lazy **views** that transform, filter, slice, or combine ranges without copying. Compose them with the pipe operator `|` to form readable pipelines; nothing runs until you iterate. C++23 added `views::zip`, `enumerate`, `chunk`, `slide`, `join_with`, and `std::ranges::to`, which **materializes** a view into a real container.

NOTES

- Views evaluate on demand, element by element
- `views::iota(a)` is infinite; bound it with `take`
- Views reference the source: keep it alive while iterating
- `std::ranges::to<std::vector>()` builds a container (C++23)

PIPELINES

```cpp
#include <ranges>
namespace vw = std::views;

std::vector<int> nums{1, 2, 3, 4, 5, 6, 7, 8};
auto evensSquared = nums
    | vw::filter([](int x) { return x % 2 == 0; })
    | vw::transform([](int x) { return x * x; });    // lazy: 4 16 36 64
for (int x : evensSquared) std::print("{} ", x);

auto firstFive = vw::iota(1) | vw::take(5);          // 1 2 3 4 5
auto v = nums | vw::drop(2) | vw::reverse
              | std::ranges::to<std::vector>();      // 8 7 6 5 4 3
std::vector<std::string> names{"Ada", "Bo"};
for (auto [n, a] : vw::zip(names, std::vector{36, 31}))   // C++23
    std::println("{} {}", n, a);
for (auto ch : nums | vw::chunk(3)) {}               // [1 2 3] [4 5 6] [7 8]
auto csv = names | vw::join_with(',')
                 | std::ranges::to<std::string>();   // "Ada,Bo"
```

CODE EXPLAINED

- → `|` — feeds a range into an adaptor
- → `filter` then `transform` — nothing computed until the loop
- → `iota(1) | take(5)` — bounded infinite sequence
- → `ranges::to<std::vector>()` — materialize into a container
- → `zip` — iterate two ranges in lockstep
- → `chunk(3)` / `join_with(',')` — group and flatten

COMMON ADAPTORS

VIEW	PRODUCES
`filter(pred)`	matching elements
`transform(f)`	f(x) for each x
`take(n) / drop(n)`	first n / skip n
`take_while / drop_while`	by predicate
`reverse`	back to front
`keys / values / elements<N>`	tuple members
`split(d) / join`	split / flatten

C++23 ADDITIONS

VIEW	PRODUCES
`zip(a, b)`	tuples from each range
`enumerate`	(index, element)
`chunk(n)`	groups of n
`slide(n)`	sliding windows
`adjacent<N>`	N neighbors as tuple
`join_with(d)`	join with delimiter
`cartesian_product`	all combinations

⚠ COMMON MISTAKES

- Returning a view over a local container from a function; it dangles
- Modifying the underlying container while a view (especially `filter`) is in use
- Expecting a view to store results; it recomputes them each pass. Materialize with `ranges::to` when needed

✓ TIPS & BEST PRACTICES

- Alias the namespace (`namespace vw = std::views;`) to keep pipelines short
- Put each adaptor on its own line for readability
- Check library support for C++23 views (`zip`, `chunk`, `to`): GCC 13–14+, Clang/libc++ 17+, MSVC 17.4+

QUICK SUMMARY

- `std::views::*` — lazy range adaptors
- `|` — compose pipelines
- `ranges::to` — materialize (C++23)
- `zip / enumerate / chunk` — C++23 additions